

GRIP-TK

A NODE-GRAPH TOOLKIT FOR IMMERSIVE AUTHORING AND EVALUATION OF XR INTERACTION TECHNIQUES

by

Eduard Munteanu
Aarhus University
Aarhus, Denmark
au705518 / 202108177

Enok Malik Maj
Aarhus University
Aarhus, Denmark
au667066 / 202005732

Péter Gergely
Aarhus University
Aarhus, Denmark
au777945 / 202402636

Supervisor:

Ken Pfeuffer

Co-supervisor:

Mathias Lystbæk



Department of Computer Science
Aarhus University
Denmark

15th June, 2026

Abstract

When effectuating a XR evaluation design, developers often need to construct similar experimental basegrounds across different studies. For example, one developer might want to compare several interaction techniques in a docking experiment, while another might want to conduct a Fitts Law selection task with a different set of techniques. Even though the concrete task changes, these experiments still share a common core structure: building up the interaction technique, linking it to Meta XR prerequisites such as the Camera Rig, and connecting it to an environment that reacts to the state of the interaction itself. This repeated setup work is the baseground that developers often have to recreate and extend repeatedly, alongside the logging and analysis structures needed to evaluate the resulting interaction techniques.

This is the overall conceptual procedure that the GRIP-TK toolkit aims to support. The thesis presents GRIP-TK as a node-based framework for composing XR interaction techniques together with their experiment logic, logging, and visualization flow. To demonstrate its mechanism and extensibility, the toolkit is presented through a general walkthrough, showcasing how one of the interaction techniques can be constructed in the system and later extended by defining a parallax-mitigated node. Regarding the general walkthrough, it is used by us as a demonstration of how the toolkit can construct one of the interaction techniques that is then used in the Fitts Law study.

GRIP-TK is then further demonstrated in a more academic-style experiment, where we conducted a controlled Fitts' Law study with five interaction techniques: Gaze and Pinch, Gaze and Finger, Gaze and Finger Parallax Mitigated, HeadCrusher, and HeadCrusher Parallax Mitigated. This shows that the toolkit can support an evaluation design not only by supporting the experiment runtime itself, but also by connecting the interaction techniques to the data collection and analysis pipeline required to evaluate them. Through this, the toolkit supports not only the construction of the study environment, but also the broader evaluation workflow needed to investigate the research question in a concrete empirical setting. More concretely, the resulting evaluation showed that the parallax-mitigated variants did not improve equally: Gaze and Finger benefited substantially more from the mitigation than HeadCrusher, with clearer gains across measures such as Throughput, Error Rate, and Hand Movement. In this way, the toolkit did not only support the implementation of the experiment, but also the full procedure needed to reach a concrete conclusion about the compared interaction techniques, while still leaving the formal usability and productivity of the toolkit itself to future evaluation.

Acknowledgements

We want to extend our thanks to our knowledgeable supervisor Ken Pfeuffer and ever-resourceful Mathias Lystbæk, without whose help and openness this work would have been unrecognizably different. Thanks to Lauren Zerbin for providing code and assistance for the parallax mitigation as well as feedback in our Pilot Study.

Declaration of use of Generative AI

We used generative AI tools in preparing this thesis.

Tools used

- **Anthropic Claude 4.5 Haiku:** LLM accessed via pi coding agent and Claude Code CLI.
- **Google NotebookLM:** research assistant with 78-source notebook (Fitts' Law, ISO 9241-9, NASA TLX, ART/ANOVA, VR selection).
- **GitHub Copilot:** VS Code inline completion for C# and Python.

How GAI was used

Thesis text. Claude generated initial drafts of several chapters (results, discussion, conclusion). Output was iteratively either removed, refined, edited for accuracy and voice, and cross-checked against primary sources. AI text was subsequently humanized and verified against the data.

Fact-checking. NotebookLM exclusively for verification: factual claims checked against 78 sources, statistical conventions confirmed, results cross-referenced against benchmarks. NotebookLM generated no thesis text. All facts traced to original papers and cited directly.

Statistical analysis. Claude assisted with ART procedure, RM-ANOVA, post-hoc comparisons, and effect size computation. Code validated against collected experimental data.

Code: Unity C#. Claude and Copilot assisted primarily with testing and bug-fixing: unit tests, runtime diagnostics, edge cases in selection/logging logic. Also assisted with node-graph architecture, logging infrastructure, and selection implementations. Core experiment logic was human-designed and validated through AI-assisted debugging.

Code: Python pipeline. Claude assisted with analyzing Fitts' Law mathematics and generating additional graphs beyond the core comparisons, working from technical prompts that specified the variables, statistics, and axis conventions. The initial data-handling script and core comparison design were human-written. All output was validated by running the full pipeline and cross-checking against the raw experimental data.

LaTeX formatting. Claude assisted with LaTeX document formatting, table construction, figure integration, and layout adjustments.

List of potential literature for related works. Claude and ChatGPT provided some potential forward and backward references, making the basis of the literal review part - i.e. linking papers. Also did some citations.

Reflection

GAI tools accelerated research and writing substantially, particularly in testing, debugging, and statistics. All AI output was critically reviewed: text edited, code tested and debugged, statistics validated against raw data, factual claims verified via NotebookLM and primary sources. Intellectual responsibility for all claims, analyses, and conclusions remains our own.

Declaration of Authorship

Enok Malik Maj

I hereby declare that this thesis, submitted for the degree of Master of Science in Computer Science, is my own work and has not been submitted for any other degree or award at this or any other institution. All sources used are acknowledged as references.

My contributions to this report are as follows:

- Related works (Chapter 2, §2.1): literature review, comparative analysis of XR interaction techniques and prototyping frameworks.
- Background sections on the Heisenberg Effect and the basic study setup.
- Introductory text from Introduction about the abridged history of interaction techniques
- Co-authorship of the introduction; supporting in GRIP-TK section and joint review and editing of the manuscript.
- Heisenberg analysis in the discussion
- Parallax theory and coauthorship for other technique issues
- Initial Evaluation section without data processing and logging (edited by all after)
- High level coding contributions: experiment outside of logging and interaction technique setup; front end of toolkit

Péter Gergely

I hereby declare that this thesis, submitted for the degree of Master of Science in Computer Science, is my own work and has not been submitted for any other degree or award at this or any other institution. All sources used are acknowledged as references.

My contributions to this report are as follows.

Péter was the main author of the Fitts' Law theory and experimental design aspect driving the quantitative evaluation. He handled the data visualisation and (numerical) analysis, and coauthored the discussion (Chapter 6).

Eduard Munteanu

I hereby declare that this thesis, submitted for the degree of Master of Science in Computer Science, is my own work and has not been submitted for any other degree or award at this or any other institution. All sources used are acknowledged as references.

My contributions to this report are as follows:

- Theory section of the interaction techniques used in this thesis, including the background of the modular interaction technique architecture.
- Describing the the toolkit architecture, including the concrete implementation for experiment-based interaction technique analysis, extensibility points, menu-based visualization, and contributions to the overall node-graph visual representation.
- Evaluation Design reasoning section
- Evaluation Design interaction techniques definition.
- Qualitative data analysis.
- Future work and limitation writting
- Coauthored the Introduction, Abstract and the Conclusion section

If not mentioned, the contribution is to be understood as having been shared amongst the authors.

Contents

Abstract	ii
Acknowledgements	iii
Declaration of use of Generative AI	iv
Declaration of Authorship	vi
List of Figures	xi
List of Tables	xii
1 Introduction	1
1.1 Motivation	2
1.2 Objectives and Methodology	2
1.2.1 Concrete objectives	3
1.3 Research Questions	4
1.4 Contribution Statement	4
2 Background	6
2.1 Related Work	6
2.1.1 Early XR Authoring Tools	6
2.1.2 Classifying XR Design Support	7
2.1.3 Visual and Demonstrational Programming Approaches	7
2.1.4 Systems Closest to GRIP-TK	8
2.1.5 Evaluation of Toolkits	8
2.1.6 Summary of Positioning	8
2.2 Theory	9
2.2.1 Relevant Interaction Techniques	9
2.2.2 Toolkit Evaluation	11
2.2.3 The Heisenberg Effect	11
2.2.4 Common Usability Issues	12
2.2.5 In-Depth Fitts' Law Explanation	15
2.2.6 ANOVA	18
3 GRIP-TK	20
3.1 Motivation	20
3.2 Toolkit Implementation	21
3.2.1 Node Graph Concept	21
3.2.2 Backend	24

3.2.3	Frontend	31
3.2.4	Technical Performance	33
3.3	User interaction	33
3.4	Extensibility: Toolkit as a framework	34
3.4.1	Node-internal extensibility	35
3.4.2	Structural extensibility	35
3.4.3	Macro-level extensibility	36
3.4.4	Environment extensibility	36
3.4.5	Effect extensibility	37
3.4.6	Frontend and plumbing support	37
3.4.7	Exporting Technique Feature	37
3.5	Walkthrough	38
3.5.1	GRIP-TK pre-runtime configuration	38
3.5.2	Loading and inspecting the HeadCrusher technique	39
3.5.3	Adding parallax-related behaviour	40
3.5.4	Exporting the HeadCrusher technique for a different use case	41
4	Evaluation	43
4.1	Motivation	43
4.2	Study Design	43
4.2.1	Task Design	43
4.2.2	Apparatus and System Implementation	45
4.2.3	Interaction Technique Implementations	46
4.2.4	Data Collection and Metrics	48
4.2.5	Evaluation Metrics	49
4.2.6	Procedure	49
4.3	Data Processing	51
4.3.1	Effective Throughput per ISO 9241-9	51
4.3.2	Statistical Tests	51
4.3.3	Validation	52
4.3.4	Analysis Pipeline	52
4.4	Study Results	53
4.4.1	Participants	53
4.4.2	Statistical Methods	54
4.4.3	Objective Measures	54
4.4.4	Subjective Measures	60
5	Overall Discussion	62
5.1	RQ1: Generalizing XR Experiment Patterns with a Node-Graph Toolkit	62
5.1.1	What Patterns Were Generalised	62
5.1.2	How the Node-Graph Captures These Patterns	63
5.1.3	Trade-offs and Remaining Challenges	63
5.2	RQ2: Parallax Mitigation Impact on Plane-Based Techniques	64
5.2.1	Quantified Effect on Performance	64
5.2.2	Gap to Gaze&Pinch Baseline	65
5.2.3	Speed–Accuracy Tradeoff	65
5.2.4	Interaction Effects	66
5.3	Comparison with Wagner et al. 2023	66
5.4	Limitations and Future Considerations	68

CONTENTS

5.4.1	Toolkit Limitations	68
5.4.2	Fitts' Law Evaluation Limitations	69
6	Conclusion	70
6.1	Research Questions Restated	70
6.2	Summary of Findings	70
6.3	Contributions	71
6.4	Practical Implications	72
6.5	Broader Implications	72
6.6	Future Work	73
6.7	Closing Reflection	74
A	Questionnaires	75
A.1	Preliminary Questionnaire	75
A.1.1	Demographics	75
A.1.2	Pre-Session Sickness Baseline	75
A.2	NASA Task Load Index (NASA-TLX)	76
A.3	Technique Rankings	76
A.3.1	Forced-Choice Preference Ranking	76
A.3.2	Post-Session Sickness	77
A.4	Administration Order	77
A.5	Data Completeness	77
B	Technique Ordinal Position Analysis	78
	References	81

List of Figures

1.1	GRIP-TK supports the full immersive workflow, from authoring interaction graphs to running custom experiments and exporting techniques.	1
2.1	The finger image in the non-dominant view is shifted by the projected correction Δx , causing the two perceived finger images to align.	14
3.1	UML diagram of the Backend	24
3.2	The UI of the RayPointerMacroNode in a flattened view.	27
3.3	The UI of the HoverMacroNode in a flattened view.	27
3.4	The UI of the TargetCommitMacroNode in a flattened view.	28
3.5	The UI of the TriggerMacroNode in a flattened view.	28
3.6	The UI of the SelectionMacroNode in a flattened view.	29
3.7	The UI of the DraggingMacroNode in a flattened view.	29
3.8	The UI of the DisplayMode node in a flattened view.	41
4.1	Gaze and Pinch	46
4.2	Gaze and Finger	47
4.3	HeadCrusher	47
4.4	Task completion time by technique, shown per depth (1 m, 3 m), per amplitude (10°, 30°), and overall. Bars show means; whiskers are 95% confidence intervals. Significance: * $p < .05$, ** $p < .01$, *** $p < .001$.	55
4.5	Effective throughput (ISO 9241-9) by technique, per depth, per amplitude, and overall.	55
4.6	Effective target width $W_e = 4.133 \times \sigma_{\text{along}}$ by technique.	57
4.7	Error rate by technique, per depth, per amplitude, and overall.	57
4.8	Mean per-trial index-fingertip displacement by technique.	58
4.9	Fitts' Law regression at condition-mean level. $MT = a + b \cdot ID_e$.	59
B.1	Effective throughput (ISO 9241-9) by technique ordinal position. Each panel shows one technique's mean throughput across participants who experienced that technique at each ordinal position (1st–5th). Error bands show ± 1 SEM.	78

List of Tables

4.1	Outlier filter removal rates per technique.	51
4.2	Interaction parameters across versions.	51
4.3	Participant demographics ($n = 20$).	53
4.4	ART RM-ANOVA results for all dependent variables.	54
4.5	Interaction effects from two-way RM-ANOVA on effective throughput.	56
4.6	Parallax mitigation gain by technique family. $\Delta TP_E = \text{mitigated} - \text{vanilla}$	56
4.7	Fitts' Law regression parameters ($MT = a + b \cdot ID_e$).	59
4.8	NASA-TLX subscale medians by technique (1–7 scale, 7 = high). RTLX = mean of per-participant unweighted subscale means.	60
4.9	Friedman test results for NASA-TLX subscales and RTLX ($n = 20$, $df = 4$). Kendall's W for effect size.	60
4.10	Participant preference rankings (1 = best, 5 = worst).	61
5.1	Parallax mitigation gains. $\Delta = \text{mitigated} - \text{vanilla}$	64
5.2	Throughput comparison and confounds with Wagner et al. (2023).	67
5.3	Multi-metric results comparison with Wagner et al. (2023).	68
B.1	Effective throughput by technique and ordinal position.	79
	Further guidance on master thesis writing:	

Chapter 1

Introduction

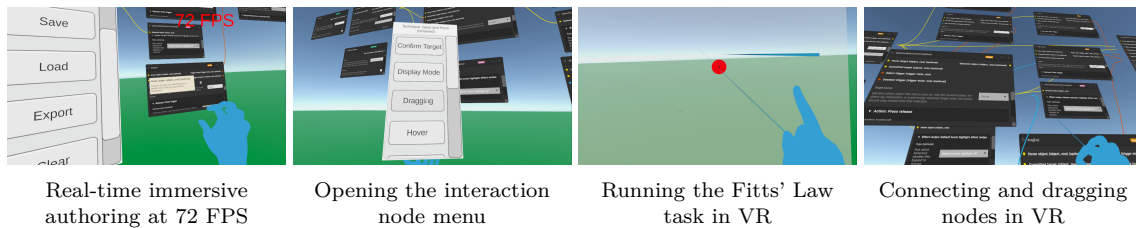


Figure 1.1: GRIP-TK supports the full immersive workflow, from authoring interaction graphs to running custom experiments and exporting techniques.

Interaction techniques act as the bridge between hardware inputs and software. It is therefore important that the techniques (ITs) work not just functionally, but seamlessly. A good interaction technique allows you to embody the system you are using, leaving only work intention, while a poor technique results in frequent breakdowns, making you conscious of the interaction operation (Bertelsen and Bødker [1]).

Making these coveted techniques is non-trivial. Particularly for new systems - and increasingly so - as the systems appear to be getting more and more expressive as time goes on (Grudin [2]). From command-line interfaces, using primitive language to commandeer the computer, to the advent of the trackball[3]. This technological evolution in hardware was later incorporated into graphical user interfaces (GUIs), shifting the paradigm. A ubiquitous keyboard is no longer the base requirement of interacting with the machines, marking great improvements in usability. The IT's design, however, was far from optimized, and it wouldn't be until 1968 and the "mother of all demos"[4] that the now standard way of interacting with computers: computer mice were shown to the public. This trajectory is still observed in the present day. Google recently announced a mouse pointer fit for the 21st century, with the Magic Pointer[5], inferring intent from the context of its use through artificial intelligence. Whether it will supersede the enduring standard pointer is yet to be seen.

The dust appears to have settled in the realm of extended reality with it having its own "mouse pointer moment", with Gaze&Pinch[6], and it growing in use outside of academia with its default use in the Apple Vision Pro[7]. But for anyone to get to the point of being adopted by a design giant, or creating an ubiquitous interaction device; making it so seamless is a process of iterative improvement, exploration, prototyping, user studies and more. A process of reimplementations of boilerplate code, debugging error introduction in those reimplementations and being bogged

down in time by cyclical compiling, switching back and forth between reality and VR, and optimizing the parameters of the ITs. For these reasons, we have decided to underpin the development of the current day genus of the interface - the XR interface, by creating GRIP-TK: the Graph Based Interaction Prototyping Toolkit. GRIP-TK is based on the paradigm of immersive authoring (Or What You Experience is What You Get - WYXIWYG) as coined by Lee et al. [8]: refining and evaluating the to-be artefact within immersive, extended reality. This paradigm, as was discussed in Zhang and Oney [9] and Schultheis et al. [10] has a its case specific dominances (such as a more fluid workflow) over non-immersive interfaces - is combined with visual programming through node graphs. These concepts working together to simplify the writing phase and lowering the barrier to entry - a persistent hurdle in extended reality.

We evaluate the usefulness of GRIP-TK through a walkthrough user study (n=20, 8 female) and find even with parallax correction on compared-to techniques, Gaze&Pinch reigns supreme in terms of throughput, movement time and workload, as well as being the preferred method of 12 of the 20 participants. Parallax mitigation asymmetrically helped the image plane techniques, with Gaze&Finger gaining far more from the aid than HeadCrusher. In terms of toolkit utility, GRIP-TK successfully supported the implementation and data analysis of a Fitts' Law experiment, supporting its claims of being a capable tool for researchers.

1.1 Motivation

As a scientific tool, the motivation behind the research is to showcase that the current process of creating individual techniques, creating a study and setting to test them, defining the flow of the experiment, collecting runtime data, and doing statistical analysis with the experiment results can all be fused into one emancipating solution. A solution for which "the whole is greater than the sum of its parts" rings particularly true for in this case. This proof of concept (despite *not* being the first of its kind; see 2.1) is intended to channel the powers of immersive authoring to support and expedite human computer interaction researchers' efforts in going from research question to results with less work.

GRIP-TK further allows for extension and customization of its own experimentative and auctorial features through custom nodes, runtime scenes and general hardware (whichever Unity supports), to further cut down on the time used to design and deploy seamless, scientifically supported ITs.

1.2 Objectives and Methodology

We will showcase a subset of these capabilities (composing and using the technique in a study) using a toolkit demonstration as described in Ledo et al. [11], involving an investigation into 5 interaction techniques and their response to parallax mitigation (see: 2.2.4), namely Gaze&Pinch (baseline), Gaze&Finger, and HeadCrusher (last two having a mitigated variant for comparison). The study is directly based on Fitts' Law with a standard NASA-TLX questionnaire (with the aside of an eye strain question) compared with Friedman tests across the 5 techniques, followed

by Conover comparisons with Holm-Bonferroni correction; Kendall’s W as effect size. Finally, preference, discomfort and open-ended questions were further used for discussion. The argument for using this protocol is that it is a widely documented model, and that it was used in Wagner et al. [12] - a parallax focused interaction technique study that we wished to build on.

The 5 interaction techniques are based on their work.

This exemplifies the toolkit’s abilities through a case study while adding to our contribution by analysing how impactful visual, direct parallax mitigation is on image plane interaction techniques. The parallax mitigation itself is a reimplementa-tion from Zerbin [13] supported by shader code and guidance from the author.

The study lastly is motivated by quantifying the discrepancy between the state of the art technique, Gaze and Pinch, and both the default and the parallax mitigated image plane based interactions, showing how much the parallax mitigation helped move the interaction techniques towards this current optimum.

Altogether, this toolkit demonstration illustrates how GRIP-TK can be used to underpin a concrete and *actual* experiment, and how such studies can be generalized and streamlined using the toolkit.

1.2.1 Concrete objectives

The primary objective of the thesis is to achieve generalizability for the procedure of experiment design in the XR medium, specifically with respect to ITs. This includes supporting common patterns of IT behaviour, e.g., when interacting with the target: creating the effect of hovering over it, and the action triggered when applying it. Altogether, all of these IT-aspects are used to interact with the environment, which produces some follow-up state. The methodology for reaching this objective is through documenting the sections in which general experiments can be analysed, and afterwards to show - for validity - an example (in our case the Fitts’ Law experiment) with the 5 interaction techniques.

The secondary objective is that we have concerns the user experience with the toolkit itself. Particularly, keeping the interface usable and maintaining a good flow between the toolkit medium and the environment medium, where the user will carry out the toolkit implementation. The methodology behind this objective is to design for avoiding breaks of the medium, between the Toolkit UI and the experiment environment. This means that the user predominantly remains in runtime, seamlessly changing focus between toolkit editing and testing. We further require that the script generalizes into a simple but thoroughly documented paradigm, like creating a visual-based language for each component, or visual programming. This supports lesser experienced users in being able to use the toolkit.

Regarding advanced users, we should allow the possibility of the toolkit being modified through code - giving users the possibility of extending the toolkit’s features and capabilities. This is accomplished through self-defined interfaces for the nodes, well-defined responsibilities, and generally by making the toolkit coexist as a framework with self-explanatory open edges for further scaling or modifications or extensions.

In summary, there are two objectives for the thesis. First, to design and im-

plement a modular node-graph toolkit, GRIP-TK, for composing and evaluating interaction techniques in immersive virtual environments. Second, to conduct a controlled user study comparing parallax-mitigated image-plane selection techniques against their unmitigated counterparts and a non-parallax control; using the ISO 9241-9 Fitts' Law evaluation methodology. The methodology combines Unity-based VR experiment development, high-frequency rich data logging, a Python analysis pipeline for ISO-compliant effective metric computation, and statistical evaluation following current best practices in 3D XR Fitts' Law research.

1.3 Research Questions

Our thesis is mainly split into two sections: the research question regarding the overall toolkit, as well as the research question regarding the concrete experiment in which we showcase the usability of our toolkit.

To start off, the research question of our toolkit can be formulated as:

RQ₁ *"To what extent can a reusable node-based toolkit support the construction, modification, and execution of multiple XR interaction techniques within the same experimental framework?"*

This research question gives us a clear and understandable dilemma, but at the same time, no objectively correct answer can be given. It is instead about analysing a common pattern that we can observe, trying to interpret the concrete implementation and further refine them with our own, supported perspectives.

Since our project uses a concrete thesis-like experiment as a demonstration and answer to the question above, the experiment itself deserves its own research questions. The research question for the experimental example can be formulated through 2 research questions:

RQ₂ *"How do the parallax-mitigated selection techniques fare against a technique that does not suffer from double vision?"*

RQ₃ *"How do the parallax-mitigated techniques fare against their vanilla counterparts?"*

These research questions let us analyse the problem behind the parallax effect on some of the most popular image plane techniques, Gaze&Finger and HeadCrusher, while also allowing us to identify the improvement gained by specifically by applying the mitigation and finally discussing whether it can compete with their standard implementations.

1.4 Contribution Statement

Such generalization frameworks for an interaction medium have already been discussed in diverse HCI-related papers (e.g., FlowMatic [9]), especially regarding the visual programming philosophy using, e.g., graphs composed of nodes and edges in order to provide the user with an easier language, as well as also discussing potential interaction mediums with the toolkit for the user, but there are a couple

of conceptual gaps that are not addressed and that we have addressed in our own toolkit. One of the gaps points is the seamless medium of changing between the environmental play mode and the editing medium of the interaction medium. More concretely, our contribution would be to design a system which lowers the risk of work discontinuity by allowing runtime switching and treating a specific interaction medium, split up into the environmental nodes, interaction nodes, and data-driven nodes, as a specific file, and treating all of those different media as editable files, similar to how we interact with files in other situations, like on a PC, in order to give the user a common point of familiarity.

Another contribution is to focus the concept of the toolkit more as a classic VR experiment generalization tool regarding the definition of an interaction technique, such as Gaze and Pinch, having redefined the nodes and their connections to facilitate this specific generalization instead, as well as also enabling multiple mediums of extensibility, from more invasive to less invasive mediums, but in both cases making it trivial to understand the flow of the base nodes and the architectural system.

The toolkit is not only focused on defining the interaction technique itself, but also on connecting it to a concrete experiment medium, in our case a Fitts' Law environment, such that the interaction, the environment logic, and the experiment flow can all be treated as part of the same generalized system instead of being handled as separated parts.

Furthermore, the toolkit gives the possibility of the data collection and analysis perspective, where the toolkit is used not only to create and test the experiment, but also to support the logging of the relevant data and the later statistical analysis based on the collected results. In that sense, the contribution is also about treating the experiment pipeline more holistically, from creating the interaction medium, to testing it in the environment, to collecting the data, and finally to analysing the results.

Finally, another contribution is the use of a concrete experiment case in order to validate and show the toolkit in practice, more specifically by analysing the impact of parallax mitigation on the plane-based techniques and comparing them against a more optimal interaction medium. This gives the toolkit not only a technical contribution as a system itself, but also a practical contribution through the concrete findings that can be discussed from the experiment that was built with it.

To sum up our general contributions, we have (1) created a toolkit for general experimentation of interaction techniques and (2) applied its methodology to facilitate a Fitts' Law based study (n=20) exploring a parallax mitigation technique on a set of 2 image plane ITs, comparing it to Gaze&Pinch.

Chapter 2

Background

2.1 Related Work

To position the paper, we will begin by looking at some of the original and pioneering toolkit papers that were made for extended reality. We will then digress into different classifications of these toolkits, followed by some concrete and similar approaches to ours, and general toolkit evaluation. Finally, we list the works that most closely align with this one, and close with a summary of the positioning.

2.1.1 Early XR Authoring Tools

Creating useful tools for designers and non-programmers working with extended reality is something researchers have been chipping away at since Ivan Sutherland first presented his augmented reality demo dubbed the "Sword of Damocles" after the ancient moral anecdote about how being in a position of great power gives way to paralleling immediate great problems. While head mounted displays, like the Apple Vision Pro[7] and Meta Quest series (formerly Oculus Rift),[14] have broadly paved the way for incredible experiences to be had on these powerful and power hungry devices, the problems inherent with the medium have yet to be bested - though coming from far less lethal and scary origins than that of ancient myth.

The seminal paper DART by MacIntyre et al. 2004 sought to take one of the first jabs at this task, with the AR authoring application Designers Augmented Reality Toolkit (DART), built intentionally as a plug-in for Macromedia Director, the "definitive" authoring tool for multimedia content at the time; [15, 16] making it a greater fit than narrow research tools. Moreover, the tool provided an interface for high-level visual coding by means of the score, triggers, direct manipulation, and scripting if required for advanced applications. One of the enduring successes of the application is its ability to permit users to rapidly create low-fidelity prototypes[17] using sketches and 3D puppets, content that can later, after being scrutinised against other options, be further developed with the use of production-ready content. It also allowed the user to experience the AR scenes that they had made off-location, which cut down severely on resource demands during their own user study of student made projects. We adopt this mentality in the design of this toolkit, in that it has been built on top of one of the definitive AR development software packages, providing users with specialised tools within a high fidelity commercial product, and making it extensible at a low level for further conceptual growth when required. In respect

to allowing low level prototypes, it is less actualizable in the domain of ITs, but we used the design pattern of music writing software like FL Studio[18], where the score is preconfigured with basic instruments, meter and tempo, to quickly equip the user with the tools needed to start trying ideas out. In our toolkit, this is expressed through a simple, predefined interaction technique, which greets the user upon opening the application, for teaching and prototyping.

2.1.2 Classifying XR Design Support

At the end of the following decade after the publication of DART, Nebeling and Speicher released a position paper, in which they subdivide these creative tools into 5 discrete classes: 1. Basic Mobile Screens and Interactions, such as [19, 20]; 2. Basic AR/VR Focused Interaction as [21, 22]; 3. AR/VR Focused Interactions [23]; 4. 3D content [24, 25]; and finally 5. Advanced AR/VR Content & Interactive Behaviour [26, 27, 28]. They further postulate that while there are many tools (this being a challenge in itself of 3), not enough of them take up the space between and within the 5 classes, leaving designers to create "patchwork workflows" from these fragmented solutions. This partition was further explored in [29, 30] wherein the authors identified 30 artefacts from 39 resources, identifying common tools, trends, opportunities, and challenges faced by AR designers and developers. These artefacts, they categorised as Physical/Low-Fidelity; Techniques; Commercial/General Software, and Presentation/Video, and implying opportunities such as early stage quality control, using VR as a testbed and doing more research in the topic of low-fidelity prototyping. While our toolkit is capable of supporting continued development outside of GRIP-TK, we also strive to make prototyping as painless as possible. We further self organise the toolkit in the above taxonomies as an AR/VR Focused Interface or Research Prototypes/Frameworks situated within an Commercial/General Software/Advanced AR/VR Content & Interactive Behaviour class, namely Unity.[31]

2.1.3 Visual and Demonstrational Programming Approaches

To support these ideas, many toolkits resort to simplified paradigms of programming. That is, either Programming by Demonstration (PbD) [32, 33] or visual programming (e.g. "plugboarding" or data flow models; functional reactive programming [FRP]) [9, 24, 25, 34, 35]. GestureCanvas [34] allows designers to easily create Compound Freehand Interactions (CFIs) by demonstrating the gesture, editing the result afterwards, and finally exporting it to be used in outside-toolkit projects, in an immersive environment. Similarly, FlowMatic raises the ceiling of toolkits (addressing Nebeling et al's three toolkit challenge) by allowing you to program reactive systems with node graphs, effectively resembling the capabilities of game engines within a virtual environment. [9, 24, 25, 32, 33, 35] These visual programming approaches resemble ours with the major difference being the direct focus on interaction technique authorship. Keeping it to such a scope allows the system to remain fully documented and accessible to non-programmers. We implemented the export to external projects in our toolkit based on the first-mentioned paper.

2.1.4 Systems Closest to GRIP-TK

Some systems come very close to ours, such as [36, 37] where they have made systems to create non-WIMP[WIMP] interface interaction design through node graphs, as well as in the first paper’s case created a runtime place to test them. We therefore do not claim that we by any means are the first to create a tight loop between node graphs and testing them, but rather providing an immersive authoring [9, 21, 27, 28] tool for crafting interaction techniques that contains the entire development cycle of authoring, prototyping (low-fidelity to high-fidelity) and exporting the technique to be used simply outside of the system. This effectively lowers the barrier for non-programmers, designers and human computer interaction researchers to spend their time on making creative, new techniques over learning to program; code switching between 2D and 3D modes of thinking, and waiting for projects to compile simply to adjust one parameter, while testing a meaningful manner, as opposed to using the built in simulator.

2.1.5 Evaluation of Toolkits

Several toolkits relating to development for extended reality (XR) have been authored, as a prime goal of toolkits is to ease the difficulty of getting started with working and not having to write fresh boilerplate code every time. XR development adds at least an entire additional dimension of physical design space to take into account. AUIT [23, 38, pp. 1–2] explored the problem of making interfaces in and of themselves, particularly context-aware ones, that optimise for a series of goals such as occlusion and movement. They created a user study that consisted of a usage-type walkthrough [11, pp. 3–4], where users of varying degrees of expertise had to create 2 UIs with the toolkit.

After this, they would ask the participants about how easy it was to use and how it compared to their own work process. They saw favourable opinions regarding the new approach. We are not doing a usage evaluation and are opting for a demonstration instead. Specifically, we want to perform a combination of case study, replicated example and how-to scenario. This covers three bases: showcasing the breadth and depth of the toolkit, investigating the case of parallax mitigation on top of their work, and also showing a potential userbase how to use it. [39, 40]

2.1.6 Summary of Positioning

In summary, we are drawing inspiration from previous papers in how to best support our target demographic of designers who do not necessarily know coding, but provide a thought-out framework for extension, while further contributing domain-specific support and setting us apart by filling the cross-class gaps in AR and VR applications through immersive authoring, node-graph programming and integrated, empirical evaluation (i.e., Fitts’ Law), with detailed logging. We are going to evaluate the utility of this through Ledo et al.’s framework. Through this toolkit, we aim to contribute to overcoming the perils Sutherland unearthed with the invention of the yet tamed technology of augmented reality.

2.2 Theory

2.2.1 Relevant Interaction Techniques

Eye Tracking

Eye tracking refers to the hardware and software capability of estimating where the user is looking. In XR, this is commonly used for gaze-related interaction techniques, where the system uses the user’s eye direction as an input signal. This is relevant for techniques such as Gaze and Pinch, where looking at a target works as a pointing or hovering mechanism before the actual confirmation action is performed. In that sense, eye tracking is not only a sensing feature, but also an important building block for gaze-based interaction design.

In practice, head-worn eye tracking does not directly know what object the user intends to select. What it estimates is the direction of the eyes, from which a gaze ray can be derived and projected into the virtual scene. This makes it possible to infer which object the user is likely looking at. XR headsets that support this feature, such as the Meta Quest Pro, provide eye-tracking data at a certain sampling frequency, in our case 30 Hz. For interaction techniques, this matters because eye tracking is valued for being fast and natural, but its usefulness also depends on qualities such as accuracy, precision, and latency. These factors influence how stable and trustworthy the gaze ray is during interaction.

A limitation of eye tracking is that the tracked gaze direction is not perfectly identical to the user’s actual visual attention at every moment. Hardware-related issues such as latency, dropped samples, and estimation noise can lead to small frame-wise misalignments between the user’s real gaze direction and the state reported by the headset. This becomes more noticeable when the targets are small or when the interaction depends on precise gaze alignment.

Another important limitation is the dependence on calibration. Eye tracking accuracy can degrade when the headset is repositioned differently on the user’s face after calibration, for example when the user removes and puts the headset on again. Since the original calibration is tied to a previous physical fit, even a small shift can reduce gaze accuracy. In practice, this can often be improved by recalibration, but it still means that the quality of gaze-based interaction partly depends on stable headset placement and correct user setup. [41]

Gaze and Pinch

Based on [6, pp. 99–101], *Gaze + Pinch* is a multimodal VR interaction technique that combines gaze-based target selection with pinch-based confirmation. The key design idea is that gaze establishes the object of interest, while the pinch gesture acts as the trigger on the looked-at target. In other words, selection is guided by eye gaze, but the gesture determines when the action is committed. This allows the interaction to remain close to natural eye–hand coordination while extending it into virtual space, where the same gesture can be applied to targets at both near and far distances.

Pfeuffer et al. present *Gaze + Pinch* as more than a single selection primitive: it is a general multimodal interaction framework that supports several interaction variants, including menu interaction, 3D manipulation, scene navigation, and image

zooming. The technique is relevant not only as a confirmation mechanism, but as a broader model for combining gaze and freehand input in VR. A useful aspect of the paper for our work is that it establishes the separation between gaze as the pointing modality and pinch as the symbolic confirmation action, which is conceptually close to the gaze-plus-hand techniques used in our experiment. Later work on gaze-hand alignment also treats *Gaze&Pinch* as a baseline multimodal technique and reports that, in 3D selection tasks, it performs strongly when the target is farther from the hand, but is still affected by the interaction between gaze, hand movement, and depth [12, pp. 8–9].

HeadCrusher

From [42, pp. 39–41], *HeadCrusher* belongs to the family of image-plane interaction techniques for head-tracked immersive environments. The fundamental idea is to treat the user’s view as a 2D image plane, so that 3D objects are selected through their projected position rather than through direct physical contact in world space. In the original technique, the user positions the thumb and forefinger around the desired object in the image, and the system casts a pick ray from the location of the user’s eye through the midpoint between those two fingers. Selection is computed from the spatial relationship between the hand configuration and the image-plane projection of the object.

Pierce et al. introduce *HeadCrusher* as part of a broader set of image-plane techniques for selection, manipulation, and navigation. The technique matters because it shows that distant target selection can work from a hand frame interpreted in image space rather than world space. This makes it fundamentally different from direct reach-based selection: the user doesn’t physically touch the object, but instead frames it visually and lets the system infer the selection ray from the midpoint of the finger pair. The paper also notes that image-plane techniques may support orientation-based disambiguation when several candidate objects overlap, and it discusses how the selected object can be translated or scaled while preserving its image-plane appearance. This image-plane formulation is the conceptual basis for later HeadCrusher-style implementations and for our own use of the technique in the experiment.

Gaze and Finger

Based on [43, pp. 1–3], G&F is an instance of the broader *Gaze-Hand Alignment* principle. The paper defines this principle as a selection mechanism in which gaze provides pre-selection and the manual pointer completes the selection once both modalities are spatially aligned on the same target. In the case of *Gaze&Finger*, the manual pointer is the user’s raised index finger, and the technique is implemented as perspective-based pointing from the eye position through the finger into the scene. Selection is completed when the finger is aligned with the gazed-at target, so the interaction becomes a single eye–hand pointing action rather than two independent input steps.

Wagner et al. make the technique more concrete by formalising alignment in an object-independent way. Instead of requiring both pointers to hit the same object directly, they compute whether the gaze ray and finger ray intersect the target-depth plane within a small visual-angle threshold. This makes the technique applicable

to general 3D selection tasks, not only to menu-style object-based interfaces. The paper also introduces fixation filtering to reduce false alignments during saccades: the system keeps a gaze sample from roughly 150 ms earlier and updates the fixation gaze only if the angular difference to the current gaze remains below a threshold of about 4°. This is important because it separates stable gaze alignment from rapid eye movement and makes the selection trigger less sensitive to transient gaze noise.

Compared with *Gaze&Pinch*, *Gaze&Finger* is more spatially continuous: the finger itself acts as the selection pointer, rather than serving as a symbolic confirmation gesture. The Fitts' Law evaluation in [12, pp. 8–10] shows that this alignment-based technique is effective, but also that it is sensitive to target depth because parallax becomes stronger as the target moves further from the image plane. In particular, the technique performs best when the finger is already in the visual field and the target is close to the user's line of sight, while performance decreases as the spatial discrepancy between the hand pointer and the target increases. This makes *Gaze&Finger* a strong conceptual reference for our own implementation, since it formalises gaze and finger as a coordinated pointing strategy rather than as two separate inputs.

2.2.2 Toolkit Evaluation

Following the toolkit evaluation taxonomy exhibited in Ledo et al. [11, pp. 1–2], there are a few methods used to evaluate the *utility* of a toolkit - that is, the extent to which the toolkit is capable of supporting the path of least resistance for creating a breadth and depth of domain artefacts. For example, a usage-type walkthrough [11, pp. 3–4] is a literarily representative method for evaluating toolkits. It works as follows: A group of study participants of set degrees of expertise work with the researchers' toolkit to solve some task. After this, the participants evaluate, e.g., ease of use and how it compares to their own work process. This can be seen as a direct way of producing knowledge with respect to toolkit utility.

From the same taxonomy, they define the walkthrough demonstration as an widely used alternative/complimentary evaluation in which the authors promenade the readers through a toy example of building something with their tool. This taxonomy provides the basis for the evaluation of our own toolkit and paper form.

2.2.3 The Heisenberg Effect

The Heisenberg Effect[44, pp. 629–633] was initially observed in a publication for the HCI International proceedings regarding the complexity and lack of knowledge in designing interaction techniques and their associated technologies and applications. It concretely explores the case of the Pinch GloveTM: a set of sensor-imbued fabric gloves able to detect discrete gestural inputs. They set out to explore this technology's mapping of the aforementioned trifecta: application, technology and interaction technique. The way they did this was by developing a series of 3 applications with techniques that "pushed the envelope": they were more than the naïve solution of simply pinching for interaction. The results were favourable. People were able to learn the techniques without much ado, which was a key goal of theirs, and a big part of the reason for this success was the design of the glove.

By using discrete input, they were able to map "as many functions as can be remem-

bered" without worrying about the ambiguity of continuous input, which plagues whole-hand technologies such as hand tracking. Another aspect is that the user is able to use dual-handed interaction, which can separate the trigger from the aiming, for example in the hand ray, which is what defines the Heisenberg Effect. A conjoining of aiming and selection, as pressing a button while you are pointing at something. Think of a computer mouse. The force that you use to press the button ever so slightly manoeuvres the pointer away from where you clicked exactly. Wolf et al. [45, pp. 1–10] looked into this in the case of Spatial Interaction in Spatially Tracked Input Devices (STIDs; specifically the HTC VIVE), its characteristics and more ways to mitigate it other than splitting the 2 parts of the interaction. To look into it, they ran ISO 9241-9 (n=16, 7 female, 1 non-binary); a ballistic version and a stationary version, where the user had to stay on target for some time, before finally clicking, to remove any ballistically caused error. They found that 30.45% of overall error is caused by Heisenberg Effect, and importantly, that it could be reduced to 25.4% with a correction function, fitted on the data that they collected. Another manner in which they mitigated the effect was through so-called minimum viable Heisenberg Compensated target sizes. These also make use of the data by finding an acceptable threshold of e.g. 5% of target Heisenberg Errors and looking at the maximum Heisenberg Movement - how far from the starting point a click travels before completing (Vive's controller buttons smoothly slide from 0, not pressed, to 1, fully pressed) - and using that in this equation to determine the size of the targets.

$$\text{target size} = \text{distance_to_controller} \times \sin(1.7^\circ) \times 2$$

The problem with this method is that it assumes that the clicks start in the centre and that the new size will not impact the Heisenberg Movement or Error. Other means of mitigating the effect from the paper were, adjusting the time when the selection trigger occurs sometime before the button is at 1, e.g., 0.5. But this causes unacceptably more issues derived from, e.g., Type I errors (false positives). Similarly, you can retime the click to when the click started; but again many selections start outside of the target. The approach they recommend is using the function, in part due to the systematic upward and leftward movement occurring during clicking. They concur that this is feasibly due to the limitation of the study that they are using the Vive and not testing on other STIDs and controllers.

In discussing our demonstrative results in comparing basic HeadCrusher to the other two techniques (G&F, G&P), we will later revisit and support our interpretations relating to performance differences with this effect and its assisting strategies.

2.2.4 Common Usability Issues

There is no one-size-fits all for interaction techniques. Each one has its trade-offs that makes it better or worse depending on the context that it is used in. For example, the headray (a forward-facing ray protruding from between the eyes) technique element has heightened stability in comparison to using a handray, which is faster and doesn't give neck strain. The following sections go over some relevant issues and downsides in the IT space.

Midas Touch

Given [46, pp. 12–13], the *Midas Touch* problem refers to the difficulty of using gaze as an input modality without causing unintended activations. Jacob’s central observation is that eye movements are continuous and primarily used for perception, so a gaze-controlled interface cannot easily distinguish between ordinary visual scanning and an intentional command. This makes gaze powerful for pointing, but unreliable as a standalone selection mechanism, because simply looking at an object may be enough to trigger an action.

The importance of this problem is that it forces gaze-based interfaces to introduce an additional mechanism for intentionality. In practice, this has led to designs that combine gaze with dwell time, gestures, blinks, or other confirmation actions so that selection only occurs when the user clearly intends it. In that sense, the Midas Touch problem is not just a limitation of gaze input, but also one of the main reasons why modern gaze-based interaction techniques are multimodal rather than gaze-only.

Gorilla Arm

Gorilla Arm Effect[47, pp. 1063–1065] denotes the physiological fatigue derived from continually dispatching your arm for interaction, leaving it suspended in the air for longer than is comfortable. This eventually leads to sensations of having a very heavy arm, as your muscles tire out from the exercise. This is naturally an unwanted, but common effect of interaction techniques, as many interaction techniques (ITs) rely upon the arm and hand for aiming, directly - as is the case with handray - or indirectly - as is the case with HeadCrusher. By varying the distance between the targets, we may also observe physical movement costs such as the Gorilla Arm Effect, especially when selection becomes more demanding from continued use.

We took this into account when coming up with the user study protocol, providing the participant with prompts to take breaks in between study tasks as well as when discussing the physical demands of the tested techniques.

Parallax (Diplopia)

Binocular diplopia is a physiological phenomenon in which your brain fails to combine the images perceived by your left and right eyes, leading to both images being visible at the same time, often in a transparent way. In VR and in real life, this can be caused by slight overlap or crossing of your eyes from, e.g., fixating on something close to your eyes, like a finger. It is further caused by parallax, or the difference in view from one eye to the other from positional offset. Again, this disparity is more noticeable the closer the object of interest is to your eyes.

In image plane techniques (in VR), you rely on the relatively small parallax at certain greater distances, making your brain interpret the 2 images as 1, making them comparable to something flat like an image. But parallax can and does interfere with these techniques, as was demonstrated in Wagner et al. [12] and discussed in Pierce et al. [42], and hence with the performance in accurately accomplishing what you intend to do. Frustration ensues.

But the diplopia can be mitigated by correcting for the offset and direction of your eyes: taking the image (of, say, your finger) of your non-dominant eye making it overlap with the dominant eye’s image from the non-dominant eye’s perspective,

considering its translation and rotation - and parallax is now corrected. The downside of this correction is that it may feel strange - as if someone painted the image of the finger onto your eyeballs. Moreover, there is still one question needing answering: Where exactly should the non-dominant eye's finger image be drawn on its view?

In practice, the disparity grows with an inverse relationship between the depth of the parallax-affected object and the depth of the point of fixation [48][49], something that can be captured using eye tracking and the 3D position in scene of the first mentioned (e.g., the user's index finger) Zerbin [13]. We then multiply the inter-pupillary distance (IPD) by the difference in percentage badness: correction strength, giving us the offset, Δx . The position of the image in the non-dominant eye for making it align with the image in the dominant eye then becomes

$$\text{Non dominant image position} = \text{Non dominant object position} + \text{IPD} \left(1 - \frac{d_h}{d_c} \right)$$

where d_h is the distance to the mitigated object, and d_c is the distance to the convergence point of the lines of sight (provided it exists); from the dominant eye and centre of the head respectively¹. Figure 2.1 illustrates the overall process and variables.

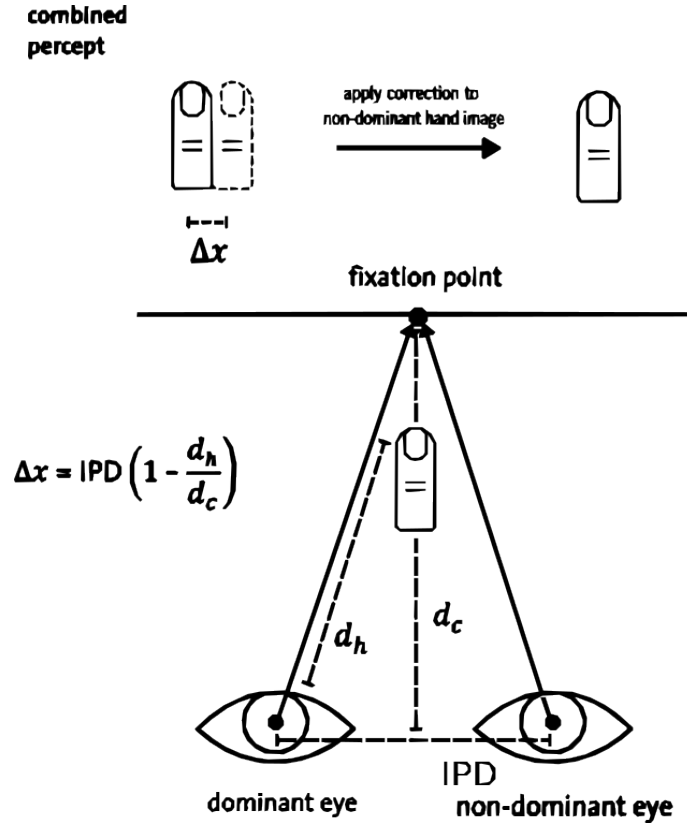


Figure 2.1: The finger image in the non-dominant view is shifted by the projected correction Δx , causing the two perceived finger images to align.

¹This is how our parallax correction is implemented in the demonstration study. Calculating the distance from the same origin reduces the parallax error further.

2.2.5 In-Depth Fitts' Law Explanation

Having listed some common problems, we will now move on to one of the environments in which the problems may be observed: Fitts' Law [50, pp. 381–391]. This law, having its theoretical basis in information theory[51], is a widespread way to test interaction techniques of anything from touch screen interaction to Gaze&Pinch. This has become a straightforward and standardized way to techniques against one another, through the processing of different variables such as throughput (basically, how fast and accurate the user is at doing the task) over a predefined set of task executions. The tasks themselves, consist of hitting a set of targets commonly arranged in a 2D circle by some means of selection.

The sections ahead go more into details about the thesis relevant equations, background and rationale behind variables.

Information-Theoretic Origins

Fitts' Law is rooted in an information-theoretic view of human motor performance, where target acquisition difficulty is expressed in bits and movement is treated as a form of information transmission [50, 52]. This perspective motivates the use of *throughput* as a combined measure of speed and accuracy:

Throughput is computed over a sequence of trials as the ratio of task difficulty to mean movement time:

$$TP = \frac{ID}{MT} \quad (2.1)$$

With ID in bits and MT in seconds, TP is expressed in bits per second (bps). In this way, throughput captures how efficiently an interaction technique supports target selection under a given task difficulty, which is why it remains one of the central comparative measures in Fitts' Law evaluations [52].

Formulations of the Index of Difficulty

Several refinements to the Index of Difficulty have been proposed since Fitts' original, each fixing problems with the formulation that came before it.

Original Formulation Fitts' original ID uses a ratio of $2A/W$, assuming the movement starts at the centre of one target and ends at the centre of another at distance A :

$$ID_{\text{original}} = \log_2\left(\frac{2A}{W}\right) \quad (2.2)$$

This gives $ID = 0$ when $A = W/2$, which means zero difficulty for movements shorter than the target width. That doesn't make much sense [53, p. 96].

Welford Refinement Welford fixed this by adding 0.5 to the ratio [54, pp. 147–148]:

$$ID_{\text{Welford}} = \log_2\left(\frac{A}{W} + 0.5\right) \quad (2.3)$$

This stops ID from going negative or hitting zero for very short movements. But the 0.5 constant doesn't come from information theory.

Shannon Formulation MacKenzie proposed the (as of now) standard Shannon formulation, which mirrors Shannon's original Theorem 17 directly [53, pp. 96–97]:

$$ID_{\text{Shannon}} = \log_2 \left(\frac{A}{W} + 1 \right) \quad (2.4)$$

This has three things going for it: ID is always ≥ 0 ; it fits empirical data slightly better than Welford's version; and it keeps a clean link to information theory [53, pp. 97–98]. The Shannon formulation was adopted by ISO 9241-9 for evaluating non-keyboard input devices [55, §7], then by the updated ISO 9241-411 standard [56, §7]. We use it throughout this thesis.

Movement Time Model Fitts' Law relies on linear regression to predict movement time (MT) based on the index of difficulty (ID).

$$MT = a + b \cdot ID \quad (2.5)$$

Researchers must establish the intercept (a) and slope (b) through direct experimentation. The slope is critical. Dividing 1 by this slope ($1/b$) provides the throughput, which measures the information processing bandwidth of the interaction technique [57, p. 607]. A lower slope means higher transmission efficiency.

Effective Metrics and the ISO 9241-9 Standard

Nominal Fitts' Law metrics only reflect what the user was *asked* to do, not what they *actually* did. ISO 9241-9 fixes this with effective metrics that adjust for real user behaviour [55, §7].

Effective Width (W_e) Effective width captures the accuracy side of the speed-accuracy trade-off. Instead of the nominal target diameter, W_e is derived from the spread of the user's actual selection endpoints:

$$W_e = 4.133 \cdot \sigma \quad (2.6)$$

where σ is the standard deviation of selection coordinates projected onto the task axis [55, §7.2.3]. The constant 4.133 covers ± 2.066 standard deviations of a normal distribution: exactly 96% of the area. This normalises all conditions to a standard 4% error rate, so you can compare techniques, users, and studies fairly [52, pp. 355–356].

The standard deviation uses the sample formula ($N - 1$ divisor). For a sequence with fewer than two selections, W_e is undefined.

Effective Amplitude (A_e) Effective amplitude measures the actual distance the user travelled. Instead of the nominal centre-to-centre distance between targets, A_e is the mean distance between consecutive *actual selection coordinates*:

$$A_e = \frac{1}{n} \sum_{i=1}^n d_i \quad (2.7)$$

where d_i is the projected distance along the task axis from the $(i - 1)$ -th hit point to the i -th hit point [55, §7.2.3]. This stops participants from gaming the system by aiming at the nearest edge of each target instead of its centre [53, p. 98].

First-Trial Exclusion ISO 9241-9 says the first trial of any sequence gets thrown out of all effective metric calculations. The first selection only establishes a starting coordinate. It does not have a prior hit point to measure amplitude or variance from [55, §7.2.2]. For a circular layout of 11 targets: discard the first, analyse the remaining 10.

Effective Throughput The Effective Index of Difficulty and Effective Throughput combine the adjusted measures:

$$ID_e = \log_2 \left(\frac{A_e}{W_e} + 1 \right) \quad (2.8)$$

$$TP = \frac{ID_e}{MT} \quad (2.9)$$

One critical point: throughput must be computed *per sequence*, not per trial. Computing TP for a single trial is mathematically invalid, since W_e needs a distribution of hits to compute standard deviation from [52, pp. 358–359], [58, pp. 770–772]. The "means of means" method says: compute TP for each uninterrupted sequence, then average those sequence-level throughputs across conditions and participants [55, §7.3].

Angular Fitts' Law for Distal Pointing

The formulations above were built for one- and two-dimensional pointing on flat surfaces. Distal pointing in VR, where the user aims a ray from their hand, head, or eyes at a 3D target, involves different biomechanics entirely.

Rationale for Angular Formulation Distal pointing depends on *rotational* movements of the wrist, arm, head, or eyes, not the *translational* hand movements of desktop pointing [59, pp. 604–605]. So the right formulation uses angular measures: angular amplitude α (the visual angle from starting direction to target direction) and angular target width ω (the visual angle the target subtends from the user's viewpoint).

Converting from world-space dimensions to visual angles at depth d :

$$\text{angle}_{\text{deg}} = 2 \cdot \arctan \left(\frac{\text{size}_{\text{meters}}}{2 \cdot d} \right) \cdot \frac{180}{\pi} \quad (2.10)$$

For small angular sizes relative to depth, this is close to a linear relationship.

Angular Shannon Formulation The angular version of the Shannon formulation swaps linear measures for angular ones:

$$ID_{\text{ANG}} = \log_2 \left(\frac{\alpha}{\omega} + 1 \right) \quad (2.11)$$

Alternative Distal-Pointing Formulations Alternative formulations for distal pointing have also been proposed, most notably by Kopper et al., who assign target width a stronger influence than in the standard angular Shannon variant [59, pp. 608–609]. However, more recent studies conducted on modern head-mounted displays report better empirical fit for the angular Shannon formulation in contemporary VR pointing tasks. For that reason, the angular Shannon formulation is used throughout this thesis [60, pp. 235–237].

Angular Effective Metrics Effective metrics extend naturally to angles. The effective angular target width ω_e is computed directly from the standard deviation of *angular* selection coordinates rather than by converting a world-space standard deviation to degrees. The difference is small at the narrow widths typical in Fitts’ Law studies ($< 5^\circ$), but computing in the angular domain avoids distortions from the nonlinear arctangent and matches current best practice in VR pointing research [61].

$$\omega_e = 4.133 \cdot \sigma_{\text{angular}} \quad (2.12)$$

$$TP_{\text{angular}} = \frac{\log_2(\alpha_e/\omega_e + 1)}{MT} \quad (2.13)$$

Computing these directly from per-trial angular values dodges perspective distortion and correctly captures the variance of human rotational movement.

Speed-Accuracy Trade-off

Fitts’ Law models the speed-accuracy trade-off directly. A user who selects quickly but sloppily produces a large spread of hit points, giving a large effective width and a lower effective *ID*. Someone slow and precise gets a small W_e and a higher ID_e .

Throughput combines both into a single number [52, pp. 355–358]. Two techniques can hit the same throughput through different paths: one fast but messy, the other slow but neat. That’s exactly why throughput works as the primary dependent variable in ISO 9241-9 evaluations [55, §7.3].

2.2.6 ANOVA

Experimental Design

Over 92% of VR Fitts’ Law studies use within-subjects designs, where each participant tests all techniques and conditions. Repeated Measures ANOVA (RM-ANOVA) is the go-to statistical test, used in 79.2% of studies [see 62]. The problem: VR selection data, especially error rates and movement time distributions, often break the normality assumption that parametric tests need.

Aligned Rank Transform

When normality fails (check with Shapiro-Wilk or skewness/kurtosis), the Aligned Rank Transform (ART) procedure goes before RM-ANOVA. ART enables non-parametric factorial analysis that can test interaction effects between multiple independent variables [63].

The procedure has three steps [63]:

1. **Align.** Remove estimated effects of all factors except the one being tested, then compute residuals.
2. **Rank.** Convert aligned values to ranks (midranks for ties).
3. **ANOVA on ranks.** Run standard RM-ANOVA on the ranked aligned values with Greenhouse-Geisser sphericity correction and generalized η_g^2 effect sizes [64]. Post-hoc pairwise comparisons use paired t -tests on ART-ranks with Holm-Bonferroni correction.

Standard non-parametric tests like Friedman cannot model interactions. They are fine for simple ordinal data like Likert-scale responses, but not for this.

Common Methodological Pitfalls

For a systematic review of recurring issues in VR Fitts' Law studies (inconsistent formula reporting, unit errors, ignoring depth and parallax, and flawed per-trial throughput averaging), we refer to Amini et al. [65].

Chapter 3

GRIP-TK

This chapter is divided into four sections, each describing a different aspect of the toolkit and its role in the thesis.

First, the motivation section explains why the toolkit was designed in the first place. It introduces the need for a modular and configurable interaction architecture, especially in a study context where several techniques must be implemented, compared, and adapted without rebuilding the entire system for each condition.

Second, the implementation section describes how the toolkit is built. This includes the overall architecture, the node graph concept, the backend and frontend structure, and the way interaction logic is represented as connected functional units that can be configured and executed at runtime.

Third, the extensibility section explains how the toolkit can be treated not only as a fixed implementation, but also as a framework that can be expanded. In particular, it describes how new behaviour can be introduced both inside individual nodes and at the structural level of the graph, so that the toolkit can support future interaction techniques and research scenarios.

Finally, the walkthrough section shows how these ideas come together in practice. It uses the construction of the HeadCrusher interaction experiment, and later the addition of parallax mitigation, as a concrete example of how the toolkit is configured and how information flows through the system from setup to runtime behaviour.

3.1 Motivation

The motivation behind using **GRIP-TK** is both technical and practical. In a technical sense, it provides a **modular**, **reusable**, and **configurable** structure for interaction techniques, instead of hardcoding each technique separately inside a scene. This is useful because it allows different interaction behaviours to be built on the same architectural foundation, which reduces duplication and makes the system easier to extend, maintain, and compare across different scenarios. By separating the **interaction logic** from the **environment-specific response logic**, the Toolkit also improves **separation of concerns**. This is especially important when working with multiple **experimental conditions**, since the interaction setup can be adjusted without rewriting the entire implementation each time.

In a practical sense, the Toolkit was motivated by the need for faster iteration when developing and testing VR interaction techniques. In our case, even small changes to an interaction technique or its surrounding environment could otherwise

require going back into the scene setup and modifying a more rigid implementation. The Toolkit was therefore designed to provide a scalable structure that can be reused and customized, making it easier to define, adjust, and compare interaction techniques and environments in a more controlled way.

This did not come out of nowhere. Myers et al. argue that user interface software tools matter because they reduce repeated implementation work and make iteration more realistic in practice [66]. In XR, works such as DART, the survey by Ashtari et al., and AUIT show a similar concern from a more domain-specific angle: authoring systems are valuable when they let users build, revise, and test interaction setups without starting over each time [15, 29, 38]. Likewise, Jacob et al. describe non-WIMP interaction in terms of explicit structure between input, state, and behaviour, while Figueroa et al. argue for reusable 3D interaction components rather than one monolithic implementation [36, 37]. The current Toolkit builds directly on that line of thinking, but narrows it toward comparative VR experiment construction, where interaction logic, environment logic, and study configuration all have to stay editable without forcing a rebuild of the whole setup.

3.2 Toolkit Implementation

The toolkit is divided into a frontend and a backend. The frontend consists of the parts that the user sees and interacts with in the immersive environment. This includes the visual node graph, its ports and edges, the configurable elements of the individual nodes, as well as the surrounding menu and interaction layer. The backend is responsible for the actual execution of the interaction technique. It stores the graph structure, propagates data through the connected nodes, and evaluates the interaction logic at runtime.

The frontend and backend are connected through a bridge layer. This means that changes made through the visible node graph, such as editing values or changing connections, are reflected in the backend representation of the technique. The backend then uses both user-defined values and runtime data, such as tracked hand and head information, to execute the interaction flow.

3.2.1 Node Graph Concept

Overview

Before explaining the core implementation of the backend, it is first important to describe the general idea behind the node graph itself. The purpose of the graph is to organize interaction logic in a modular way, so that different behaviours can be built, reused, and adapted more easily depending on the scenario. Instead of placing all interaction logic in one fixed implementation, the backend separates it into smaller functional units that work together as a graph. In this way, the interaction technique is not treated as one monolithic script, but as a composition of connected parts. That follows the same basic reasoning as Figueroa et al., who describe 3D interaction techniques as structures that can be assembled from reusable pieces with distinct roles in the overall flow [37].

That design choice is not only about software neatness. In practice, interaction techniques in XR are often similar in some parts and different in others. A graph

structure makes that kind of partial reuse much easier to reason about, because it lets shared responsibilities stay reusable while the changing parts can be adjusted more locally. That is one of the main reasons node- and component-based interaction models have remained useful in prior work as well: they let the designer think in terms of interaction responsibilities instead of one long control script.

Nodes and Roles

The basic unit of the graph is the **Node**. A node represents a specific class that is responsible for one part of the total graph flow. Depending on its purpose, a node may receive data, produce data, or do both. In the backend, nodes are grouped into a set of architectural roles: **Source**, **Transform**, **Stage**, **Effect**, and **Bridge**.

A **Source** node provides data without depending on graph input, and therefore typically acts as an entry point into the flow. A **Transform** node processes incoming data and produces a new output from it. A **Stage** node represents a meaningful step in the interaction pipeline, such as hover, commit, selection, or dragging. An **Effect** node consumes interaction state and applies a visible or behavioural consequence. Finally, a **Bridge** node is used where the interaction graph must communicate with environment-specific logic outside the core interaction pipeline.

These roles do not mean that the graph is split into isolated subsystems. Rather, they provide a structured way of reasoning about the responsibility of each node. This makes the graph easier to understand, validate, and extend.

Data and Runtime Context

A node can work with data in multiple ways. First, it can contain user-defined data, meaning values that are directly configured by the user. This makes the node reusable, since the same node structure can be used with different parameters in different interaction techniques. Second, it can receive data from other nodes through graph connections. This is the main dataflow mechanism of the backend, where the output of one node becomes the input of another. Third, it can access runtime context data, meaning information that already exists in the current execution environment, such as tracked head and hand data or shared interaction state. This matches the kind of explicit relation between input, state, and behaviour that Jacob et al. describe for non-WIMP interface models [36].

This combination makes each node both configurable and context-sensitive. A node is therefore not only a passive value container, but an active unit that reacts to graph input, user-defined settings, and runtime data. That point matters because XR interaction rarely depends on only one kind of information. A selection step usually depends partly on authored settings, partly on the result of previous processing, and partly on live sensor state. Treating those as separate but combinable inputs makes the logic easier to inspect and change later.

Ports and Links

The connectivity of the graph is defined through **ports** and **links**. A port is the point where data enters or leaves a node, while a link is the connection that transfers data from an output port of one node to an input port of another. In this sense,

the graph is not merely a visual arrangement of boxes, but a structured dataflow system.

Each node has a predefined set of ports depending on its role and purpose. These ports are defined by the developer, while the user works with the available ports when constructing the graph. A connection is only valid if the chosen ports exist, the data types are compatible, and the receiving port allows that form of connection. Some ports accept only a single incoming link, while others can accept multiple inputs. This gives the graph flexibility, while still enforcing structural rules that prevent invalid or meaningless connections. Again, this is close to Jacob et al.'s argument that interaction structure should be explicit enough to describe which relationships are valid and which are not [36].

This is one of the places where the graph stops being just a visual metaphor. Ports and connection rules are what make the structure usable at runtime and editable at authoring time. Without those constraints, the graph would be easy to draw but much harder to trust, because every node could in principle be wired to anything else.

Execution and Lifecycle

A node also follows a clear lifecycle. It is first initialized when it is added to the graph, then executed as part of the graph update, and finally cleaned up when it is removed or no longer needed. This is important because the graph is not just a static structure for authoring, but an active runtime system.

Initialization is used to register ports and prepare internal state. Execution is where the node reads its inputs, evaluates its logic, and writes its outputs. Cleanup is used to release state or disconnect runtime behaviour when the node is destroyed. This lifecycle makes the backend more manageable over time, especially when techniques are created, modified, removed, or rebuilt at runtime.

Graph Flow and Validation

Once the nodes and their connections are defined, the graph behaves like a directed pipeline. Data moves step by step through the established links, starting from source nodes, continuing through intermediate processing nodes, and eventually reaching consumer nodes such as effects or environment bridges. Each node therefore contributes one specific part to the final interaction behaviour. In that sense, the graph behaves much like the compositional 3D interaction structures described by Figueroa et al., where the technique emerges from the coordination of smaller functional units [37].

This structure is useful because it keeps the backend modular and understandable. An interaction technique can be adjusted by replacing or reconnecting only the relevant nodes, rather than redesigning the entire system. In addition, the graph can be validated. Validation helps ensure that required ports are connected, that multiplicity rules are respected, and that the overall structure remains consistent. This is especially important in a toolkit where the graph is meant to be modified and reused across different techniques and environments. Put differently, the graph is not just there to show the interaction. It is there to make controlled variation possible.

3.2.2 Backend

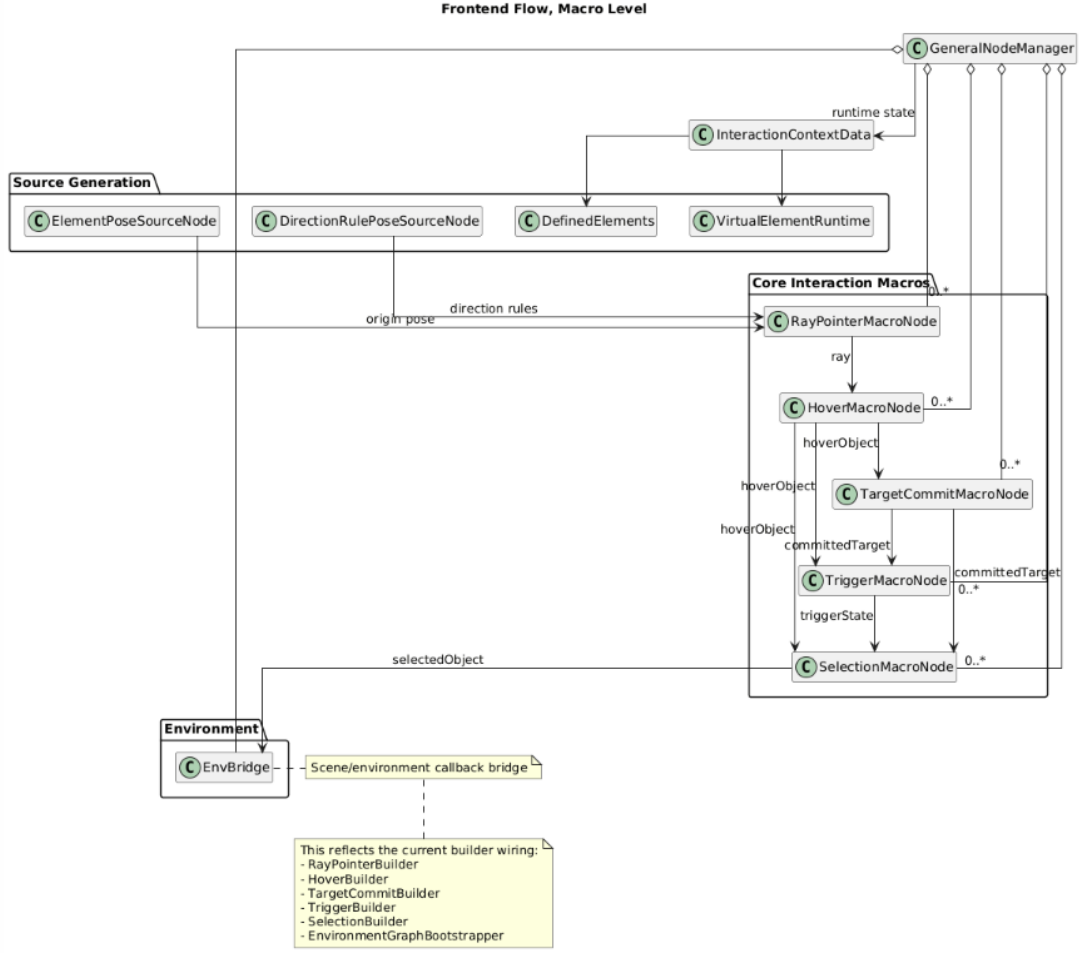


Figure 3.1: UML diagram of the Backend

In this section, we apply the concepts introduced in the previous subchapter to the actual backend implementation of the Toolkit. More specifically, we explain the parts that define the interaction technique itself, the supporting structures that provide runtime data, and the way the final interaction result is passed into the customized environment. To explain the backend in a structured way, Figure 3.1 will be used as a reference for the overall flow of the system.

Backend Overview

The backend can be understood as the runtime layer of GRIP-TK. While the frontend shows the graph to the user and makes it editable in the immersive environment, the backend is responsible for storing the graph structure, propagating data through the connected nodes, and executing the interaction logic at runtime. In this sense, the backend is the part that turns a configured graph into an actual interaction technique. That separation between authoring structure and executable behaviour follows the general toolkit logic described by Myers et al., where software tools are valuable partly because they let users work at a higher level than the final implementation itself [66].

This split between authoring and execution follows a fairly basic toolkit idea: the user should be able to work at the level of reusable interaction structure, while the system takes responsibility for carrying that structure through into actual behaviour. That is one of the reasons the runtime layer exists as its own concern here rather than being folded directly into the visual editor.

At a high level, the backend can be divided into three parts: **Source Generation**, **Core Interaction Macros**, and **Environment**. Each part has a different responsibility, but together they form the complete interaction pipeline. The first part provides runtime data, the second part defines the actual interaction flow, and the third part connects the interaction result to the surrounding scene logic.

Source Generation and Runtime Context

The first part is **Source Generation**. This part is not made up of authorable macro nodes itself, but of scene-level setup components and shared runtime context. In practice, this means that the relevant XR elements and scene references are attached to setup components so that the backend can access the data required by the interaction nodes during execution. These elements form the starting point of the interaction pipeline, because they expose the runtime data that will later be used by the graph. This is close to the way AUIT separates reusable interface logic from the contextual information needed to make that logic work in a concrete XR situation [38].

The reason for structuring it this way is fairly practical. XR interaction depends on many scene- bound references that should stay stable even when the authored technique changes. Treating these as shared sources keeps the graph focused on interaction logic, while still giving the runtime access to the tracked and scene-defined data it needs.

This includes, for example, tracked hand and head references, hover layer definitions, ray visuals, and other shared elements that the interaction logic depends on. In addition to this common setup, GRIP-TK also supports environment-specific dependency packets, so that a given environment can carry its own manager and scene references without forcing all environment data into one shared component. In this way, Source Generation acts as the entry layer between the scene setup and the backend graph.

Core Interaction Structure

The second and most important part of the backend is the **Core Interaction Structure**. This is where the main interaction logic is defined. At a general level, the structure follows a main interaction backbone, where information moves from pointing, to hovering, to optional confirmation, to selection, and, when relevant, to dragging. In other words, the core flow can be understood as a progression such as **Ray** → **Hover** → **Commit** → **Selection** → **Dragging**, where some stages are optional depending on the technique. This kind of ordered interaction structure is very much in line with Figueroa et al.’s view that 3D interaction techniques can be decomposed into recurring functional parts that are then recombined depending on the task [37].

This backbone was chosen because these stages tend to recur across many XR techniques even when the exact implementation differs. The toolkit therefore does

not try to reinvent those stages every time. Instead, it treats them as stable structural ideas that can be recombined, skipped, or specialised depending on the technique being authored.

Not every node in the backend has the same structural role, however. Some nodes belong directly to this main interaction flow, while others act more as attached logic. Trigger nodes are an example of this. They do not define the main interaction medium by themselves, but rather control when a given interaction step is allowed to happen. In a similar way, effect nodes are not part of the main decision flow either, but consume interaction state in order to make that state visible or otherwise noticeable to the user. This distinction is important, because it means that the backend is not just a list of nodes connected in any possible way, but a structured interaction system where some nodes define the backbone and others modify, constrain, or expose its state.

That distinction is important theoretically as well. It separates the question of what the interaction state is from the question of when it becomes valid and how it should be presented. In a comparative experiment toolkit, keeping those concerns separate makes it much easier to vary one part of the technique without accidentally rewriting the rest.

Core Interaction Macros

The main interaction logic is represented through **Core Interaction Macros**. These macro nodes describe the major stages of the interaction flow as user-facing building blocks. Internally, however, they are composed of smaller micro nodes that handle the detailed behavior. The reason for this two-level structure is to provide two different levels of abstraction. On the one hand, the macro nodes offer a clear and practical overview of the interaction technique. On the other hand, the micro nodes allow more detailed control when a specific behavior needs to be adjusted. This makes the backend usable both for simpler setups and for more advanced customization.

The first Macro Node, and also the first node in the interaction backbone, is the **RayPointerMacroNode**, shown in Figure 3.2. This macro node represents the full ray generation process as one higher-level unit. At the micro level, the ray pointer is composed of smaller nodes that handle the origin of the ray, the direction information that guides it, and the final ray construction itself. Additional internal nodes can then be used to stabilize or validate the result, and a visual node can be attached so that the user can see the pointing direction in the scene. This is useful because the full ray behavior can be treated as one interaction block, while the internal parts still remain adjustable if needed.

Figure 3.3 shows the second Macro Node, the **HoverMacroNode**. This macro node represents the hover phase of the interaction flow, meaning the stage where the system determines whether a ray is currently intersecting with an object in the scene. It transforms pointing information into hover information that can then be used by the rest of the backend. At the micro level, one part performs the actual hover evaluation, while other parts maintain hit information and optional visual feedback, such as the hit disk. In this way, the hover phase becomes a reusable interaction stage rather than a single hardcoded scene-specific action.

The third Macro Node is the **TargetCommitMacroNode**, shown in Figure 3.4, which is an optional node in the interaction flow. This macro node is used when

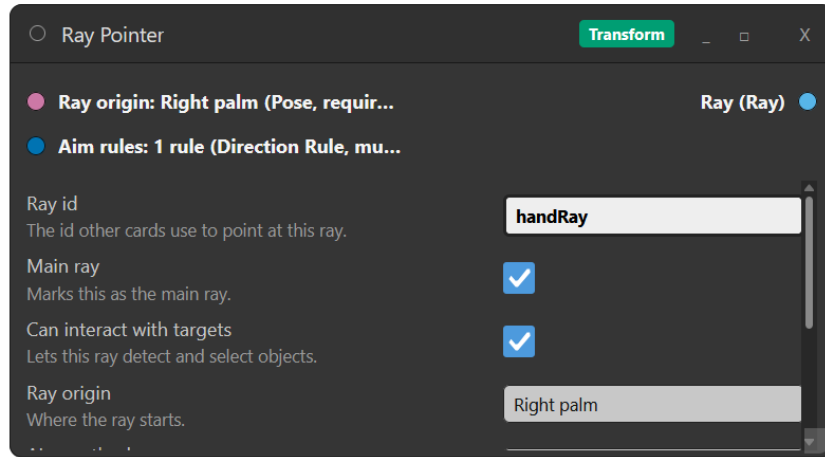


Figure 3.2: The UI of the RayPointerMacroNode in a flattened view.

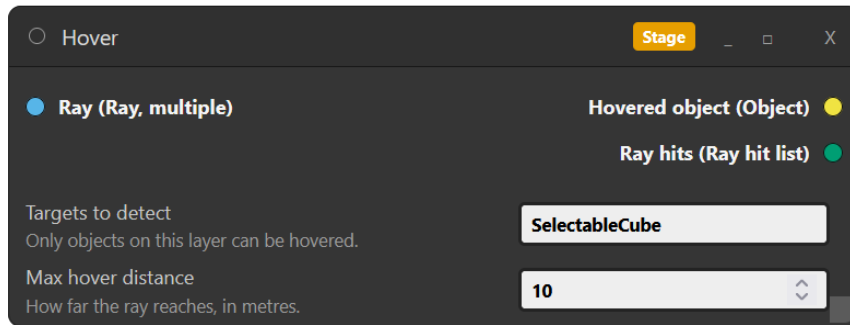


Figure 3.3: The UI of the HoverMacroNode in a flattened view.

the system needs a separate confirmation stage between hovering over a target and fully selecting it. In this way, it helps distinguish between a target that is merely being pointed at and one that has already been deliberately confirmed. At the micro level, one part handles the logical decision of when a target should be considered committed, while another part can represent the progress of that commitment visually. This is useful because the decision itself and the visual representation of that decision are related, but not identical.

The fourth Macro Node, **TriggerMacroNode**, seen in Figure 3.5. This macro node represents the activation logic of the interaction flow. Compared to the previous macro nodes, this one does not primarily define a structural stage of the backbone, but rather controls when another part of the backbone is allowed to act. A trigger can depend on a specific user action, such as a pinch gesture, but it can also depend on additional contextual information, such as the hovered object, the currently active ray, or the currently committed target. Internally, the trigger logic is separated into a trigger action and a trigger condition. The trigger action defines how the trigger becomes active, while the trigger condition defines whether it is allowed to activate in the current situation. This makes the trigger stage one of the most adaptable parts of the backend.

The fifth Macro Node is the **SelectionMacroNode**, as displayed in Figure 3.6.

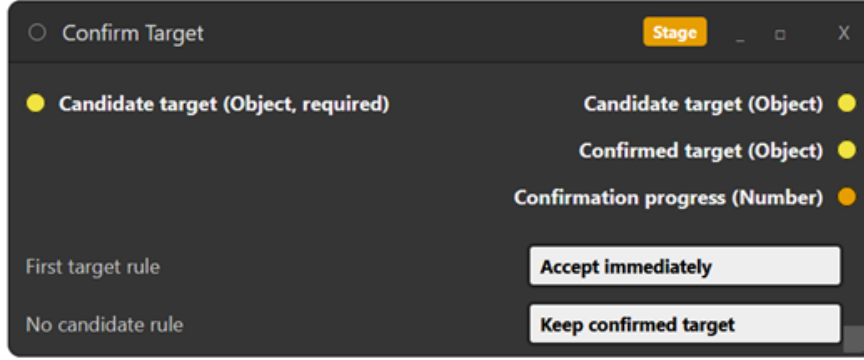


Figure 3.4: The UI of the TargetCommitMacroNode in a flattened view.

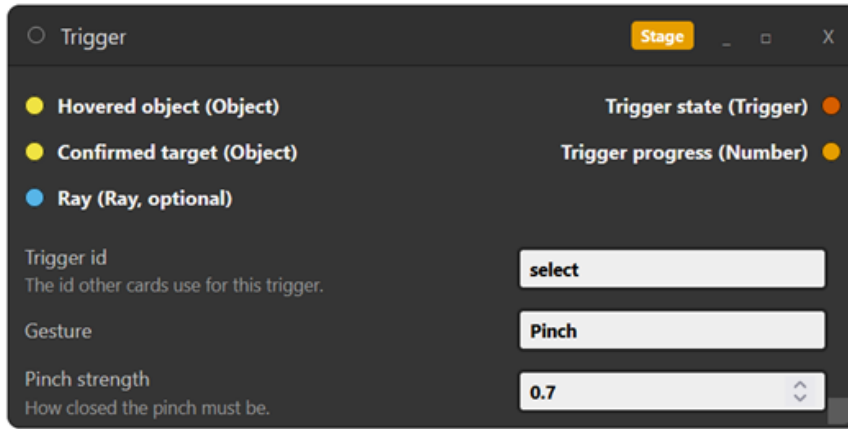


Figure 3.5: The UI of the TriggerMacroNode in a flattened view.

This macro node represents the final selection stage of the interaction flow, meaning the point where a target is no longer only hovered or confirmed, but is actually stored as the selected object. At this stage, the backend combines the current target, the trigger state, and the chosen selection strategy into one coherent behavior. One important aspect is that the target used for selection is not always the same. In some techniques, selection is based directly on the hovered object, while in others it is based on a committed target that has already passed a previous confirmation stage. This makes the selection node flexible, because the same structure can support both simpler and more controlled multi-step interactions.

When object manipulation is needed, this can be extended by the **Dragging-MacroNode**, whose structure is shown in Figure 3.7. This macro node represents the stage where an already hovered or selected target is moved through space based on a chosen dragging strategy. Structurally, dragging depends on the interaction state that comes before it, especially hover information and, in some techniques, selection state as well. In this sense, dragging can be treated as a continuation of the interaction flow rather than as a completely isolated mechanism.

Finally, the backend also contains **effect nodes**. These are attached nodes that consume interaction state and make it visible or noticeable to the user. Hover effects, selection effects, and commit effects belong to this category. They are not responsible

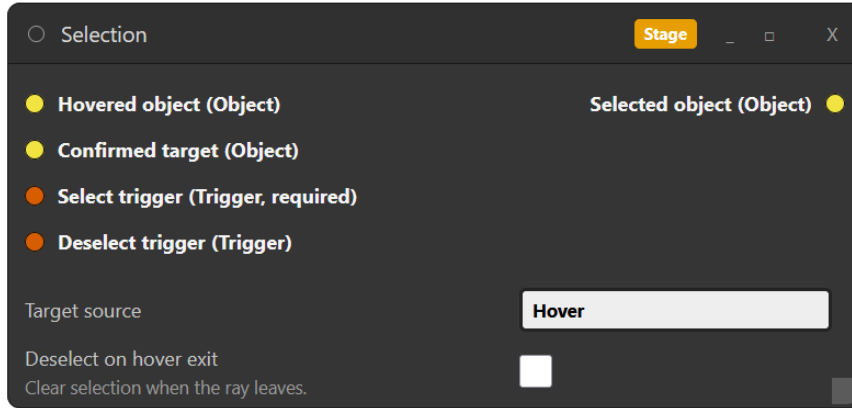


Figure 3.6: The UI of the SelectionMacroNode in a flattened view.

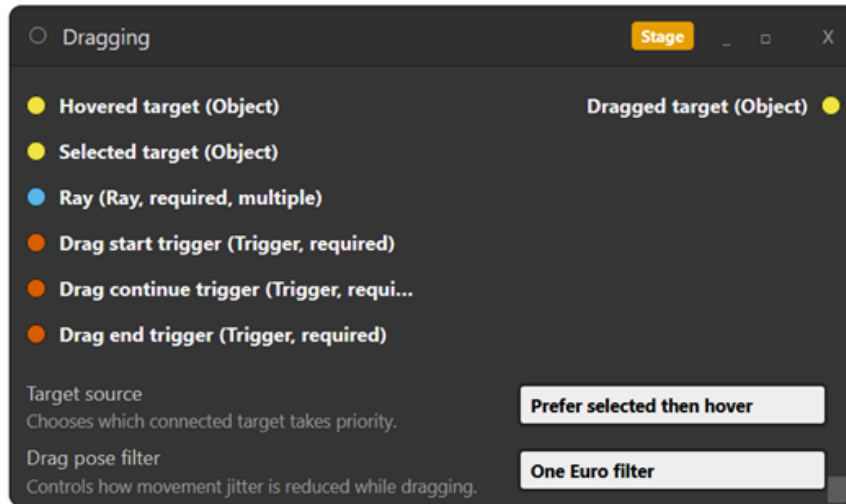


Figure 3.7: The UI of the DraggingMacroNode in a flattened view.

for deciding the interaction state itself, but for exposing that state through visual or behavioral feedback.

Environment Integration

The last part of the backend is the **Environment** section, and its central element is the **EnvBridge**. This part is where the interaction result is passed out of the main graph flow and into the surrounding scene logic. In other words, the backend determines the current interaction state, while the environment decides what should happen in response to it.

More concretely, the interaction result produced by the backend, such as the currently hovered object, the currently committed target, or the selected object, is connected to the EnvBridge through its input ports. The EnvBridge then compares the current interaction state with the previous one and can notify the rest of the application when a meaningful change happens, for example when an object becomes selected or deselected. In this way, it does not perform the interaction itself, but rather acts as the point where the backend interaction becomes available to the

environment.

This is useful because it separates the interaction logic from the scene-specific response logic. The backend decides *what* the current interaction state is, while the environment decides *what should happen next*. This may include updating the active target in a Fitts' Law setup, changing visual feedback, constraining object movement, or informing another manager that the state has changed [29].

Environment Nodes and Managers

The environment-specific behavior itself is represented through dedicated environment nodes, such as a Fitts' Law environment node or a dragging environment node. These nodes are connected to the EnvBridge and act as the backend side of the customized environment logic. They do not replace the actual scene manager written by the developer, but instead connect the graph-based interaction state to that manager in a structured way.

In practice, the environment node receives the interaction state through the EnvBridge, forwards the relevant information to the corresponding manager, and allows the scene-specific environment to react to the technique that is currently running. This means that the interaction technique itself can remain general and reusable, while the concrete task or environment logic remains specialized. For example, the same selection result can be used by a Fitts' Law manager to advance targets, or by a dragging environment manager to activate a constrained manipulation scenario.

Defined Elements and Supporting Plumbing

Besides the visible macro nodes, the backend also depends on supporting plumbing. This includes the shared setup components that provide runtime references, the hidden micro nodes that are created at runtime to implement the internal behavior of the macro nodes, and the environment-specific packets that group the scene dependencies of a particular environment. In this way, the user-facing graph can stay focused on the important interaction structure, while lower-level implementation details are kept in the background.

A particularly important part of this is the distinction between shared and environment-specific dependencies. Common references, such as tracked body elements and shared visuals, are kept in the general `DefinedElements` structure. Environment-specific references, however, can be kept in their own dedicated packet, such as a Fitts' Law defined-elements component or a dragging environment defined-elements component. This makes the architecture more scalable, because new environments can be introduced without forcing all scene references into one shared container.

Overall, the backend was designed as a modular and extensible graph structure that separates interaction logic into clear functional parts. By combining source generation, macro nodes, attached trigger and effect logic, environment bridging, and supporting runtime plumbing, the system can represent interaction behavior in a structured way while still remaining flexible enough to adapt to different scenarios. This architecture provides the foundation for the interaction techniques described in the following sections.

3.2.3 Frontend

The frontend of the toolkit is the part of the system that the user directly sees and interacts with inside the immersive environment. While the backend is responsible for storing node data, executing connections, and updating the interaction logic at runtime, the frontend is responsible for making that structure visible, editable, and testable in VR. In practice, this means that the frontend contains both the **visual node graph interface** and the **in-world menu interface** through which the user creates, configures, and switches between techniques.

Visual Node Graph Interface

The main frontend layer is the spatial node graph editor. Its purpose is to display the authored interaction technique as a set of visible node cards in the 3D scene, so that the user can inspect the structure of the technique, move nodes around, create links between them, and change parameter values directly in VR. Instead of editing the technique through a traditional desktop editor, the user interacts with the graph from within the same immersive environment in which the technique is later tested.

Each visible node is represented as a world-space card. These cards contain a title, optional type badge, input ports, output ports, parameter fields, and node-specific controls such as collapsing, maximising, and deletion. The cards are rendered through Unity UI Toolkit, but presented as three-dimensional objects inside the world. In this sense, the graph is not shown as a separate external editor window, but as part of the immersive workspace itself.

The visual graph is managed through a frontend node management layer which is responsible for creating node cards, assigning visual identities, placing them in space, updating links, and removing nodes when needed. Whenever the user creates or deletes a node, modifies a value, or changes a connection, the frontend updates the visible representation so that it remains consistent with the underlying backend graph. This makes the frontend a live view of the currently authored technique rather than a static visualisation.

Connections between node cards are shown as curved visual links between ports. These links are colour-coded based on the type of data they represent, which helps make the flow of information through the graph easier to read [67]. The purpose of these connections is not only decorative, but explanatory: they make it possible to understand which nodes depend on which others, and how the interaction technique is composed from smaller parts.

Frontend Representation Pipeline

In implementation terms, the frontend follows a model-to-view structure. The authored node data is translated into a visual node representation, which defines the layout and configurable content of the card, and this is then wrapped into a world-space object that can be positioned, rendered, and interacted with in the scene. This allows different node types to reuse the same general visual structure while still exposing their own ports, labels, descriptions, and parameter controls.

This separation is important because it keeps the frontend scalable. New node types can be added without redesigning the whole visual system, since they can reuse the same card logic and simply define their own specific contents. As a result,

the frontend does not need to know the full behaviour of every node in detail; it only needs to know how that node should be presented and which editable values and ports it should expose to the user.

Interaction with the Frontend

Because the graph exists as a spatial interface in VR, the frontend also needs a way to translate three-dimensional pointing into standard user-interface interaction. This is handled through an interactable quad layer that converts raycast hits on the world-space node surfaces into pointer events for the UI Toolkit content. In practice, this allows standard interface elements such as buttons, toggles, sliders, dropdowns, and text fields to be used through the same ray-based interaction system that is also used for the interaction techniques themselves.

This is an important design choice, because it means that the toolkit can be authored and configured using the same interaction medium that it is intended to prototype and evaluate. In other words, the frontend is not only spatial in appearance, but also spatial in how it is used. This follows ideas from direct manipulation and immersive interface design, where interaction becomes more immediate by placing controls inside the same space as the activity itself [68, 69].

Menu Interface

In addition to the node graph itself, the toolkit also contains a separate menu interface for higher-level control. This menu is not part of the UI Toolkit node editor, but is instead implemented as a standard Unity *uGUI* world-space canvas. In the scene, it behaves as a small in-world control panel that can be shown, hidden, attached to the hand, or pinned in space.

The role of this menu is operational rather than structural. Whereas the node graph is used to inspect and edit the internal composition of a technique, the menu is used to control the toolkit as a whole. Through this menu, the user can switch between editor mode and runtime mode, create nodes, delete selected nodes, save and load techniques, load predefined demos, clear the current graph, and adjust display-related options. This makes the menu a separate control layer around the graph authoring interface.

The menu is attached to the user’s palm and oriented so that it remains readable while still feeling like part of the embodied interaction setup. This kind of hand-attached menu is common in VR applications because it gives fast access to controls without requiring a permanently fixed desktop panel, following standard VR platform design guidelines [70]. In the toolkit, this helps separate **authoring actions** from **graph actions**: the user opens the menu to manage the system, but interacts with the node cards to modify the technique itself.

Menu Logic and Mode Control

The menu also acts as a mode controller. When it is opened, it can temporarily disable direct interaction with the node graph in order to avoid accidental overlap between menu input and graph input. This is especially important in VR, where the same ray or gesture input may otherwise affect multiple targets at once. The

menu therefore does not only present commands, but also helps regulate which part of the toolkit is currently active and interactable.

Its internal structure is organised into button-driven panels such as *Edit*, *File*, and *Mode*. These panels expose the available operations of the toolkit in a way that is closer to a classic command menu structure, even though the toolkit itself is spatial and node based. This mix of spatial interaction and structured menu navigation was useful in practice, because it gave the user both a freeform graph workspace and a more controlled way of accessing global actions [71].

3.2.4 Technical Performance

For Virtual Reality experiences and development, having a consistent frame rate is important so you can confidently assert the performance and experience of the interaction techniques that you are authoring. Furthermore, distributors of XR software like Meta have strict guidelines for the allowed performance of the applications that are being shipped, and the performance budget is tighter due to the high resolution and hardware limitations regarding the GPU and CPU. Therefore, we benchmarked the toolkit in its 2 states: Editor and Test under the conditions of a HeadCrusher graph, which we deem to be representative of a medium-sized interaction technique developed using the system. The headset that we tried the benchmark on was a Meta Quest 3 as well as a Meta Quest Pro. Please see the specs of the Pro in Section 4.2.2. The Meta Quest 3 has a display resolution of 2064×2208 pixels per eye and supports refresh rates of 72, 90, and 120 Hz. Other than some minor lag spikes on adding new nodes to the graph, the application runs at a smooth 72 frames per second, at which the headsets are capped (see 1.1). The lag spikes, while mildly bothersome for the flow of the design phase, do not detract significantly from the utility and usability of the toolkit as a design and testing tool.

The view of the frames per second counter can be disabled within the menu interactor.

Frontend Role in the Overall Toolkit

Overall, the frontend serves as the visual and interactive authoring layer of the toolkit. It does not itself define the runtime interaction behaviour of the technique. Rather, it exposes that behaviour in a form that can be configured, inspected, and manipulated by the user. The backend decides what the technique does, while the frontend decides how that technique becomes visible and editable in the immersive environment.

This frontend structure is important for the overall purpose of the toolkit. Since the goal is to prototype and test interaction techniques quickly in VR, the user needs a way to both **see** the current interaction logic and **change** it without leaving the immersive context. The frontend therefore acts as the practical interface through which the toolkit becomes usable.

3.3 User interaction

As the graphs can become rather large both vertically and especially horizontally, it is paramount to be able to easily, quickly and accurately alter your view or the

world state to one that best suits your current focus needs. Furthermore, the quality of present-day HMDs is not good enough to be able to discern writing from far away, something that is needed when working with node graphs. Hence, there are 3 different ways to solve this problem. The first is by dragging the entire graph by aiming, pinching, and dragging the air (i.e., the hand ray's raycast does not collide with any nodes). The second method is dragging individual nodes closer to you, by means of pinching and dragging the title bar, as you would a window in any PC operating system, using direct manipulation [68]. The third is moving via MetaXR's default teleportation system. The first 2 allow for 3 degrees of freedom (meaning "zooming" is possible), while the last provides the ability to rotate around your axis.

The nodes themselves contain different input fields, such as dropdown menus (for backend macros), toggles, incremental and keyboard input types, as well as pure text input for IDs. The IDs can then be rediscovered in consumer nodes through dynamically updated dropdown menus, ensuring that the IDs indeed exist. The nodes are further created at a fixed 1:3 ratio, fitting the longest row on elements. The remainder of the card will overflow using default UI Toolkit scrollwheels.

Three buttons in WIMP style are located at the top right of the cards, on the title bar. One for deletion, an x; a minimization icon; and a maximization or resizing icon. These work as expected and come in handy when managing the large cards of the program. If you press the deletion icon, as per Shneiderman's [72] error prevention rule, a pop-up inquires about your intention to delete it. This is more important in VR given the complicated laser-pointer-style interaction in use.

Finally, you are able to resize the cards to any size you desire by dragging the corner of it, making the cards act as a familiar metaphor for typical application usage on PCs, lowering the learning curve necessarily. In the same stroke of design thinking, every part of the user interface is documented specifically (all values giving useful information about what the different knobs do) as well as including hover tips for elements such as input and output ports, badges and the title.

All parts of the cards are interactable through a hand ray with pinch as the trigger, while some input fields bring up a virtual keyboard to input string-based values.

3.4 Extensibility: Toolkit as a framework

The toolkit was not built only to support a fixed set of interaction techniques. A large part of the idea behind it is that the system can be extended without rewriting the whole architecture every time a new technique, environment, or behaviour is needed. In practice, this extensibility happens on several levels. Existing nodes can be modified internally, new node types can be added, new macro-level structures can be introduced, and new environments can be connected to the runtime through the same general graph model. Framed in Ledo et al.'s terms, this is exactly the kind of toolkit value that matters in HCI research: not only that a system works once, but that it can support reuse, variation, and further construction beyond a single fixed example [11, pp. 1–2].

A concrete example in this project is the parallax mitigation logic, which extends the original interaction setup with additional behaviour that was not part of the initial toolkit structure. That example is explained more concretely in the **Walkthrough** chapter. Here, the point is to describe the general way in which the

toolkit can be extended.

3.4.1 Node-internal extensibility

One way of extending the toolkit is to modify nodes that already exist. This does not necessarily mean changing their overall role in the graph. In many cases, the node stays where it is, keeps the same input and output responsibilities, and still participates in the same broader interaction flow. What changes is the logic inside the node.

This kind of extensibility can vary quite a lot. Some modifications are small and mostly affect the decision logic of the node. Others go deeper and change how the node processes or preserves data over time.

Node modifiers

A deeper modifier changes the internal processing model of a node rather than only adjusting a parameter or helper strategy. One example would be changing the ray logic from a purely frame-based process into a temporally managed one. Instead of only using the current gaze direction, a modified `RayPointerNode` could briefly preserve previously stable ray information and use it during pinch onset. This would make the interaction less sensitive to small timing mismatches between gaze and pinch, for example when the user is still looking at the intended target but the gaze signal shifts slightly before the pinch is fully registered. The general idea follows work on temporal correction heuristics in gaze-and-pinch interaction, where selection is treated as a short time-windowed event rather than a single instantaneous sample [73]. In the context of this toolkit, such a change is a deep modifier because the node no longer only forwards present input state, but also uses remembered information to affect later hover, trigger, and selection behaviour.

Other modifications are more local. A good example is the strategy mechanism used by the `SelectionNode`. The node still receives target-related and trigger-related input, and it still outputs a selected object. But the internal rule for how selection is maintained can differ. One strategy may keep the object selected only while the trigger is held. Another may let the same object stay selected until a second trigger action explicitly deselects it. In that case, the surrounding graph does not need to change, but the node behaviour is still meaningfully extended.

So node-internal extensibility is not only about changing values. It can also mean changing the behavioural rule set of a node while keeping the wider graph structure intact.

3.4.2 Structural extensibility

The toolkit can also be extended more structurally by adding new node types to the graph. This means that the architecture itself gains new building blocks, rather than only reusing or adjusting the ones that already exist. This follows the same compositional logic described by Figueroa et al., where extending the interaction space means adding or recombining functional units rather than rewriting the whole technique [37].

Creating a new node starts with defining its role in the graph. That role already constrains a lot: what the node should consume, what it should output, where it

belongs in the interaction flow, and which other nodes it is expected to cooperate with. After that, the node can define its input ports, output ports, and the type of values it works with. Once these parts are in place, the runtime logic of the node can be implemented.

Compatibility matters here. If a new node outputs a type of value that is already used in the toolkit, then it can often be integrated fairly directly. But if the new node produces a new kind of data, then the receiving node must also be extended so that it can interpret that data correctly. And that change can spread further through the pipeline if later nodes also depend on that same output. The same applies in the other direction for new input requirements.

Not every new node has to fully implement a concept by itself. Some nodes act more like bridges between scene-defined logic and the graph architecture. In such cases, the node mainly exposes parameters, forwards relevant data, and connects external runtime logic to the toolkit structure. This is important because it allows the toolkit to connect to specialised functionality without forcing every implementation detail to live directly inside the general node system.

3.4.3 Macro-level extensibility

In the current toolkit architecture, extensibility does not only happen at the level of individual micro-nodes. It also happens at the level of macro structures. This is an important distinction.

A macro node represents a larger interaction concept by grouping together smaller responsibilities that logically belong together. In this project, this includes concepts such as ray definition, hover handling, triggering, selection, dragging, effects, and environment behaviour. Because of this, extending the toolkit often means more than adding a single isolated processing step. It can also mean introducing a new reusable interaction block that fits into the same overall graph model.

This is useful because it keeps the authored graph at a more understandable level. Instead of forcing the user to think only in very small technical units, the toolkit can also expose larger interaction concepts that still map down to a structured backend. From an extensibility perspective, this means the toolkit supports both lower-level and higher-level growth. That kind of layered abstraction is also consistent with Myers et al.’s general argument for interface software tools: good tools do not only automate implementation, they also let the user work at a more useful level of description [66].

3.4.4 Environment extensibility

A particularly important part of the current architecture is that environments can also be added as extensions. The environment is not treated as something entirely separate from the toolkit, but as a part of the system that can be connected through the graph in a structured way.

To add a new environment, it is generally not enough to only create scene objects. The architecture also expects a matching environment-side manager, a corresponding environment node in the graph, and the relevant scene-bound references that the environment needs in order to function at runtime. In the current implementation, this is supported through the environment bridge and through environment-specific

defined elements.

This matters because different environments may react differently to the same interaction state. One environment may care mainly about selection. Another may need hover or dragging information as well. By exposing environments through dedicated nodes and bridge logic, the toolkit can support that kind of variation without hardcoding all environment behaviour into the central interaction pipeline. Here again the reasoning is close to AUIT: the reusable interaction structure stays general, while the environment-specific response can remain contextual and situation-dependent [38].

3.4.5 Effect extensibility

Extensibility in the toolkit is also about effects. Interaction does not end with detecting a ray, hover state, or selection event. In many cases, the system also needs to define what visible or behavioural response should happen when that state occurs.

For that reason, effects are treated as their own extendable concern. A hover stage may need a highlight effect, a selection stage may need a persistent material change, and another stage may need something more specialised. This means that extending the toolkit can involve adding new effect logic or new effect recipes, not only changing the core interaction stages themselves.

This separation is useful because it avoids mixing state detection with response logic more than necessary. The graph can therefore remain clearer: some nodes are mainly about detecting or deciding, while others are mainly about what should happen as a result.

3.4.6 Frontend and plumbing support

One practical point is that adding a new runtime idea is not always enough by itself. If a new node is meant to be authored, configured, and reused through the toolkit interface, then the frontend usually also needs to support it. This means giving the node a visible representation, configurable fields, and a way to appear as part of the authoring flow.

The same is true for scene-bound resources. Some extensions require additional references, prefabs, transforms, materials, or manager connections to be made available to the runtime. In the toolkit, that kind of support is handled through the surrounding plumbing, such as defined elements and environment-specific support objects.

So in practice, extending the toolkit is not just about adding code to a node class. It can also involve extending the authoring interface and the scene-side support structure that allows the node to run properly in context.

3.4.7 Exporting Technique Feature

In addition to saving and loading techniques inside the toolkit itself, the current implementation also supports exporting a technique into a more standalone form. The purpose of this feature is to make the authored interaction technique less dependent on the original toolkit scene. Otherwise, the graph would remain tied to the

immersive authoring environment in which it was created, which would go against the broader toolkit goal of reuse beyond one fixed setup [11, pp. 1–2].

The feature still builds on the same backend structure already used for persistence and runtime execution. So exporting does not create a second interaction model. The same graph and the same runtime logic are kept, but moved into a form that is easier to use outside the normal authoring workflow.

What is exported is mainly the interaction technique itself: the authored graph structure, the node configuration, and the relevant links that define the final interaction flow. In addition to that, the exporter also captures the scene-side tracked reference bindings that the technique depends on, such as the eye, hand, palm, and fingertip hierarchy. This matters because the exported technique still needs a way to reconnect itself to an XR rig in another scene. Without that information, the graph structure alone would not be enough to run meaningfully.

As a conclusion, the exported result is not a copy of the full toolkit scene, and not a copy of the whole authoring environment either. It is more like a packaged version of the authored technique together with the information needed to bind it back to the expected runtime setup. That is also why exporting serves a different purpose from ordinary saving and loading. Saving and loading are about continuing work inside the toolkit. Exporting is about taking the same authored technique and making it portable enough to be reused elsewhere.

3.5 Walkthrough

Following the toolkit evaluation perspective described by Ledo et al., who argue that toolkit research should be presented and assessed in ways that make actual use visible [11, pp. 1–2], this chapter uses a concrete walkthrough to show what the toolkit actually looks like in use. That fits this project better than only describing the system in abstract terms. The toolkit is easier to understand when it is tied to one specific example, because the reader can then follow how a real interaction technique is loaded, inspected, modified, and connected to an environment. For that reason, this walkthrough focuses on the **HeadCrusher** technique and then uses it as a basis for showing how the toolkit can also be extended with parallax-related behaviour.

There is also a more practical reason for presenting it this way. A toolkit like this is easier to judge when the reader can see where the abstractions hold and where they start to matter in a real setup. The walkthrough therefore does more than illustrate the interface. It shows why the graph, the separation of concerns, and the extension points are useful once an actual technique has to be loaded and changed.

3.5.1 GRIP-TK pre-runtime configuration

Before any authored interaction technique can be tested, the toolkit scene already needs to contain the shared runtime setup that the graph depends on. This mainly concerns the references that are used by the backend, the visual editor that is used by the frontend, and the bridge layer that keeps both sides in sync.

On the backend side, the tracked scene elements that nodes depend on must already exist. This includes the relevant head, eye, hand, palm, and fingertip references. In the current architecture, these references are collected through the shared

`DefinedElements` structure, while environment-specific references are grouped in their own dedicated support objects. The purpose of this setup is simple: when a node later asks for eye position, fingertip position, palm pose, ray visual prefabs, or environment-bound objects, that data is already available in a predictable place.

The frontend also has to be present from the start. The node editor is not something that appears after a technique is running. It is already part of the scene and acts as the immersive authoring workspace. The same is true for the menu. The user therefore enters a scene where the toolkit interface itself is already interactable in VR before the authored technique is active.

This is important because the toolkit uses its own interaction setup for authoring. In other words, the user can already interact with the graph editor and the menu even before the technique being designed has been executed. That separation matters. The toolkit needs one interaction layer for editing and another for the interaction technique that is being authored and tested.

The final part of the preparation is the API and bootstrap layer. This is the part that ties together the runtime graph, the visual node graph, and the surrounding environment logic. It is responsible for exposing creatable node types to the frontend, synchronising authored graph content with runtime execution, and switching between editor-side behaviour and environment-side behaviour when the user wants to test a technique. Once that shared setup is in place, the toolkit is ready to be used as an authoring environment.

3.5.2 Loading and inspecting the HeadCrusher technique

After the scene setup is in place, the HeadCrusher technique can be loaded through the toolkit interface. In the current implementation, this does not mean that the user writes the whole technique manually from scratch every time. Instead, the technique can be loaded as a predefined graph, inspected through the node interface, modified if needed, and then executed in the connected environment.

This is a useful starting point, because the HeadCrusher technique shows quite clearly how the toolkit expresses an interaction technique as a sequence of connected processing stages rather than as one hidden script. At the authoring level, these stages are represented through macro-level node concepts such as ray definition, hover handling, triggering, selection, and environment reaction. So even though the runtime logic underneath is still structured in smaller parts, the user sees and works with a more understandable interaction-level graph.

The technique begins with the ray-definition stage. In the HeadCrusher case, the ray is not simply a forward ray from the head. Instead, it is defined from the user's eye anchor toward a point that depends on the relationship between the thumb and the index finger. This is one of the reasons why HeadCrusher is a good walkthrough example. It makes it very clear that the toolkit is not just loading a final behaviour, but exposing how that behaviour is constructed.

Once the ray has been defined, the next stage is hover evaluation. Here, the directional ray is tested against selectable objects in the scene so that the system can determine what is currently being targeted. At this point, a purely geometric signal becomes meaningful task-related interaction data. The walkthrough benefits from this step because it shows that the toolkit separates the construction of a pointing signal from the interpretation of that signal in relation to objects in the

environment.

The next stage is trigger evaluation. In the HeadCrusher example, the confirming action is a pinch gesture. This means that the toolkit does not treat pointing and confirmation as the same thing. A user may continuously aim at a target, but a valid selection only happens when the trigger condition is also satisfied. Keeping these stages separate makes the graph easier to inspect and reason about.

After that comes the selection stage. This stage combines the current target information with the trigger state and applies the chosen selection rule. That is the point where the interaction becomes an actual selection rather than only a potential one. The system moves from "this object is currently under the ray" to "this object is now the selected outcome of the interaction."

In the current HeadCrusher configuration, the selection logic also includes a freeze-related stabilisation step. To reduce the Heisenberg effect during pinch onset, the relationship freeze option is enabled. The freeze mode is set to the local point reference, having the right hand palm as the reference element. The freeze threshold is set to 0.045 metres, which means the relationship point freezes once the thumb and index fingertip distance reaches 4.5 cm or less. The release threshold is set to 0.07 metres, so the point becomes live again only once the fingers move at least 7 cm apart. Using two different thresholds creates a small hysteresis region, which helps avoid rapid switching between frozen and unfrozen states when the hand is close to the threshold.

The final relevant stage is the handover to the environment. Once an object has been selected, the result is forwarded through the environment-side connection so that the surrounding task logic can react to it. In the current toolkit architecture, this is not treated as an ad hoc scene callback, but as an explicit part of the graph-to-environment relationship. In the HeadCrusher case, that means the selected target is passed into the connected Fitts' Law environment, where the experiment logic can evaluate and respond to the result.

Seen as a whole, the HeadCrusher example shows the intended authoring model of the toolkit rather well. The technique is represented as a graph of reusable interaction stages, the graph can be inspected and modified in VR, and the result can then be connected to a specific environment without collapsing the whole setup into one hardcoded behaviour.

3.5.3 Adding parallax-related behaviour

The final part of the walkthrough shows how the toolkit can be extended beyond the base HeadCrusher configuration. In this case, the extension concerns parallax-related behaviour. The important point is that this does not replace the HeadCrusher interaction technique itself. The same pointing logic can still remain in place. What changes is the condition under which that technique is visually experienced or experimentally configured.

This is where the `DisplayModeNode`, displayed in Figure 3.8, becomes relevant. Its role is not to create a new ray, a new hover rule, or a new selection mechanism. Instead, it controls the display condition under which the technique is being used. In that sense, it belongs to a different kind of concern than the core interaction stages. The ray stage still answers how the user points. The trigger and selection stages still answer how a target is confirmed. The display node instead answers under which

viewing condition this interaction takes place.

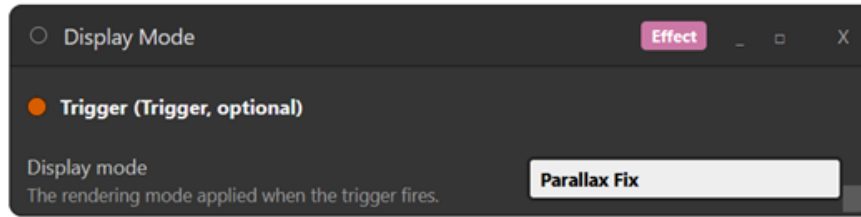


Figure 3.8: The UI of the DisplayMode node in a flattened view.

That distinction matters. In the HeadCrusher case, display condition is not just a cosmetic detail. It is part of the actual experimental manipulation. Representing that explicitly inside the toolkit makes the setup easier to inspect and reason about, because the user can see that the interaction technique and the display condition are related but not identical parts of the overall design.

At the same time, the current HeadCrusher configuration is not only about display mode. There is also extra spatial logic that contributes to stabilising the behaviour during pinch input. So the `DisplayModeNode` should not be read as the entire mitigation by itself. It is simply the clearest graph-level example of how parallax-related variation can be made explicit inside the toolkit authoring flow.

This final example shows an important strength of the toolkit. A technique does not have to be built as one closed and fixed object. A base interaction technique can first be loaded or assembled, and then additional behaviour can be layered on top when a study requires it. That is one of the main reasons the toolkit is useful as a framework rather than only as a one-off implementation.

3.5.4 Exporting the HeadCrusher technique for a different use case

After the HeadCrusher technique has been loaded, inspected, and optionally modified, the user can also export it through the *File* menu. This makes it possible to take the currently authored version of the technique and package it into a more standalone form.

In the walkthrough, this is simply the next step. The technique has already been loaded, inspected, tested, and extended with the parallax-related variation. Once the graph is in the desired state, the export option can be selected from the menu, and the toolkit uses the current graph as the basis for the exported result.

What is carried over is mainly the technique definition itself together with the tracked reference bindings it depends on. In the HeadCrusher case, this includes the authored chain from ray definition through hover, trigger, selection, and environment handoff, as well as the bindings to the relevant tracked anchors that make the technique executable in XR. The point is not to export the whole Fitts' Law scene, but to preserve this specific authored version of HeadCrusher so that it can later be reconnected to another runtime setup without rebuilding the graph from scratch.

This matters in the walkthrough because it shows that the toolkit does not stop at immersive editing. It can also hand off the resulting technique for later reuse. So

the workflow does not end once HeadCrusher works in the current scene. It can be preserved in a form that is easier to carry into a different use case.

A video demonstration of the toolkit, covering node-graph authoring, runtime inspection, and the walkthrough workflow described above is available at youtu.be/eNQSgjRg51w.

Chapter 4

Evaluation

4.1 Motivation

The developer is interested in testing their techniques with real users. They set up 2 research questions for the purpose: how do their parallax-mitigated selection techniques fare against a) a technique that does not suffer from double vision (control), and b) the parallax-mitigated techniques' vanilla counterparts. The developer wants to use an ISO 9241-9 Fitts' Law study design for this, partially replicating the techniques and methodology of Wagner et al. [12, pp. 1–15].

When a participant moves their hand through the air and pinches their fingers to select a target, the raw data is a stream of timestamps, positions, and bone rotations. This chapter explains how we turn that stream into Fitts' Law throughput figures and, critically, why those figures can be trusted. Every design choice in the pipeline traces back to a specific requirement from the Fitts' Law and ISO 9241-9 literature. We walk through the whole procedure: what signals we capture, how each metric is computed from theory, and what safeguards ensure the numbers are correct before they appear in any graph.

This study was not preregistered.

The Unity application, data processing pipeline, and analysis scripts are available in a private GitHub repository and can be shared upon request from the authors. Raw experimental data (participant logs and questionnaire responses) are stored on encrypted university servers and are available to examiners and reviewers under the university's data management policy.

The study design and reporting follow the evaluation guidelines of Bergström et al. (2021) for VR object selection and manipulation studies, and the 3D XR Fitts' Law reporting framework of Amini et al. (2025). A compliance summary mapping each guideline to its corresponding thesis section is provided in Appendix ??.

4.2 Study Design

4.2.1 Task Design

Specification

The task comprises selecting 1 of 11 targets situated in a circular pattern with some amplitude in $A := \{10^\circ, 30^\circ\}$, defined as the distance between any 2 opposing targets.

The targets have a width of $W := \{5^\circ, 8^\circ\}$ and are distanced from the test subject by depth $D := \{1\text{m}, 3\text{m}\}$ to represent a broad set of values [50, pp. 381–391]. Each combination of technique $\times$ parameter is shown to the test subject once. The depth is changed after all values of A and W have been used. All techniques begin with an extendable training phase with $A_{\text{training}} := \{15^\circ\}$, $W_{\text{training}} := \{7^\circ\}$, $D_{\text{training}} := \{2\text{m}\}$, configured to be easy and not represented in the actual study trials.

We finally define the set of techniques

$$T := \left\{ \begin{array}{l} \text{Gaze\&Pinch} \\ \text{HeadCrusher} \\ \text{Gaze\&Finger} \\ \text{HeadCrusher}^* \\ \text{Gaze\&Finger}^* \end{array} \right\}.$$

* indicates parallax mitigation. These are counterbalanced by a Latin square using Williams design [74, pp. 149–168] with 10 different orders, each being used by 2 participants.

Reasoning

This section explains the rationale behind the task choices used in the evaluation. We first justify the use of the amplitude, width, and depth variables, along with the selected values, and then explain the choice of interaction techniques and execution order.

Variables justification This section explains why we used the A , W , and D variables. Since we only use two values per variable, the study stays manageable in length and does not become too exhausting for the participants. At the same time, using only two levels still gives us a clear low and high condition for each variable, which is enough to observe how the interaction techniques behave under different task demands without creating too many condition combinations.

The reason for including amplitude A in our Fitts’ Law experiment is that it directly affects the Shannon index of difficulty ((2.4) from §2.2.5). As amplitude increases relative to target width, the index of difficulty also increases, which makes the task more demanding. This is useful for our study because it allows us to examine how the interaction techniques perform across different movement distances and difficulty levels. In particular, varying A helps us test whether the parallax-mitigated techniques remain effective when the pointing distance becomes larger and the user has to perform a more challenging selection. By varying the distance between the targets, we can also observe physical movement costs such as the **Gorilla Arm Effect** [47, pp. 1063–1065], especially when selection becomes more demanding.

Width W is included for the same reason, but in the opposite direction: larger widths reduce the index of difficulty, while smaller widths increase it. This makes width an important variable because it lets us observe technique behaviour under both demanding and easy conditions. A technique may be more suitable for large, forgiving targets even if it becomes less efficient for smaller targets, so including multiple widths helps us capture that difference. Smaller widths also increase the likelihood of selection errors, which makes it easier to observe trade-offs between precision, speed, and comfort. In particular, narrower targets can reveal whether a

technique becomes more sensitive to ray jitter, which in turn may affect error rate and selection stability. They can also highlight whether a technique reduces accidental activation, for example by helping to avoid the Midas Touch, the ambiguity between looking to perceive and looking to select [46, pp. 12–13], at the cost of additional effort or slower performance.

Depth D is included because we need to explain how the same target can behave differently depending on how far away it is from the test subject. In a VR setting, depth changes the visual angle of the target and therefore also changes how difficult it is to point at. This makes depth relevant for our study, since it allows us to observe how the interaction techniques perform when the selection task is closer or further away. It is also especially useful for our topic because the depth mismatch between the near-hand pointer and the far target becomes more pronounced at greater distances, which means that the effect of parallax mitigation can be observed more clearly. In other words, depth helps us test whether the techniques remain stable in ideal conditions as well as further away where misalignment risk grows.

The reason for using only two values for A , W , and D is to keep the experiment manageable in length. More values per variable would have increased the number of condition combinations and therefore made each round longer, which could have caused more tiredness and frustration for the participants. At the same time, the chosen pairs still give us a clear low and high condition for movement demand, precision demand, and viewing distance, respectively, so we can observe how the techniques behave under easier and harder task demands without making the study too exhausting.

Interaction techniques justification The interaction techniques used are G&P, G&F, and HC, as well as parallax-mitigated versions of G&F and HC. Concretely, the reason for including G&P is to have a baseline based on a common interaction technique used in Fitts’ Law experiments, so the other techniques can be compared against it.

G&F and HC are view-dependent selection techniques, meaning that the user selects targets based on their projected position from the user’s point of view rather than by physically reaching them in world space. Because the selection depends on this projected alignment, both techniques are affected by parallax, especially when the target is placed at different depths. This is why we compare them with their parallax-mitigated versions: the mitigated and unmitigated techniques are otherwise the same family of techniques, so the comparison isolates the effect of the mitigation itself. It also gives us a cleaner baseline comparison, while showing whether the mitigation reduces the mismatch between the perceived target position and the hand-based pointer.

4.2.2 Apparatus and System Implementation

The study is implemented with a Meta Quest Pro (1800×1920 pixels per eye, supported refresh rates of 72, 80, and 90 Hz, and 30 Hz eye tracking), with the software being built using Meta XR All-in-One SDK 83.0.4 [75] in Unity LTS 6000.3.12f1.

4.2.3 Interaction Technique Implementations

In this chapter, we will explain how the experiment interaction techniques were implemented using the toolkit, as well as the differences we made compared to related work.

Gaze and Pinch

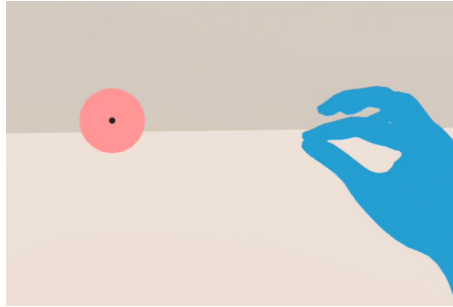


Figure 4.1: Gaze and Pinch

The G&P configuration is shown in Figure 4.1. It was implemented as the simplest of the evaluated techniques and serves as the control condition in the study. The pointing ray originates from the centre eye position and follows the gaze direction. In other words, the participant points by looking at the target and confirms the selection by pinching with the preferred hand.

We deliberately used the raw gaze ray without additional smoothing. Since the targets were large enough to tolerate a small amount of jitter, preserving responsiveness was more important than removing every small fluctuation. Adding filtering here would likely have introduced extra latency without giving a meaningful benefit for the intended task difficulty.

Once the ray intersects a valid target, the system treats that target as hovered. Selection is then confirmed through a pinch gesture with a threshold of 0.7. A pinch only counts while a valid target is currently being hovered, which prevents empty-air activations. The resulting behaviour is therefore straightforward: look at a target, pinch to confirm, and pass the confirmed selection to the Fitts' Law task logic.

Gaze and Finger

As illustrated in Figure 4.2, G&F was implemented as a two-ray technique that separates target indication from target confirmation. The first ray represents gaze and is used to determine which target is currently being looked at. The second ray represents the finger direction and is used as the confirmation signal. Selection therefore requires the participant to both look at the target and align the finger with it in view.

To keep the hovered target stable, the gaze ray is smoothed, whereas the finger ray is left unfiltered. We made this choice because the gaze-proxy signal is the noisier of the two, while the finger signal already comes from hand tracking and benefits more from responsiveness than from additional smoothing. Filtering the finger ray as well would have added latency to the alignment test without giving much practical benefit.

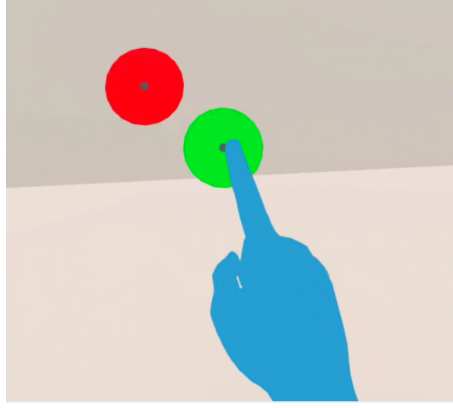


Figure 4.2: Gaze and Finger

The selection trigger is based on angular alignment between the gaze ray and the finger ray. Unlike prior GHA implementations [Lystbaek2022], our build omits the 150 ms dwell, relying purely on angular alignment. A selection registers when both rays intersect the target plane within the angular threshold. The allowed angular difference is adaptive rather than fixed: it is derived from the eye-to-finger geometry and then scaled by a multiplier of 0.6. In our build, the base visual angle is set to 4° or 6° , depending on the variant being tested. This makes the criterion more sensitive to the actual viewing geometry instead of relying on one static threshold for all cases.

Unlike some more staged interaction flows, this implementation does not insert a separate commit phase between hover and selection. The decision therefore depends directly on the current hover state and the live gaze-finger alignment. The advantage is that the technique stays simpler and more responsive. The drawback is that brief gaze jitter or a rapid target switch can immediately affect the outcome, since no extra commit buffer is present.

HeadCrusher

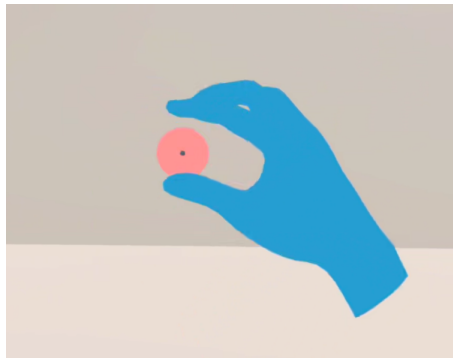


Figure 4.3: HeadCrusher

Figure 4.3 depicts the HeadCrusher configuration. It follows the same overall interaction flow as G&P, but its pointing ray is defined in a very different way. Instead of simply following the forward direction of the headset, the ray starts from the user’s viewpoint and is directed by the relationship between the thumb and

index finger. This makes the technique view-dependent: the user aims by aligning the hand-based relationship with the target as seen from the head.

In our implementation, the relationship point is biased toward the thumb. The reason is practical rather than theoretical: during pinch preparation, the thumb tends to provide a slightly more stable reference than the index fingertip. This helps reduce visible jitter in the pointing direction while still keeping the ray responsive to the user’s hand movement.

We also enabled a freezing mechanism for the thumb-index relationship during the closing phase of the pinch. Once the two fingertips move within 4.5 cm of each other, the relationship is stabilised relative to the hand and only becomes live again once the fingers separate beyond 7 cm. Using different thresholds for activation and release creates a small hysteresis region, which prevents the ray from rapidly switching between frozen and unfrozen states near the pinch boundary. We stabilise the relationship relative to the hand rather than locking it in world space, because this makes the technique easier to control during the final selection phase.

After the ray has been constructed, the rest of the interaction remains similar to G&P. The system detects the currently hovered target, and the preferred-hand pinch confirms selection with the same threshold of 0.7. As before, the pinch only counts when a valid target is being hovered. This keeps the comparison with G&P clean, while isolating the main difference to the way the pointing ray itself is generated [42, pp. 39–41];[12, pp. 1–15].

4.2.4 Data Collection and Metrics

The Unity application logs every trial to CSV: target coordinates, selection timestamps, ray directions, and body tracking data. We record continuous hand and head poses at headset refresh rate alongside discrete selection events. The experiment pauses recording during transitions and idle periods so menu interactions and scene loading do not leak into the timing data.

Angular Conversion

Fitts’ Law operates in visual angles, not world-space metres. A 20 cm arm movement spans roughly 11° at 1 m depth but only 6° at 2 m. Without angular conversion, a near-plane trial and a far-plane trial with the same perceptual layout would produce incomparable world-space amplitudes. We convert all spatial metrics to visual degrees using (2.10) (§2.2.5), where D is the headset-to-target distance at selection time [59, pp. 604–605];[60, pp. 235–237]. The angular Shannon formulation for index of difficulty follows (2.11).

Core Metrics

Movement time (MT) is the interval between consecutive selections, measured from the moment the previous selection completes until the current one fires [55, §7.2.1]. We filter out trials with sub-millisecond MT (double-click artefacts) and trials where the selection fired before the experiment start marker (stale timers from a previous block).

Hit deviation measures where the selection ray landed relative to target centre. We decompose the vector into an along-axis component (overshoot or undershoot in

the task direction) and an across-axis component (orthogonal jitter). The along-axis spread feeds into the effective width calculation. Hand movement is the cumulative frame-to-frame displacement of the index fingertip from experiment start to each selection.

4.2.5 Evaluation Metrics

The study measures the following metrics:

1. Classic throughput (bps)
2. Effective throughput (bps)
3. Hit deviation
 - Along-axis deviation ($^{\circ}$)
 - Across-axis deviation ($^{\circ}$)
 - Magnitude deviation ($^{\circ}$)
4. Body movement
 - Left/right hand cumulative travel
 - Left/right index finger cumulative travel
 - Head translation and rotation
5. Error rate
6. Movement time (MT)
7. Fitts' Law regression (R^2 , slope)
8. Learning trajectory (trial-by-trial throughput and MT)
9. Questionnaires
 - Preliminary questionnaire (see Appendix A)
 - NASA-TLX (per interaction technique)
 - Preference ranking (once after experiment)

Throughput and MT answer *how fast*; hit deviation answers *where*; body movement reveals physical effort. Questionnaires capture perceived workload and preference, which objective metrics miss.

4.2.6 Procedure

Pre-Study Setup

The developer has created a GDPR form for the test subject to fill out, noting collection of age and gender. After this, a preliminary questionnaire is taken on the developer's PC, while the developer disengages from the participant to give them privacy to fill it out without the Hawthorne effect [76]. The questionnaire denotes

the test subject’s state of being (e.g., headache or nausea), their age, XR experience, (tested) eye/hand dominance and use of glasses in general and within the headset. Then they are asked to put on the head-mounted display.

The experiment has been primed with a specific order of techniques using the study participant ID. This is simply the number of participants that has come before them + 1. All questionnaires are fitted with a field for this to match the headset data with the questionnaires, and questionnaires with one another.

Calibration and Training

The developer ensures that the Meta Quest eye-tracking calibration has been completed, and then instructs the test subject to follow the instructions on display. They practise with the technique on 11 targets, and then move on to the real trial, given they are ready to do so, without the help of the cursors.

Task Execution

The trial proceeds as the test before it. When the test subject is done, the text instructs them to call over the experimenter to initiate the next stage, which is filling out a 7-point NASA-TLX about the technique with added questions about their opinion about the technique and their updated state of being. Completion of this repeats the steps from after the eye-tracking 4 times for a total of 5 techniques. Then, a final questionnaire records their preference ranking of all 5 techniques. No 2 techniques can share the same position on the ranking. They are finally asked to elaborate on why they picked their number one as they did, helping to decipher the raw numerical data.

Filtering

Selection errors were recorded and the next target appeared immediately; participants were not required to re-attempt misses. Outlier filters remove genuine artefacts while keeping natural misses in the endpoint distribution for effective width computation per ISO 9241-9. Spatial filters drop trials where effective amplitude is less than half the nominal amplitude or the hit point exceeds twice the target width from centre. A temporal SD filter drops trials where MT exceeds the per-technique mean by 2.5 standard deviations. Sanity bounds (MT 150–10 000 ms, ID 0.5–10 bits, TP 0.1–15 bps) catch remaining artefacts. Across all sessions, 23.8% of selection trials were removed (range 19.0% Gaze&Pinch to 27.9% HeadCrusher). The per-technique breakdown is shown in Table 4.1.

Pilot Studies To optimise the parameters of the techniques and iron out any oversights in the experiment, a pilot study was conducted, consisting of 1 external expert test subject performing the study, in combination with 3 internal experiments with an added alternative version per technique and comparing performance. We acknowledge that the greatest gain from this is from the autobiographical process more than statistical significance.

The table below (Table 4.2) showcases the differences between the 2 versions, with the chosen one highlighted in grey. The rationale behind the configuration

Table 4.1: Outlier filter removal rates per technique.

Technique	Raw trials	After filter	Removed (%)
Gaze&Pinch	1756	1422	19.0%
Gaze&Finger	1735	1260	27.4%
Gaze&Finger*	1744	1410	19.2%
HeadCrusher	1759	1269	27.9%
HeadCrusher*	1760	1309	25.6%
Total	8754	6670	23.8%

designs is entirely attributed to autobiographical design: what felt right between the developers.

Table 4.2: Interaction parameters across versions.

	Gaze & Pinch	Gaze & Finger	Headcrusher
Configuration 1	1€ filtered gaze input	1° threshold between finger and gaze input	Cursor freezes positionally when thumb and index finger are within 4.5 cm (released at 7 cm); 60 % biased towards the thumb.
Configuration 2	Raw gaze input	3° threshold between finger and gaze input	No freezing; 90 % biased towards the thumb.

A video recording of a full participant session, from calibration through task execution to post-technique questionnaires, is available at youtu.be/t1RpXop8voI.

4.3 Data Processing

4.3.1 Effective Throughput per ISO 9241-9

Effective metrics adjust task difficulty to match what the user actually did across a full experiment [58, pp. 770–772];[55, §7]. Per-sequence throughput computation is the only ISO-compliant method (see §2.2.5). A precise user clustering hits tightly within each target is solving a harder task than the nominal parameters describe; a sloppy user spraying hits across the boundary is solving an easier one. Effective width W_e , effective amplitude A_e , and effective throughput TP_E are computed as defined in §2.2.5 ((2.6), (2.7), (2.8), (2.9)). The constant $4.133 = \pm 2.066\sigma$ normalises effective width to a 4% error rate autocite[pp. 765–768]Soukoreff2004. Effective amplitude uses actual hit-to-hit vectors projected onto the task axis, and the first trial of each experiment is excluded per ISO 9241-9 [58, pp. 770–771].

4.3.2 Statistical Tests

RM-ANOVA assumes normality, sphericity, and interval-level measurement. VR pointing data frequently violate all three: throughput is right-skewed, error rates

are binomial and bounded at zero, and NASA-TLX ratings are ordinal [77]. The Friedman test handles one-way repeated measures but cannot test interactions. Our experiment crosses technique (5 levels) with depth, amplitude, and width, requiring factorial interaction tests without normality assumptions.

We use the Aligned Rank Transform (ART) as described in §2.2.6. Technique, depth, amplitude, and target size are treated as within-subjects factors. ART is recommended for VR interaction data because Fitts’ Law metrics, particularly throughput and error rate, are frequently non-normal [12]: an easy technique produces tightly clustered fast trials, while a difficult technique produces a long tail of slow trials interspersed with occasional fast ones. Standard ANOVA assumes normality; ART transforms the data to ranks before testing, making it resistant to the skew and heteroscedasticity that characterise pointing data. Sphericity violations get Greenhouse-Geisser correction. For main effects we report generalised eta-squared (η_g^2), following [64], who argued that partial eta-squared inflates effects and makes studies hard to compare across designs (small ≥ 0.02 , medium ≥ 0.13 , large ≥ 0.26). For interactions we also report partial η_p^2 ; the generalised measure divides by total variance including subjects, which can bury interaction terms in modest- n designs, and Wagner et al. [12] use η_p^2 , so sticking with it keeps the replication comparison honest. Post-hoc pairwise comparisons use Holm-Bonferroni correction.

We ran ART for each dependent variable (TP, MT, error rate, W_e , ID_e) in three models:

1. Technique $\times$ Depth (5×2 : 1 m, 3 m)
2. Technique $\times$ Amplitude (5×2 : 10° , 30°)
3. Technique $\times$ Width (5×2 : 5° , 8°)

Factors were not fully crossed because the design varies one at a time; each pair is a self-contained sub-experiment. ART was implemented via the `ARTool` R package (Kay et al.), following the procedure of Wobbrock et al. [63]. For Likert-scale subjective measures such as NASA-TLX workload ratings, the non-parametric Friedman test with Conover post-hoc comparisons (Holm-Bonferroni corrected) replaces ANOVA.

4.3.3 Validation

Before aggregation, the Python pipeline checks every session: all required event types present, timestamps monotonic, block and experiment IDs populated, and non-Fitts event ratio within bounds. Movement time is computed independently in C# and Python; divergence triggers investigation. Every filtered trial carries a quality flag that propagates through all downstream reports.

A three-way consistency check runs at the bottom of every aggregated report: the total trial count from the overview must equal the sum across per-session reports and the sum across per-technique groups. Any mismatch flags a filtering or partitioning error before results are interpreted (see §4.3.1).

4.3.4 Analysis Pipeline

The Python pipeline runs quality checks, per-experiment diagnostics, per-session comparisons, cross-session aggregation, and statistical tests. All plots use a

colourblind-safe palette with colour-per-technique held constant across every figure. Per-session comparison grids show techniques as columns with four metric rows (TP, MT, error rate, hit deviation). Aggregated results pool all participants and include Fitts’ regression (MT vs. binned ID per technique) with per-participant overlays.

4.4 Study Results

4.4.1 Participants

Twenty participants ($n = 20$, university students and staff) completed the full protocol. We ran pilot sessions early on to dial in the technique parameters; one pilot hit a rendering bug where the virtual hand appeared in only one eye on the parallax-mitigation variant. We fixed it, folded the feedback into the design, and restarted participant numbering from ID 1. Nobody was excluded mid-block.

Simulator sickness checks before and after the session (dizzy, nauseous, eyes tired, headache; 1–5) barely shifted (range -0.09 to $+0.14$). Nobody quit from cybersickness.

All 20 sessions contribute trials from all five techniques ($n_{\text{trials}} = 6\,670$, range 378–400). Four sessions came in slightly under the nominal 400; the ISO 9241-9 first-trial exclusion strips trial 0 so block-transition timing doesn’t leak into movement time. The other 16 sessions hit the full 400. Everyone handed in complete questionnaire data: valid NASA-TLX and ranking submissions from all 20.

Table 4.3: Participant demographics ($n = 20$).

Characteristic	Category	n
Sex	Male	12
	Female	8
Handedness	Right	20
	Left	0
Eye dominance	Right	18
	Left	2
VR experience	Never used	10
	A few times/year or monthly	7
	Daily	3
Age (years)	Range	20–52
	M (SD)	30.8 (10.4)

The study complied with the ethical guidelines of Aarhus University’s Department of Computer Science. All participants gave written informed consent before participation and were reminded of their right to withdraw at any time without consequence.

4.4.2 Statistical Methods

Shapiro-Wilk tests came back non-normal for most dependent variables: error rate in all 5 technique groups, hit deviation magnitude in all 5. Throughput was normal across the board, but we switched all measures to ART [63] with Greenhouse–Geisser correction across all quantitative measures. Main effects use η_g^2 (small ≥ 0.02 , medium ≥ 0.13 , large ≥ 0.26 [64]); interaction effects use η_p^2 , matching Wagner et al. [12]. Post-hoc pairwise comparisons with Holm-Bonferroni. NASA-TLX and ranking data: Friedman test with Conover post-hoc.

Table 4.4: ART RM-ANOVA results for all dependent variables.

Dependent Variable	$F(df)$	p	η_g^2	ε_{GG}
Effective Throughput	$F(4, 72) = 13.16$	< 0.001	0.268	0.616
Movement Time	$F(4, 76) = 44.45$	< 0.001	0.696	0.573
Error Rate	$F(4, 76) = 6.16$	< 0.001	0.235	0.789
Hit Deviation (Along)	$F(4, 76) = 37.70$	< 0.001	0.639	0.831
Hit Deviation (Across)	$F(4, 76) = 1.75$	.147	0.080	0.781
Hit Deviation (Magnitude)	$F(4, 76) = 4.28$	.004	0.172	0.834
Hand Movement	$F(4, 76) = 16.24$	< 0.001	0.425	0.687

4.4.3 Objective Measures

Task Completion Time

Technique had a large effect on movement time ($F(4, 76) = 44.45$, $p < .001$, $\eta_g^2 = 0.696$). Gaze&Pinch led (577 ms), Gaze&Finger trailed (1041 ms); full ordering in Figure 4.4. All pairwise contrasts were reliable except HC vs. HCP ($p = .157$).

Figure 4.4 also shows descriptive depth and amplitude panels; inferential tests for movement-time interactions were not part of this graph.

Throughput

Throughput differed across techniques ($F(4, 72) = 13.16$, $p < .001$, $\eta_g^2 = 0.268$, large effect). Gaze&Pinch led (3.512 bps), Gaze&Finger was lowest (2.183 bps); see Figure 4.5. Post-hoc: Gaze&Pinch outperformed all others ($p < .01$, Holm-Bonferroni). No other pairwise contrast reached significance ($p > .05$), including GF vs. GFP ($p = .180$) and HC vs. HCP ($p = 1.000$).

Interaction analyses. Two-way RM-ANOVAs on TP_E found no Technique $\times$ Depth interaction ($F(4, 72) = 1.24$, $p = .302$, $\eta_g^2 = .007$). Depth by itself did nothing ($F(1, 18) = 0.02$, $p = .892$). Per-family 2×2 follow-ups confirmed the asymmetry: Gaze&Finger showed a main effect of parallax ($F(1, 19) = 7.22$, $p = .015$) with no

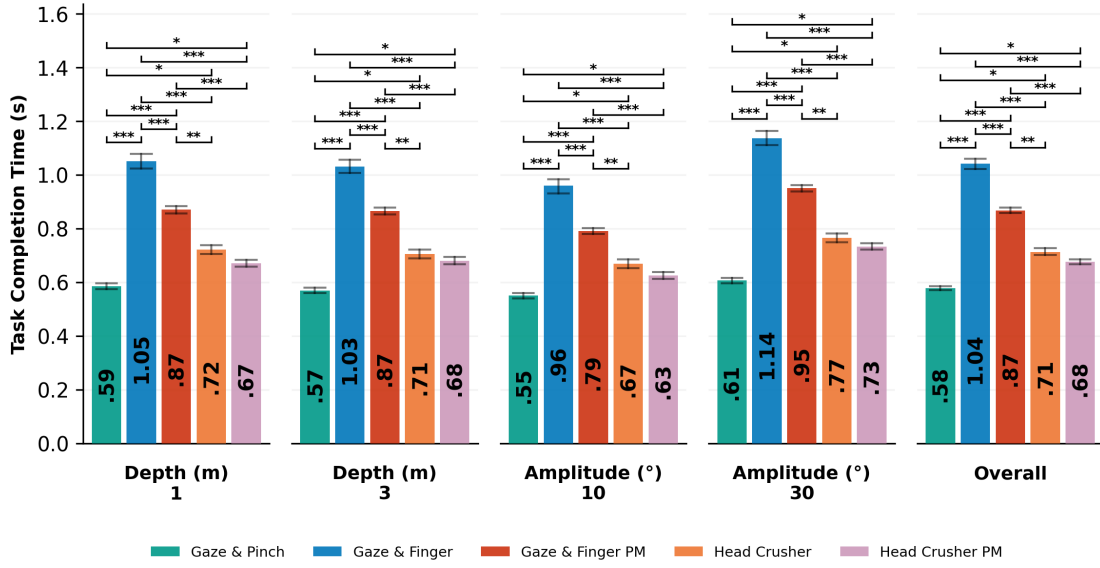


Figure 4.4: Task completion time by technique, shown per depth (1 m, 3 m), per amplitude (10°, 30°), and overall. Bars show means; whiskers are 95% confidence intervals. Significance: * $p < .05$, ** $p < .01$, *** $p < .001$.

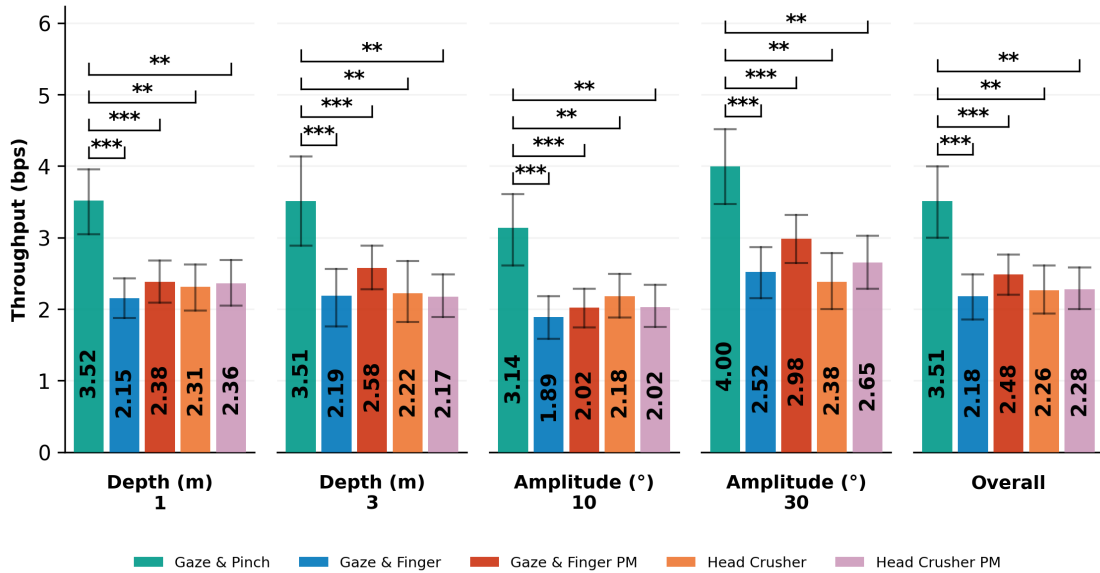


Figure 4.5: Effective throughput (ISO 9241-9) by technique, per depth, per amplitude, and overall.

depth interaction ($p = .222$); HeadCrusher showed no effect of parallax ($p = .953$) or depth ($p = .083$). Technique $\times$ Amplitude ($F(4, 64) = 3.30$, $p = .016$, $\eta_g^2 = .018$) and Technique $\times$ Width ($F(4, 72) = 5.94$, $p < .001$, $\eta_g^2 = .027$) interactions were both reliable. Generalized and partial η^2 are reported in Table 4.5.

Table 4.5: Interaction effects from two-way RM-ANOVA on effective throughput.

Interaction	$F(df)$	p	η_g^2	η_p^2
Technique $\times$ Depth	$F(4, 72) = 1.24$	.302	.007	.064
Technique $\times$ Amplitude	$F(4, 64) = 3.30$	.016	.018	.171
Technique $\times$ Width	$F(4, 72) = 5.94$	$< .001$	.027	.248
Per-family parallax $\times$ depth follow-ups:				
GF Parallax $\times$ Depth	$F(1, 19) = 1.60$	.222	.003	.077
HC Parallax $\times$ Depth	$F(1, 18) = 0.27$	.609	.002	.015

Parallax mitigation gain. Gaze&Finger gained +0.302 bps from parallax correction (+13.8%, per-family $p = .015$). HeadCrusher showed negligible change (+0.013 bps, +0.6%, $p = .953$). Table 4.6 shows the gap between families.

Table 4.6: Parallax mitigation gain by technique family. ΔTP_E = mitigated – vanilla.

Family	ΔTP_E (bps)	Gain (%)	p
Gaze&Finger (GF $\rightarrow$ GFP)	+0.302	+13.8	.015
HeadCrusher (HC $\rightarrow$ HCP)	+0.013	+0.6	.953

Nominal vs. effective throughput. $\Delta = TP_E - TP_N$ measures the precision bonus: how much tighter a participant’s hit spread is than the nominal target size would predict. Gaze&FingerParallax showed the largest positive delta ($\Delta = +0.283$ bps, +12.9%), followed closely by Gaze&Finger ($\Delta = +0.277$ bps, +14.5%). Gaze&Pinch also beat nominal ($\Delta = +0.156$ bps, +4.6%). Only the head-anchored techniques fell below nominal: HeadCrusher ($\Delta = -0.434$ bps, -16.1%) and HeadCrusherParallax ($\Delta = -0.467$ bps, -17.0%). Their wide endpoint scatter inflates W_e , dragging effective TP below nominal.

Effective Width

Effective width captures endpoint spread ($W_e = 4.133 \cdot \sigma_{\text{along}}$; (2.6)). Gaze&Finger was tightest ($W_e = 3.98^\circ$), HeadCrusher widest (5.27°, 32% spread); full ranking in Figure 4.6. The Heisenberg effect likely drives HC’s spread: the pinch mechanically disturbs the thumb–index ray [45, pp. 1–10].

Parallax mitigation narrowed things: HCP brought along-axis mean near zero (-0.07°) vs. HC’s overshoot ($+0.16^\circ$). Hit deviation magnitude differed ($F(4, 76) =$

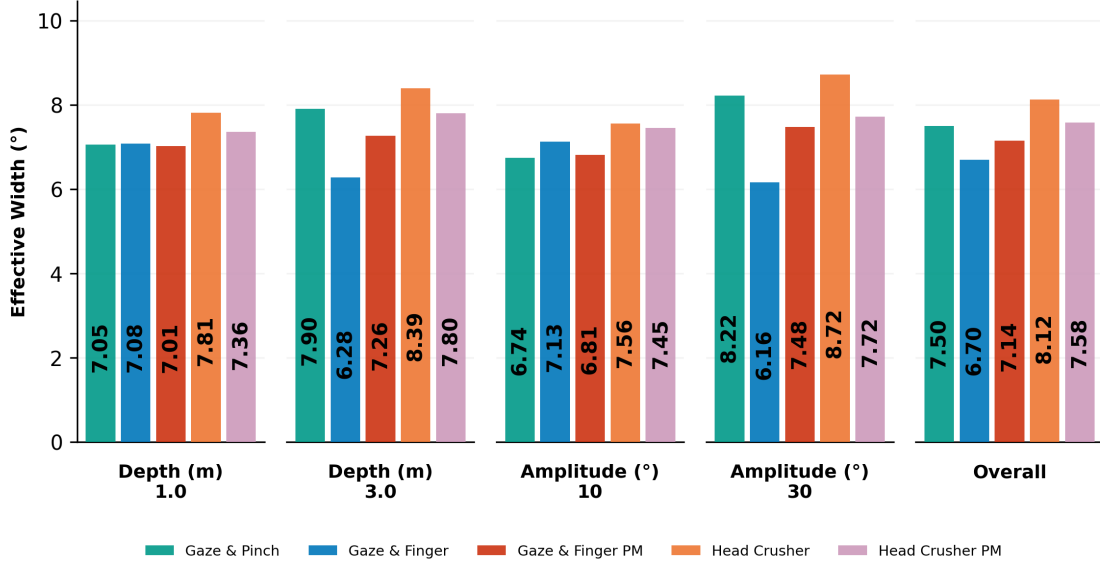


Figure 4.6: Effective target width $W_e = 4.133 \times \sigma_{\text{along}}$ by technique.

4.28, $p = .004$, $\eta_g^2 = 0.172$), and along-axis deviation showed a large technique effect ($F(4, 76) = 37.70$, $p < .001$, $\eta_g^2 = 0.639$). HC was the only overshooting technique; all others undershot (GP most: -1.17°).

Error Rate

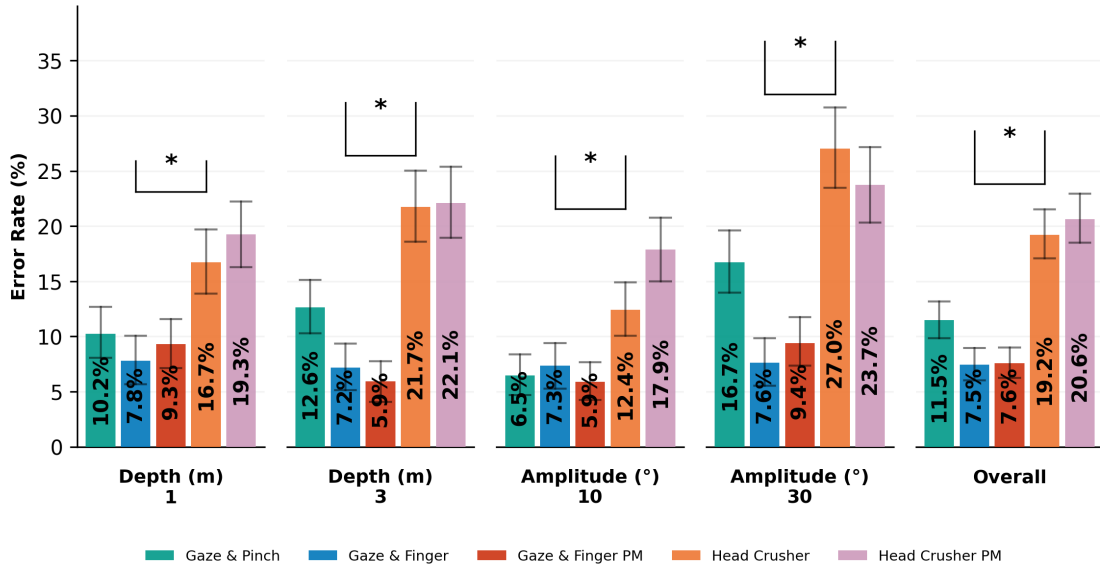


Figure 4.7: Error rate by technique, per depth, per amplitude, and overall.

The overall miss rate was 13.2% across 6670 trials, with a medium technique effect ($F(4, 76) = 6.16$, $p < .001$, $\eta_g^2 = 0.235$). Head-anchored techniques averaged 19.9% errors, gaze-anchored 8.9%—a stark split. But the post-hoc paints a murkier picture: only Gaze&Finger vs HeadCrusher cleared the Holm-Bonferroni threshold ($p = .028$). Gaze&Finger vs HeadCrusherParallax ($p = .063$), Gaze&FingerParallax

vs HeadCrusher ($p = .051$), and Gaze&FingerParallax vs HeadCrusherParallax ($p = .051$) all missed it. The omnibus says technique matters; the pairwise cannot tell us exactly where. Gaze&Finger was most accurate (7.5 %), HeadCrusherParallax least (20.6 %); see Figure 4.7.

Error patterns. Errors clustered late in blocks (fatigue). 5° targets produced $2.4\times$ more errors than 8° targets. Depth had negligible effect.

Hand Movement

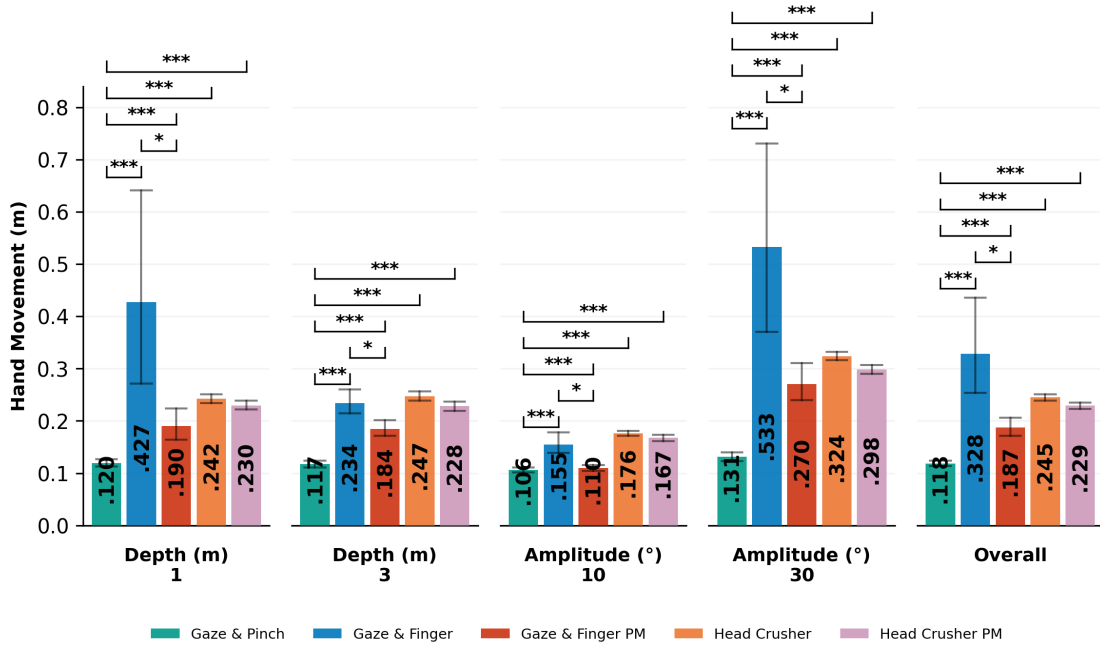


Figure 4.8: Mean per-trial index-fingertip displacement by technique.

Hand movement showed a large technique effect ($F(4, 76) = 16.24$, $p < .001$, $\eta_g^2 = 0.425$). Gaze&Pinch required minimal motion (0.085 m/trial), Gaze&Finger the most (0.194 m); full distribution in Figure 4.8. Gaze&Pinch was reliably lower than all others (all $p < .001$).

Fitts' Law Models

We ran Fitts' Law regressions at the condition-mean level using the model (2.5) (see §2.2.5). Table 4.7 gives intercept, slope, and fit per technique; Figure 4.9 shows the regression lines.

Gaze&Pinch was nearly flat ($b = 0.10$ s/bit, MT ranged 542–672 ms across 1.61–2.79 bits): gaze saccades are ballistic and pinch duration is fixed, so ID contributes almost nothing to MT. Gaze&Finger showed the steepest slope ($b = 0.53$ s/bit, MT 1380–2186 ms), reflecting the motor cost of arm movement through the image plane. The head-anchored techniques fell between ($b = 0.21$ – 0.27 s/bit), confirming that head pointing carries a measurable but moderate ID-dependence.

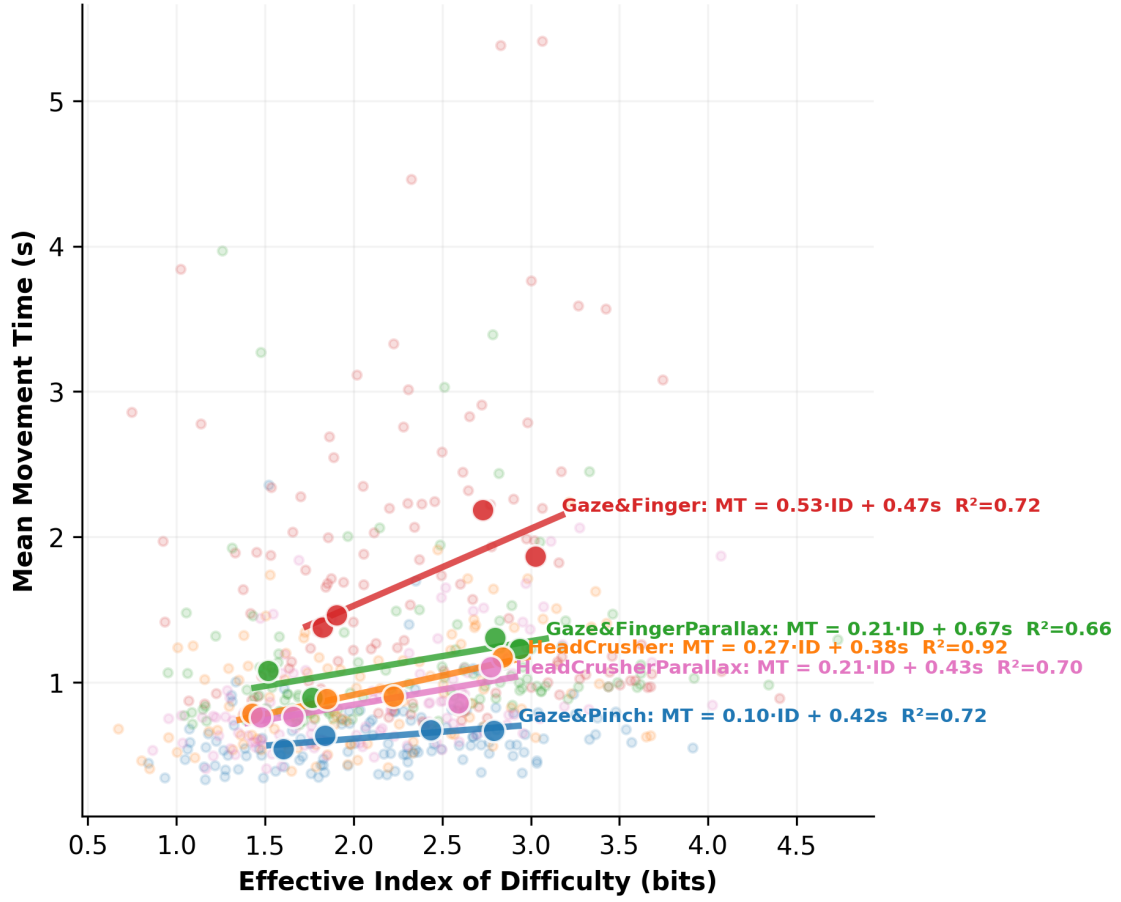


Figure 4.9: Fitts' Law regression at condition-mean level. $MT = a + b \cdot ID_e$.

Table 4.7: Fitts' Law regression parameters ($MT = a + b \cdot ID_e$).

Technique	a (s)	b (s/bit)	R^2
Gaze&Pinch	0.42	0.10	0.72
Gaze&Finger	0.47	0.53	0.72
Gaze&FingerParallax	0.67	0.21	0.66
HeadCrusher	0.38	0.27	0.92
HeadCrusherParallax	0.43	0.21	0.70

4.4.4 Subjective Measures

NASA TLX

Friedman tests were significant on the omnibus for all seven NASA-TLX subscales and RTLX (all $p < .05$; Table 4.9). Raw TLX: $\chi^2(4) = 33.15$, $p < .001$, $W = 0.414$. Frustration showed the widest spread ($\chi^2(4) = 49.58$, $p < .001$, $W = 0.62$); Gaze&Finger frustrated participants the most (Mdn = 5.0). Physical Demand was lowest for Gaze&Pinch (Mdn = 1.0). Eye strain ran opposite: Gaze&Pinch scored highest (Mdn = 3.0), but Holm-Bonferroni-corrected post-hoc tests found no reliable pairwise differences on eye strain. Looking takes no arm effort but strains the eyes.

Table 4.8: NASA-TLX subscale medians by technique (1–7 scale, 7 = high). RTLX = mean of per-participant unweighted subscale means.

	RTLX	Mental	Physical	Temporal	Perform.	Effort	Frustr.	Eye Str.
GP	2.17	2.0	1.0	2.0	3.0	1.5	1.0	3.0
GF	3.59	2.5	3.0	3.0	4.0	4.0	5.0	3.0
GFP	2.92	2.0	3.0	3.0	3.0	3.0	3.0	2.5
HC	2.90	2.0	3.0	2.0	4.0	3.0	3.0	2.0
HCP	2.34	2.0	3.0	2.0	3.0	2.0	2.0	2.0

Table 4.9: Friedman test results for NASA-TLX subscales and RTLX ($n = 20$, $df = 4$). Kendall’s W for effect size.

Subscale	χ^2	p	W	Sig.
Mental	19.18	< .001	0.240	*
Physical	34.80	< .001	0.435	*
Temporal	10.34	.035	0.129	*
Performance	14.45	.006	0.181	*
Effort	24.31	< .001	0.304	*
Frustration	49.58	< .001	0.620	*
Eye Strain	11.94	.018	0.149	*
RTLX	33.15	< .001	0.414	*

Post-hoc: Conover test with Holm-Bonferroni correction. Only Frustration (8 pairs), Effort (4), Physical (4), Mental (2), Performance (2), and RTLX (8) retained significant pairwise differences after correction. Temporal and Eye Strain had no significant pairs post-correction.

User Preferences

Friedman test on preference ranks: $\chi^2(4) = 21.00$, $p = 0.0003$, Kendall’s $W = 0.263$. At the extremes, what people liked matched what performed: Gaze&Pinch won on both counts, Gaze&Finger lost on both.

User Feedback

Physical fatigue. HeadCrusher caused the most fatigue (gorilla arm); participants supported their dominant arm mid-block (P9, P17, P19, P20). Gaze&Finger produced milder shoulder tiredness. Gaze&Pinch drew no fatigue complaints, matching its low physical demand scores (Table 4.8).

Table 4.10: Participant preference rankings (1 = best, 5 = worst).

Technique	Mean Rank	#1 Votes
Gaze&Pinch	1.80	12
Gaze&Finger	4.00	2
Gaze&FingerParallax	2.95	2
HeadCrusher	3.40	1
HeadCrusherParallax	2.85	3

Gaze&Pinch. Fast to learn: experienced users found it natural, novices picked it up quickly. P19: "Love gaze and pinch"; P8 called it fastest and most precise. No one needed hover feedback, unlike HeadCrusher. The main issue was gaze–pinch desynchronization during target transitions, occasionally causing misselection (see Limitations).

Gaze&Finger. Inexperienced users struggled with the dual-ray mental model (P8, P11, P15, P17, P18). Many tried to "tap" the target as if it were touch-based. The selection trigger felt inconsistent: gaze was on target but the angle threshold wasn't met. Midas Touch caused accidental activations during transitions; P8 restarted training by mistake. Our implementation omitted the 150 ms dwell from prior work, likely making this worse (see Limitations).

Gaze&Finger Parallax Mitigated. Clear qualitative–quantitative alignment: participants found aiming easier and took fewer breaks. P8 felt it easier to aim; P15 improved speed and reduced hovering episodes. Small-target frustration remained. Midas Touch less frequent but not eliminated.

HeadCrusher. Most users could not form a stable mental model of the cursor (P6, P9, P10, P11). Some used hover feedback as a "hunt" strategy, sweeping their hand until the target reacted; feedback itself became part of aiming. Fatigue was common, but the deeper problem was poor mapping transparency between hand posture and target hover.

HeadCrusher Parallax Mitigated. Improvement smaller than for Gaze&Finger. Some noticed no difference (P20). P6 still had misselections without a clear mental model. Reduced visible frustration but did not resolve the core interaction problem that parallax correction fixed for Gaze&Finger.

Chapter 5

Overall Discussion

The toolkit and the empirical study were presented separately, but their value is tighter than that separation suggests. This chapter synthesises both halves and answers the research questions through the lens of the data. Broader implications, design recommendations, and contributions are consolidated in the Conclusion (Chapter 6).

5.1 RQ1: Generalizing XR Experiment Patterns with a Node-Graph Toolkit

RQ1 asks to what extent a reusable node-based toolkit can support the construction, modification, and execution of multiple XR interaction techniques within the same experimental framework. The answer is a node-graph architecture that decomposes interaction logic into reusable, composable building blocks running inside that framework.

5.1.1 What Patterns Were Generalised

Five core patterns recur across XR interaction experiments regardless of technique or task:

- **Interaction backbone.** Every pointing technique follows the same progression: define a ray, detect hover over targets, optionally confirm intent, execute selection, and optionally drag. This sequence is universal across gaze, hand, and head techniques [37].
- **Node roles.** The building blocks split into five responsibilities: **Source** (entry point for runtime data), **Transform** (process input), **Stage** (meaningful interaction step), **Effect** (visual/behavioural feedback), and **Bridge** (connection to environment logic). Every technique uses the same set of roles with different internal logic.
- **Two-level abstraction.** Macro nodes represent high-level interaction concepts (ray, hover, selection); micro nodes handle the detailed internal behaviour. This lets researchers work at the concept level while keeping fine-grained control available when needed.

- **Environment separation.** The interaction logic and the scene-specific response are decoupled through an environment bridge. The backend decides *what* the current state is; the environment decides *what happens next*. This separation means the same technique graph can drive different experimental tasks without modification.
- **Integrated data collection.** Logging, metric computation, and analysis pipeline are part of the toolkit, not add-ons. Per-frame tracking data, trial events, and Fitts' Law metrics are captured automatically once the technique is configured.

5.1.2 How the Node-Graph Captures These Patterns

Each backbone stage maps to a macro node with configurable ports and internal micro logic. To build a technique, the researcher connects macro nodes in the expected sequence, configures parameters (e.g. ray origin, trigger gesture, selection strategy), and connects the bridge node to the environment. The graph is authored and edited inside VR through the frontend interface, then executed by the backend at runtime. A mode switch toggles between editor and test without leaving the headset.

This produces three concrete time savings:

- **No rewriting common infrastructure.** The backbone structure, hover detection, logging, and environment bridging are provided. The researcher implements only the technique-specific logic inside relevant nodes.
- **Runtime iteration.** Parameter changes and node reconfigurations take effect without restarting the application. This cuts iteration cycles from minutes to seconds.
- **Reusable across experiments.** Once a technique is defined as a graph, it can be saved, loaded, and connected to different environments without modification. The same Gaze&Finger graph drives a Fitts' Law task, a free exploration scenario, or a custom evaluation.

5.1.3 Trade-offs and Remaining Challenges

The generalisation is not free. Three limitations qualify the time-savings claim:

- **Learning curve.** The node-graph paradigm requires researchers to think in terms of dataflow composition rather than imperative scripts. This may slow initial adoption, especially for developers unfamiliar with visual programming.
- **Performance overhead.** Graph traversal and runtime data propagation add CPU cost compared to hand-optimised monolithic scripts. On Quest hardware, the toolkit runs at a stable 72 fps with medium-complexity graphs, but overhead becomes measurable as the graph grows.
- **Extension depth.** Adding genuinely new node types (not just reconfiguring existing ones) requires Unity C# development. The toolkit reduces ground-work for standard experiment patterns but does not eliminate the need for coding in non-standard cases.

The study in this thesis (20 participants, 5 techniques, 2 depths, 2 amplitudes, 2 target widths) was built and executed entirely within the toolkit pipeline. The extensibility mechanisms mean that future experiments can build on the existing graph library rather than starting from zero. This addresses RQ1: the toolkit demonstrably supports construction, modification, and execution of techniques within a single framework. However, the most important piece of unfinished work is a formal usability evaluation of the toolkit itself with external users; currently, all evidence of time savings comes from our own development experience.

5.2 RQ2: Parallax Mitigation Impact on Plane-Based Techniques

RQ2 asks two things: (a) how much parallax mitigation affects plane-based techniques, and (b) how much these techniques improve compared to the Gaze&Pinch baseline.

5.2.1 Quantified Effect on Performance

Parallax mitigation improved Gaze&Finger substantially but HeadCrusher only marginally.

Table 5.1: Parallax mitigation gains. Δ = mitigated – vanilla.

Family	Metric	Vanilla	Mitigated	Δ	p	η_p^2
Gaze&Finger	TP (bps)	2.183	2.485	+0.302	.180 ^a	.354
	MT (ms)	1041	868	−173	< .001	.654
HeadCrusher	TP (bps)	2.265	2.278	+0.013	1.000	.012
	MT (ms)	713	676	−37	.157	.162

^aPost-hoc in 5-technique model (n.s.); per-family main effect of parallax is significant: $F(1, 19) = 7.22$, $p = .015$. η_p^2 = partial eta-squared from per-family RM-ANOVA (vanilla vs. mitigated, collapsing across conditions). HeadCrusher TP_E: $F(1, 18) = 0.004$, $p = .953$; MT: $F(1, 19) = 3.67$, $p = .070$.

Gaze&Finger gained +0.302 bps (per-family $p = .015$) and lost 173 ms MT, while HeadCrusher gained +0.013 bps ($p = .953$, n.s.) with 37 ms reduction in movement time. The TP benefit was therefore $> 23\times$ greater for the two-rayed Gaze.

The difference in statistically proven impact may be due to another, overpowering source of error in the selection data. HeadCrusher suffers from significant Heisenberg effect, as the selection and aiming is tied together in the hand. Selecting impacts aim which adds noise to the selection accuracy. This claim that a big part of the error coming from this design tracks with the literature regarding Heisenberg being responsible for 30.45% of pointing errors in ballistic trials of ISO9241-9[45]. The story then becomes that the statistical insignificance was derived from noise in the selection data, while the magnitude of the parallax correction impact may have been dampened by the feedback that was provided. The test subjects had no way of knowing if their invisible crosshairs were hovering the black circle in the centre of the targets by other means than intuition, making the dominant decision driver

for committing to the selection the target highlight feedback. Moreover, parallax would in this case force the user to choose between one of the two hand images, but since the targets are highlighted, they always choose correctly, with a secondary effect being confusion and uncertainty regarding the exact horizontal position of the vertical line that represents the horizontal position of the crosshairs. This secondary error is what we see driving the reduction in the mean of the throughput, though more people are needed to verify that it indeed is a reduction. So to summarize part of the error comes from mistaking one hand from the other (solved here) while a source of a smaller error is that of the visual confusion overhead of seeing 2 images, concentrating on neither. The noise and the predominant error, we think, come from the Heisenberg effect.

On the other hand, a greater percentage of the error of gaze and finger derives from the first problem - which fingertip represents my cursor? The question of where on the correct finger the cursor is trivial in comparison to HeadCrusher’s situation. This first error is further exacerbated by the finger being a narrow pointer, meaning that you are required to be very precise with where on the image plane the fingertip lines up with the target. Together, Gaze&Finger gains more from the parallax correction in its current implementation.

Under per-condition ISO throughput the Technique $\times$ Depth interaction no longer reaches significance ($F(4, 72) = 1.24, p = .302$). The per-family analyses still show the asymmetry: Gaze&Finger parallax is depth-invariant ($p = .222$ for the interaction), while HeadCrusher parallax shows no effect at either depth ($p = .953$). The practical takeaway is that parallax mitigation should not be applied uniformly: two-ray architectures benefit, single-ray techniques do not.

5.2.2 Gap to Gaze&Pinch Baseline

Gaze&Pinch led throughput and movement time: 3.512 bps and 577 ms, while requiring only 0.085 m/trial hand movement and producing the lowest RTLX (2.17). It outperformed every other technique in post-hoc throughput comparisons ($p < .01$ after Holm-Bonferroni). The RM-ANOVA confirms a large technique effect on throughput ($F(4, 72) = 13.16, p < .001, \eta_g^2 = 0.268$) and a large effect on MT ($F(4, 76) = 44.45, p < .001, \eta_g^2 = 0.696$).

The gap is architectural, not geometric. Gaze&Pinch splits pointing from confirmation: eye movements are ballistic saccades (~ 30 – 120 ms regardless of distance), while head and hand movements need voluntary muscle activation and MT scales with amplitude. The Fitts’ Law slopes quantify this (§4.4.3): Gaze&Pinch’s slope of 0.10 s/bit is near-zero; adding a whole bit of ID costs only 100 ms. Gaze&Finger’s slope of 0.53 s/bit is over $5\times$ higher, reflecting the motor cost of arm movement. Even on wide-FOV hardware, hand pointing carries a substantial ID-dependence that pure gaze pointing avoids.

5.2.3 Speed–Accuracy Tradeoff

Gaze&Pinch had 11.5% errors, $1.5\times$ more than Gaze&Finger (7.5%, most accurate). HeadCrusher variants both exceeded 19%, with HeadCrusherParallax highest at 20.6%. The fastest technique was not the most accurate; the most accurate was not the fastest.

ISO 9241-9 effective TP partially resolves this through the W_e adjustment: techniques with wide endpoint scatter are penalised. The precision bonus $\Delta = TP_E - TP_N$ quantifies this. Gaze&FingerParallax gains +0.283 bps under the effective adjustment (+12.9%, endpoint spread tighter than the target); Gaze&Finger gains +0.277 bps (+14.5%, $W_e = 3.98^\circ$, tightest spread); Gaze&Pinch gains +0.156 bps (+4.6%, $W_e = 4.96^\circ$, wider scatter). Only the head-anchored techniques lose ground: HeadCrusher loses -0.434 bps ($W_e = 5.27^\circ$, widest spread) and HeadCrusherParallax loses -0.467 bps. The adjustment is largest for the two head-anchored techniques, confirming that their mid-range nominal TP masks substantial precision costs. Which trade-off to prioritise depends on the application: precision-critical tasks (e.g., dense UI, small targets) favour the Gaze&Finger family; speed-critical tasks favour Gaze&Pinch.

5.2.4 Interaction Effects

Three interaction effects reached significance and contextualise the main findings:

- **Technique $\times$ Depth** ($F(4, 72) = 1.24$, $p = .302$, n.s.): The interaction no longer reaches significance under per-condition ISO throughput. Per-family follow-ups confirm a depth-invariant parallax benefit for Gaze&Finger ($p = .222$ for the parallax $\times$ depth interaction). HeadCrusher showed no depth effect ($p = .083$).
- **Technique $\times$ Amplitude** ($F(4, 64) = 3.30$, $p = .016$, $\eta_p^2 = .171$): The throughput gap between techniques widened at larger amplitudes. Head-anchored techniques were disproportionately slower at 30° than at 10° , consistent with the motor component of Fitts' Law being larger for head and hand movement than for gaze saccades.
- **Technique $\times$ Width** ($F(4, 72) = 5.94$, $p < .001$, $\eta_p^2 = .248$): Smaller targets (5°) accentuated technique differences more than larger targets (8°). This interaction is reliable and suggests that technique comparisons conducted at a single target width may underestimate differences at smaller (more demanding) sizes.

These interactions caution against interpreting main effects in isolation: each technique's profile depends on the specific combination of depth, amplitude, and width at which it is evaluated.

5.3 Comparison with Wagner et al. 2023

We partially replicate Wagner et al. [12]. The ordinal ranking of the three shared techniques replicated: both studies find Gaze&Pinch fastest and Gaze&Finger slowest. Wagner et al. showed that depth-dependent parallax degrades HeadCrusher performance. Our data show the complement: mitigation helps GazeFinger more. Our absolute throughput numbers are $1.70\text{--}1.72\times$ higher, but the relative ordering holds.

The $\sim 1.7\times$ gap in absolute throughput comes from multiple confounds: angular Fitts' formulation, wider targets ($5^\circ, 8^\circ$ vs. 3°), raw gaze input vs. fixation-filtered,

Table 5.2: Throughput comparison and confounds with Wagner et al. (2023).

Technique	Present study (Quest Pro, raw gaze)	Wagner et al. (HoloLens 2, fixation filter)	Ratio
Gaze&Pinch	3.512	2.06	1.70×
Gaze&Finger	2.183	1.86	1.17×
Gaze&FingerParallax	2.485	-	-
HeadCrusher	2.265	1.32	1.72×
HeadCrusherParallax	2.278	-	-

Confounds: target sizes 5°, 8° (us) vs 3° (them). Gaze: raw (us) vs fixation filter 150 ms (them). Amplitude: {10°, 30°} vs {10°, 20°, 30°}. Depth: {1, 3}m vs {0.5, 1.25, 2.0}m. Hardware: Quest Pro vs HoloLens 2. Outlier filtering: our pipeline removes 23.8% of trials.

parallax mitigation, and more aggressive outlier filtering (23.8% vs. ~3.7% removed). Each of these pushes our numbers upward independently, and their combined effect easily accounts for the ratio.

The most consequential difference is in the Fitts' Law slopes. Gaze&Pinch retained a near-flat slope (0.10 s/bit vs. Wagner's 0.35 s/bit), consistent with wide-FOV hardware eliminating the need for head rotations. HeadCrusher was shallower as well (0.27 vs. 0.49 s/bit). Gaze&Finger, however, showed a steeper slope (0.53 s/bit vs. Wagner's 0.35 s/bit): on wide-FOV hardware, gaze pointing removes amplitude-dependence, but the hand movement required for finger confirmation still carries a motor cost that scales with ID. HoloLens 2's narrow FOV ($43^\circ \times 29^\circ$) forces head rotations when targets are separated by 20° – 30° , while Quest Pro's wide FOV ($\sim 110^\circ$ horizontal) keeps the entire 30° stimulus array within a single fixation.

This hardware dependence has a methodological implication for the field. The Wagner experiment was conducted on a device whose FOV is smaller than the task space, making head rotation a de facto part of every technique. Our experiment keeps the stimulus array within the device's FOV. Neither is "wrong," but a technique's measured Fitts' Law slope is partly a property of the hardware it is evaluated on, not just the technique itself. Future VR pointing studies should report both field of view, the angular extent of the stimulus array, and hardware tracking characteristics to enable meaningful cross-study comparison.

Table 5.3 consolidates the key performance and subjective metrics across the three techniques shared with Wagner et al.

Across all metrics the pattern holds: Gaze&Pinch is the fastest and most preferred technique in both studies. Gaze&Finger is the most accurate but slowest of the gaze-assisted techniques. The HeadCrusher family incurs the highest error rates, while Gaze&Finger produces the highest workload. The absolute throughput gap (ours 1.70–1.72× higher) stems from wider targets, raw gaze, and our more aggressive outlier pipeline. The Fitts' Law slope differences reflect hardware FOV: Gaze&Pinch and HeadCrusher are shallower on Quest Pro (0.10 and 0.27 s/bit vs. 0.35 and 0.49 s/bit), while Gaze&Finger is steeper (0.53 vs. 0.35 s/bit) due to the motor cost of hand confirmation that wide FOV does not eliminate.

Table 5.3: Multi-metric results comparison with Wagner et al. (2023).

Metric	Gaze&Pinch		Gaze&Finger		HeadCrusher	
	Ours	Wagner	Ours	Wagner	Ours	Wagner
Throughput (bps)	3.512	2.06	2.183	1.86	2.265	1.32
Movement Time (ms)	577	1650	1041	1570	713	2270
Error Rate (%)	11.5	13.5	7.5	11.9	19.2	38.0
Eff. Width W_e (°)	4.96	3.16	3.98	6.20	5.27	3.62
Hand Movement (m)	0.085	0.10	0.194	0.29	0.245	0.19
Fitts' Slope (s/bit)	0.10	0.35	0.53	0.35	0.27	0.49
#1 Preference (n)	12	7	2	2	1	1
RTLX / Phys. Demand	2.17	2.0 ^a	3.59	5.0 ^a	2.90	6.0 ^a

^aWagner et al. report NASA-TLX Physical Demand (median, 7-point scale). We report RTLX (unweighted mean of 6 subscales, 7-point scale). Lower = better for both. Wagner did not report overall RTLX; Physical Demand is the closest single-subscale proxy.

5.4 Limitations and Future Considerations

This section discusses the main limitations of both the toolkit and the Fitts' Law evaluation built around it, along with the natural next steps each limitation implies. A consolidated future work agenda is presented in Chapter 6.

5.4.1 Toolkit Limitations

The toolkit lacks a formal user-based evaluation. The current evidence comes from a self-conducted walkthrough (useful as a first verification step, but insufficient to support claims about time savings or usability relative to established alternatives). A developer experiment comparing time-to-implement, error rates, or qualitative feedback from external users would validate the system more convincingly and surface adoption barriers not visible from self-use.

Two implementation limitations were observed during development. First, the CD gain and One Euro filter parameters are not currently modifiable, producing a "selfie-stick" effect when dragging nodes in depth. Making angle-based dragging thresholds, gain values, and filtering parameters configurable would improve both comfort and precision. Second, complex MacroNode graphs become difficult to inspect in the current interface: a MacroNode can contain substantial internal structure while appearing as a single node with scrollable content. A zoom or drill-down technique for inspecting contained micro nodes as a subgraph would make complex graphs easier to read and reduce the need to overload a single node representation.

5.4.2 Fitts' Law Evaluation Limitations

Fatigue. The study procedure requires participants to use one technique continuously once a block has started, without internal breaks. This risks fatigue affecting later techniques more than earlier ones. While counterbalancing mitigates systematic bias, some performance drop may reflect tiredness rather than technique differences. Future designs could shorten blocks, insert controlled rest moments, or split the evaluation across sessions.

Narrow independent variable range. Two values each for amplitude, width, and depth sufficed to detect technique differences but represent a small slice of the interaction space. More values, particularly smaller widths below 3° where technique differences may be amplified, would strengthen conclusions about where a given technique breaks down or stabilises.

Learning transfer. Some participants reported recognising the similarity between parallax-mitigated and vanilla variants despite counterbalancing. This is a confound for the intended unbiased comparison. A between-subjects design testing vanilla and mitigated versions in separate participant groups would provide cleaner evidence, though at the cost of statistical power. The order effect was not analysed quantitatively; doing so (testing for an order $\times$ technique interaction) would indicate how serious this confound is in the current data.

Hover feedback bias in HeadCrusher. Some participants used hover feedback strategically: they moved the hand around the target area and stopped once the hover effect activated. This means the evaluation did not test HeadCrusher purely on its intended interaction principle; visual feedback compensated for one of its core difficulties. Future studies should reconsider when and how much hover feedback is shown for techniques where spatial precision itself is the variable of interest.

Participant diversity. The sample ($N = 20$, mean age 30.8, range 20–52, 12 male/8 female, 18 right-eye dominant/2 left-eye dominant) provides limited information about generalisability across age, eye dominance, hand preference, and accessibility needs. Both gaze and hand interaction are affected by these factors.

Controlled task vs. real use. A Fitts' Law task with repeated target selection in a circular layout is cleaner than building, reading, and editing node graphs in practice. The study tells us about relative selection performance but does not capture how the techniques would feel in longer, messier real interaction scenarios. This gap between controlled measurement and practical usage is common in the literature, but should be acknowledged explicitly.

Chapter 6

Conclusion

6.1 Research Questions Restated

This thesis addressed two questions. RQ1 asked to what extent a reusable node-based toolkit can support the construction, modification, and execution of multiple XR interaction techniques within the same experimental framework. RQ2 asked how much parallax mitigation affects plane-based selection techniques and how much the mitigated versions improve compared to a Gaze&Pinch baseline.

Note that the Introduction (Chapter 1) originally separated RQ2 into two sub-questions: comparing mitigated techniques against a non-parallax control (Gaze&Pinch) and comparing mitigated techniques against their vanilla counterparts. The present treatment consolidates both comparisons under RQ2, which the empirical design addresses jointly.

6.2 Summary of Findings

RQ1: Toolkit support for technique construction, modification, and execution. We built a modular node-graph toolkit that decomposes interaction logic into reusable building blocks with five core roles (Source, Transform, Stage, Effect, Bridge), a universal interaction backbone (Ray → Hover → Commit → Selection → Dragging), and two-level abstraction (macro nodes for concepts, micro nodes for detail). The architecture supports construction of new techniques by connecting macro nodes, modification by reconfiguring nodes at runtime, and execution of multiple techniques within the same experimental framework.

A Fitts' Law experiment with 5 techniques, 20 participants, 6 670 trials across 2 depths, 2 amplitudes, and 2 target widths, with full ISO 9241-9-compliant effective throughput computation, was built and executed entirely within the toolkit. Without it, comparable infrastructure would require rebuilding logging, trial management, and analysis from scratch for each study: the "generalized experiment pattern" the Introduction (§1.5) promised.

The toolkit was not formally evaluated with external users. While our own experience confirms iteration time savings (parameter changes take effect without restarting the application), three qualifications apply: (i) the node-graph paradigm carries a learning curve for developers unfamiliar with visual programming; (ii) graph traversal adds CPU cost compared to hand-optimised monolithic scripts, though the toolkit runs at a stable 72 fps on Quest hardware at medium-complexity; (iii) adding

genuinely new node types requires Unity C# development. These trade-offs do not invalidate the claim, but the most important piece of unfinished work is a controlled developer study measuring time-to-implement and error rates against a baseline.

RQ2: Parallax mitigation impact. Parallax mitigation helped Gaze&Finger: +0.302 bps (+13.8%, per-family $p = .015$) throughput and -173 ms MT ($p < .001$, $\eta_p^2 = 0.654$). The throughput gain was large ($\eta_p^2 = 0.354$) and reached significance in both the per-family analysis ($p = .002$) and the post-hoc comparison ($p = .015$). HeadCrusher showed negligible benefit: +0.013 bps (+0.6%, $p = .953$, n.s.) and -37 ms MT ($p = .157$, n.s.). The asymmetry follows from pointing architecture: two-ray techniques (Gaze&Finger) suffer a coordinate-frame parallax error that correction removes; single-ray techniques (HeadCrusher) have no such error. The practical lesson: parallax mitigation should not be applied uniformly – two-ray architectures benefit; single-ray techniques do not.

No mitigated technique approached Gaze&Pinch, which outperformed every other technique in post-hoc throughput comparisons. The gap is architectural (Gaze&Pinch decouples pointing from selection), not correctable by geometric fixes. For accuracy-critical tasks, Gaze&Finger (error rate 7.5% vs Gaze&Pinch’s 11.5%) is preferable despite lower throughput. The speed–accuracy tradeoff is real: the fastest technique was not the most accurate; the most accurate was not the fastest.

Replication of Wagner et al. (2023). We partially replicate Wagner et al. [12]. The ordinal ranking held: Gaze&Pinch leads on throughput (3.512 bps vs 2.06), Gaze&Finger trails (2.183 bps vs 1.86), HeadCrusher falls in between (2.265 bps vs 1.32). Our absolute values are 1.70 – $1.72\times$ higher due to wider targets ($5^\circ, 8^\circ$ vs 3°), raw gaze (vs 150 ms fixation filter), and a more aggressive outlier pipeline (23.8% removed vs Wagner’s $\sim 3.7\%$). The divergence in Fitts’ Law slopes (Gaze&Pinch and HeadCrusher are $3.5\times$ and $1.8\times$ shallower on Quest Pro; Gaze&Finger is steeper, 0.53 vs. 0.35 s/bit) reflects the Quest Pro’s wider FOV ($\sim 110^\circ$ horizontal) against HoloLens 2’s narrow FOV ($43^\circ \times 29^\circ$) – on modern hardware, pure gaze saccades cover the task space in near-constant time, though hand-confirmed techniques still carry a motor cost.

6.3 Contributions

This thesis makes the following contributions to XR interaction research:

1. **A reusable node-graph architecture for composing and evaluating XR interaction techniques.** The architecture identifies five core roles (Source, Transform, Stage, Effect, Bridge) and a universal interaction backbone that recur across all pointing techniques. It provides runtime-reconfigurable node pipelines, seamless switching between editing and testing within the headset, and a two-level abstraction that lets researchers work at the concept level while keeping fine-grained control. The toolkit reduces infrastructure overhead for new studies: logging, metric computation, and analysis are integrated rather than add-ons.

2. **An integrated experiment pipeline.** The pipeline connects interaction technique definition to experiment environment, logging, and ISO 9241-9-compliant analysis within a single framework. Interaction logic, environment response, and data collection are treated as connected parts of the same system rather than separate concerns.
3. **Empirical evidence for asymmetric parallax mitigation benefits.** A controlled user study ($n = 20$, 5 techniques, ISO 9241-9 Fitts' Law protocol) demonstrates that parallax mitigation substantially benefits two-ray techniques (Gaze&Finger: -173 ms MT, $p < .001$) but not single-ray techniques (HeadCrusher: -37 ms MT, n.s.). The effect is architectural, not geometric: designs should evaluate interaction techniques and rendering settings jointly.
4. **A replication and extension of Wagner et al. (2023) on wide-FOV hardware.** We confirm their ordinal ranking while showing that hardware (Quest Pro vs HoloLens 2) substantially changes absolute performance and Fitts' Law slopes – a caution for hardware-dependent generalisation claims in VR pointing research.
5. **Comprehensive subjective characterisation of five VR selection techniques.** RTLX, subscale-level TLX, preference rankings, and qualitative feedback characterising the speed–accuracy–fatigue–eye strain tradeoff space. Frustration emerged as the largest differentiator across techniques (Kendall's $W = 0.62$), and eye strain was inversely related to physical demand.

6.4 Practical Implications

Gaze&Pinch as default for speed. Gaze&Pinch led throughput (3.512 bps), MT (577 ms), and physical demand (RTLX 2.17). It was the most preferred technique (12 of 20 #1 votes). However, its error rate (11.5%) was $1.5\times$ higher than Gaze&Finger (7.5%), and participants reported the highest eye strain (Mdn = 3.0/7). If precision is critical, a parallax-mitigated two-ray technique is preferable despite being slower.

Parallax helps asymmetrically. Apply parallax correction to two-ray techniques (Gaze&Finger); do not expect gains for single-ray techniques (HeadCrusher). For two-ray techniques, project rays from the interocular midpoint.

Fatigue is not optional. Image-plane techniques cause gorilla arm. Gaze fixation strains the eyes. These are independent – the fastest technique causes the most ocular fatigue; hand-based techniques cause the most arm fatigue. Keep blocks to 5–10 min with breaks. Simulator sickness was negligible across all conditions, confirming these techniques are safe for sustained use.

6.5 Broader Implications

This thesis sits at a specific intersection: a tool-building contribution enables a controlled empirical comparison that would be harder without the tool. The node-

graph approach to technique authoring, combined with an integrated evaluation pipeline, offers a template for future XR interaction studies. Rather than rebuilding logging, metric computation, and analysis for each technique comparison, future work can reuse or extend this pipeline to new techniques, apparatus, or task domains. This aligns with a broader trend in HCI tool-building, from Myers et al.’s argument that UI software tools reduce repeated implementation work [66] to domain-specific XR authoring systems such as DART and AUIT [15, 38], and extends it specifically to the interaction-technique composition layer.

The parallax asymmetry finding has a broader lesson: rendering fixes for perceptual problems in VR are technique-dependent. A solution that helps one pointing modality may do nothing for another. Designers should evaluate interaction techniques and rendering settings jointly, not as independent parameters. More generally, our Fitts’ Law slope comparison with Wagner et al. (2023) suggests that hardware FOV fundamentally changes the relationship between task geometry and movement time in pure gaze-based interaction: on wide-FOV headsets, gaze selection becomes near-constant-time. Hand-confirmed techniques, however, retain an ID-dependence that wide FOV does not eliminate – a result that challenges the applicability of Fitts’ Law slopes as a comparative benchmark across different HMD hardware without explicit FOV control.

6.6 Future Work

Several open directions follow from this work, organised by priority:

Formal toolkit evaluation. The most important piece of unfinished work is a controlled developer study comparing time-to-implement, error rates, and qualitative feedback from external users. The current claim that the toolkit reduces experiment development time is supported only by our own experience.

Broader empirical sampling. Our sample ($n = 20$, university students, predominantly right-eye-dominant) constrains generalisability. The limited range of independent variables (2 depths, 2 amplitudes, 2 widths) covers only part of the interaction space. Extended sampling across age, hand preference, eye dominance, and accessibility needs would strengthen conclusions, as would a wider range of stimulus parameters to identify where specific techniques break down.

Learning transfer and technique ordering. Participants noted similarity between parallax-mitigated and vanilla variants despite counterbalancing. Future studies should separate similar techniques more strongly, either by session, by larger inter-block gaps, or by adding filler tasks.

Heisenberg effect quantification. The data pipeline already produces drift and jitter analyses for the head-anchored techniques. Quantifying how pinch-induced hand disturbance affects HeadCrusher’s error budget and whether the Heisenberg effect interacts with parallax mitigation is a natural next step.

Extending the toolkit to non-pointing tasks. The current graph library covers only selection techniques. Extending to navigation, manipulation, and system control would test the generality claimed in RQ1 more rigorously.

Hover feedback confound. Participants used hover feedback as a "hunt" strategy in HeadCrusher, which likely biased the condition in its favour. Future evaluations should control for the presence and timing of hover feedback to isolate the technique's intrinsic performance.

Toolkit quality-of-life improvements. Two improvements follow directly from development experience. Making CD gain, filtering parameters, and dragging thresholds configurable would reduce the "selfie-stick" effect when manipulating nodes in depth. A drill-down interface for inspecting macro node internals would make complex graphs easier to read and reduce the need to overload a single node representation.

6.7 Closing Reflection

The toolkit and the empirical study were presented separately in this thesis, but their value is tighter than that separation suggests. The toolkit's generalised experiment architecture made the five-technique Fitts' Law comparison feasible within a single framework, and the empirical findings, in turn, validated the toolkit's claim to support construction, modification, and execution of multiple interaction techniques. The parallax asymmetry result (that a geometric fix helps two-ray but not single-ray techniques) demonstrates why tool-building and empirical work belong together: a toolkit that makes it easy to implement and vary techniques reveals dependencies that monolithic implementations would obscure.

For fast VR selection, Gaze&Pinch remains the gold standard: highest throughput, lowest physical demand, most preferred. But its eye strain cost and moderate error rate mean it is not universally optimal. Parallax mitigation brings a ~ 0.30 bps throughput gain and 173 ms MT improvement to two-ray techniques, with the largest gains at small angular sizes, but does not close the gap to decoupled interaction. The best technique depends on whether the application prioritises speed, precision, or comfort, and that tradeoff is itself a finding that only a multi-metric evaluation can surface.

Appendix A

Questionnaires

This appendix documents the questionnaire instruments administered during the user study. All forms were implemented as Microsoft Forms and completed on a tablet device handed to the participant between technique blocks.

A.1 Preliminary Questionnaire

Administered once before the experiment began. Collected demographic information and pre-session baseline state.

A.1.1 Demographics

- **Age:** free-text integer
- **Gender:** Man / Woman / Non-binary / Prefer not to say / Other
- **Handedness:** Left / Right / Ambidextrous
- **Eye dominance:** My left eye / My right eye / Neither / Not sure
- **VR experience:** Never / A few times a year / Monthly / Weekly / Daily
- **Vision correction:** Not corrected / Corrected (eye glasses or contact lenses) / Other
- **Glasses inside headset:** Yes / No / N/A (uncorrected vision)

A.1.2 Pre-Session Sickness Baseline

Four items rated on a 5-point Likert scale (1 = Strongly Disagree, 5 = Strongly Agree):

- I feel dizzy
- I feel nauseous
- My eyes are tired
- I have a headache

A.2 NASA Task Load Index (NASA-TLX)

Administered immediately after each of the five technique blocks. Seven items, each rated on a 7-point scale (1 = Very Low, 7 = Very High):

1. **Mental Demand:** How much mental and perceptual activity was required? (e.g., thinking, deciding, remembering, looking, searching)
2. **Physical Demand:** How much physical activity was required? (e.g., pushing, pulling, turning, controlling, activating)
3. **Temporal Demand:** How much time pressure did you feel due to the pace at which the tasks occurred?
4. **Performance:** How successful do you think you were in accomplishing the goals of the task? (1 = Perfect, 7 = Failure)
5. **Effort:** How hard did you have to work (mentally and physically) to accomplish your level of performance?
6. **Frustration:** How insecure, discouraged, irritated, stressed, and annoyed did you feel during the task?
7. **Eye Strain:** How much did your eyes feel strained during the task? (added item, not part of standard NASA-TLX)

RTLX scoring. Raw TLX (RTLX) was computed as the unweighted mean of subscales 1–6 (Mental, Physical, Temporal, Performance, Effort, Frustration). The Performance subscale was *not* reverse-scored; lower values indicate better self-assessed performance. Eye Strain was treated as a separate seventh subscale and was not included in RTLX.

A.3 Technique Rankings

Administered once at the end of the session after all five technique blocks had been completed.

A.3.1 Forced-Choice Preference Ranking

Participants ranked all five techniques from 1 (most preferred) to 5 (least preferred). No ties were permitted. The five techniques were listed by name:

1. Gaze & Pinch
2. Head Crusher
3. Gaze & Finger
4. Head Crusher (Parallax Motion)
5. Gaze & Finger (Parallax Motion)

An open-ended follow-up question asked participants to explain why they ranked their chosen #1 technique first.

A.3.2 Post-Session Sickness

The same four sickness items from the preliminary questionnaire, re-administered on the same 5-point Likert scale. The difference (post minus pre) quantifies any change in simulator sickness symptoms attributable to the VR exposure.

A.4 Administration Order

Questionnaires were interleaved with the technique blocks in a fixed sequence synchronised with the Williams-design Latin square counterbalancing:

1. **Pre-experiment:** Preliminary Questionnaire (demographics + pre-sickness)
2. **After Gaze & Pinch:** NASA-TLX (7 items)
3. **After Head Crusher:** NASA-TLX (7 items)
4. **After Gaze & Finger:** NASA-TLX (7 items)
5. **After Head Crusher (Parallax Motion):** NASA-TLX (7 items)
6. **After Gaze & Finger (Parallax Motion):** NASA-TLX (7 items)
7. **End of session:** Technique Rankings + Post-sickness items

This per-technique administration captured workload impressions immediately after each technique, minimising recall bias.

A.5 Data Completeness

All 20 participants submitted complete NASA-TLX data across all five techniques with no missing subscales, and all 20 submitted complete technique rankings. All 20 participants also contributed complete behavioural data (trials, tracking, events).

Appendix B

Technique Ordinal Position Analysis

This appendix presents an exploratory analysis of how the ordinal position of each technique within the experiment session affects its measured throughput. Because the five techniques were counterbalanced across participants using a Latin-square design, each technique appeared in each ordinal position (1st through 5th) for exactly four participants ($20 \text{ sessions} \div 5 \text{ positions}$). This allows us to examine whether a technique’s performance depends on when in the session it was experienced, a potential practice or fatigue effect.

The analysis computes each participant’s mean effective throughput across all experiments (depth $\times$ amplitude $\times$ size conditions) within each technique, then groups these per-participant means by the technique’s ordinal position in that participant’s session. Figure B.1 shows the result: one panel per technique with mean throughput $\pm 1 \text{ SEM}$ at each ordinal position.

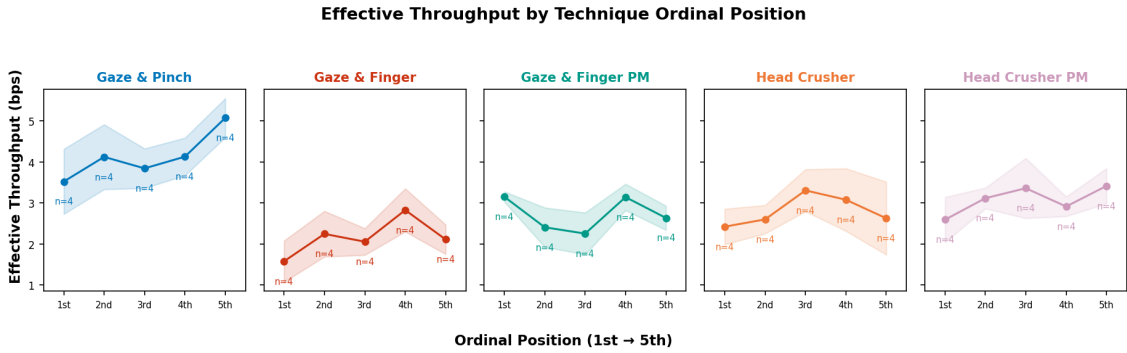


Figure B.1: Effective throughput (ISO 9241-9) by technique ordinal position. Each panel shows one technique’s mean throughput across participants who experienced that technique at each ordinal position (1st–5th). Error bands show $\pm 1 \text{ SEM}$.

Table B.1 reports the full numerical data for each technique–position combination, including the number of participants (N), mean effective throughput, standard deviation, and standard error of the mean.

Observation. No technique follows a clean monotonic trend across ordinal positions; all of them dip at least once. With $N = 4$ per cell, the group means are fragile: a single outlier can shift a position’s mean by over half a bit per second. The Gaze&Pinch numbers illustrate this clearly. The apparent $+1.55 \text{ bps}$ (44%) gain

Table B.1: Effective throughput by technique and ordinal position.

Technique	Position	N	Mean TP (bps)	SEM
Gaze & Pinch	1st	4	3.524	0.792
	2nd	4	4.124	0.790
	3rd	4	3.845	0.482
	4th	4	4.130	0.458
	5th	4	5.076	0.476
Gaze & Finger	1st	4	1.580	0.502
	2nd	4	2.248	0.556
	3rd	4	2.058	0.325
	4th	4	2.828	0.524
	5th	4	2.113	0.360
Gaze & Finger PM	1st	4	3.152	0.125
	2nd	4	2.409	0.479
	3rd	4	2.255	0.511
	4th	4	3.144	0.322
	5th	4	2.637	0.294
Head Crusher	1st	4	2.424	0.433
	2nd	4	2.601	0.346
	3rd	4	3.308	0.512
	4th	4	3.081	0.762
	5th	4	2.628	0.889
Head Crusher PM	1st	4	2.594	0.550
	2nd	4	3.118	0.254
	3rd	4	3.361	0.732
	4th	4	2.915	0.238
	5th	4	3.418	0.429

from 1st to 5th position is driven almost entirely by two participants: P18 at position 1 (mean 1.30 bps; pinch-impaired; their Gaze&Finger throughput was 2.35 bps, comfortably above average, so they were not simply a low performer) and P16 at position 5 (mean 6.41 bps, an exceptional pinch user). Excluding just these two, the 1st-to-5th difference collapses to +0.37 bps (9%), well within the noise floor. The other techniques show no trend worth chasing: HeadCrusher PM gains +0.82 bps (32%), Gaze&Finger gains +0.53 bps (34%), HeadCrusher stays flat (+0.20 bps, 8%), and Gaze&Finger PM edges downward (−0.52 bps, −16%). With the Latin-square design keeping each technique equally distributed across positions, and with no technique showing a consistent direction, there is no evidence that practice or fatigue systematically biased the technique comparisons. The counterbalanced design did its job.

References

- [1] O. W. Bertelsen and S. Bødker, “Activity theory,” in *HCI Models, Theories, and Frameworks: Toward a Multidisciplinary Science*, J. M. Carroll, Ed., San Francisco, CA: Morgan Kaufmann, 2003, pp. 291–324, ISBN: 9781558608085. DOI: 10.1016/B978-155860808-5/50011-3
- [2] J. Grudin, “The computer reaches out: The historical continuity of interface design,” in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems: Empowering People*, ser. CHI '90, ACM Press, 1990, pp. 261–268. DOI: 10.1145/97243.97284
- [3] J. Vardalas, “From DATAR to the FP-6000 computer: Technological change in a canadian industrial context,” *IEEE Annals of the History of Computing*, vol. 16, no. 2, 1994.
- [4] I. E. Sutherland, “A head-mounted three dimensional display,” in *Proceedings of the December 9–11, 1968, Fall Joint Computer Conference, Part I*, ser. AFIPS '68 (Fall, part I), Introduced the head-mounted display later nicknamed the “Sword of Damocles”, New York, NY, USA: ACM, 1968, pp. 757–764.
- [5] Google DeepMind. “Reimagining the mouse pointer for the AI era,” Accessed: Jun. 14, 2026. [Online]. Available: <https://deepmind.google/blog/ai-pointer/>
- [6] K. Pfeuffer, B. Mayer, D. Mardanbegi, and H. Gellersen, “Gaze + pinch interaction in virtual reality,” in *Proceedings of the 5th Symposium on Spatial User Interaction*, ser. SUI '17, Brighton, United Kingdom: Association for Computing Machinery, 2017, pp. 99–108, ISBN: 9781450354868. DOI: 10.1145/3131277.3132180 [Online]. Available: <https://doi.org/10.1145/3131277.3132180>
- [7] Apple, *Apple vision pro*, Official product page. Accessed 2026-06-13, 2026. [Online]. Available: <https://www.apple.com/apple-vision-pro/>
- [8] G. A. Lee, G. J. Kim, and M. Billinghurst, “Immersive authoring: What you experience is what you get (wyxiwyg),” *Commun. ACM*, vol. 48, no. 7, pp. 76–81, Jul. 2005, ISSN: 0001-0782. DOI: 10.1145/1070838.1070840 [Online]. Available: <https://doi.org/10.1145/1070838.1070840>
- [9] L. Zhang and S. Oney, *Flowmatic: An immersive authoring tool for creating interactive scenes in virtual reality*, 2020. DOI: <https://doi.org/10.1145/3379337.3415824> [Online]. Available: <https://doi.org/10.1145/3379337.3415824>

-
- [10] U. Schultheis, J. Jerald, F. Toledo, A. Yoganandan, and P. Mlyniec, “Comparison of a two-handed interface to a wand interface and a mouse interface for fundamental 3d tasks,” in *2012 IEEE Symposium on 3D User Interfaces (3DUI)*, 2012, pp. 117–124. DOI: 10.1109/3DUI.2012.6184195
 - [11] D. Ledo, S. Houben, J. Vermeulen, N. Marquardt, L. Oehlberg, and S. Greenberg, “Evaluation strategies for HCI toolkit research,” in *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems (CHI '18)*, ACM, 2018, pp. 1–17. DOI: 10.1145/3173574.3173610
 - [12] U. Wagner, M. N. Lystbæk, P. Manakhov, J. E. Grønbaek, K. Pfeuffer, and H. Gellersen, “A Fitts’ Law study of gaze-hand alignment for selection in 3D user interfaces,” in *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems (CHI '23)*, Empirical comparison of HeadCrusher, Handray, Gaze&Pinch, Gaze&Finger, Gaze&Handray, 2023, 252:1–252:15. DOI: 10.1145/3544548.3581423
 - [13] L. Zerbin, “Selective removal of parallax-induced double vision in virtual reality,” Prerelease, 2026.
 - [14] Meta, *Meta quest 3*, Official product page. Accessed 2026-06-13, 2026. [Online]. Available: <https://www.meta.com/quest/quest-3/>
 - [15] B. MacIntyre, M. Gandy, S. P. Dow, and J. D. Bolter, *Dart*, 2004. DOI: <https://doi.org/10.1145/1029632.1029669> [Online]. Available: <https://doi.org/10.1145/1029632.1029669>
 - [16] Backup, Computer History Museum, *Macromedia director*, Historical reference for Macromedia Director as an early multimedia authoring platform., 2008. [Online]. Available: <https://www.computerhistory.org/collections/catalog/102717326>
 - [17] C. Floyd, “A systematic look at prototyping,” in *Approaches to Prototyping*, R. Budde, K. Kuhlenkamp, L. Mathiassen, and H. Zullighoven, Eds., Berlin, Heidelberg: Springer, 1984, pp. 1–18. DOI: 10.1007/978-3-642-69796-8_1
 - [18] Image-Line, *Fl studio – music production software*, Official product page. Accessed 2026-06-13, 2026. [Online]. Available: <https://www.image-line.com/fl-studio/>
 - [19] M. Nebeling, J. Nebeling, A. Yu, and R. Rumble, *Protoar*, 2018. DOI: <https://doi.org/10.1145/3173574.3173927> [Online]. Available: <https://doi.org/10.1145/3173574.3173927>
 - [20] M. Nebeling and K. Madier, *360proto*, 2019. DOI: <https://doi.org/10.1145/3290605.3300826> [Online]. Available: <https://doi.org/10.1145/3290605.3300826>
 - [21] M. Cavallo and A. G. Forbes, *Cave-ar: A vr authoring system to interactively design, simulate, and debug multi-user ar experiences*, 2019. DOI: <https://doi.org/10.1109/vr.2019.8798148> [Online]. Available: <https://doi.org/10.1109/vr.2019.8798148>
 - [22] M. Nebeling et al., *Xrdirector: A role-based collaborative immersive authoring system*, 2020. DOI: <https://doi.org/10.1145/3313831.3376637> [Online]. Available: <https://doi.org/10.1145/3313831.3376637>

-
- [23] J. M. E. Belo et al., *Auit – the adaptive user interfaces toolkit for designing xr applications*, 2022. DOI: <https://doi.org/10.1145/3526113.3545651> [Online]. Available: <https://doi.org/10.1145/3526113.3545651>
- [24] R. Suzuki, R. H. Kazi, L.-Y. Wei, S. DiVerdi, W. Li, and D. Leithinger, “Realitysketch: Embedding responsive graphics and visualizations in ar through dynamic sketching,” *arXiv (Cornell University)*, 2020. DOI: <https://doi.org/10.48550/arxiv.2008.08688> [Online]. Available: <http://arxiv.org/abs/2008.08688>
- [25] J. Liao, K. Van, Z. Xia, and R. Suzuki, “Realityeffects: Augmenting 3d volumetric videos with object-centric annotation and dynamic visual effects,” *Designing Interactive Systems Conference*, 2024. DOI: <https://doi.org/10.1145/3643834.3661631> [Online]. Available: <https://doi.org/10.1145/3643834.3661631>
- [26] M. Nebeling and M. Speicher, *The trouble with augmented reality/virtual reality authoring tools*, 2018. DOI: <https://doi.org/10.1109/ismar-adjunct.2018.00098> [Online]. Available: <https://doi.org/10.1109/ismar-adjunct.2018.00098>
- [27] M. Haller, E. Stauder, and J. Zauner, “Amire-es: Authoring mixed reality once, run it anywhere,” *Proceedings of the Third IEEE and ACM International Symposium on Mixed and Augmented Reality*, 2005. [Online]. Available: https://www.researchgate.net/publication/228855505_AMIRE-ES_Authoring_Mixed_Reality_once_run_it_anywhere
- [28] U. Gruenefeld et al., “Vrception: Rapid prototyping of cross-reality systems in virtual reality,” *CHI Conference on Human Factors in Computing Systems*, 2022. DOI: <https://doi.org/10.1145/3491102.3501821> [Online]. Available: <https://doi.org/10.1145/3491102.3501821>
- [29] N. Ashtari, A. Bunt, J. McGrenere, M. Nebeling, and P. K. Chilana, *Creating augmented and virtual reality applications: Current practices, challenges, and opportunities*, 2020. DOI: <https://doi.org/10.1145/3313831.3376722> [Online]. Available: <https://doi.org/10.1145/3313831.3376722>
- [30] G. Freitas, M. S. Pinho, M. S. Silveira, and F. Maurer, *A systematic review of rapid prototyping tools for augmented reality*, 2020. DOI: <https://doi.org/10.1109/svr51698.2020.00041> [Online]. Available: <https://doi.org/10.1109/svr51698.2020.00041>
- [31] Unity Technologies, *Unity: Develop, deploy, and grow*, Official platform overview. Accessed 2026-06-13, 2026. [Online]. Available: <https://unity.com/>
- [32] G. Leiva, J. E. S. Grønbaek, C. N. Klokmoose, C. Nguyen, R. H. Kazi, and P. Asente, *Rapido: Prototyping interactive ar experiences through programming by demonstration*, 2021. DOI: <https://doi.org/10.1145/3472749.3474774> [Online]. Available: <https://doi.org/10.1145/3472749.3474774>

- [33] K. Monteiro, R. Vatsal, N. Chulpongsatorn, A. Parnami, and R. Suzuki, *Teachable reality: Prototyping tangible augmented reality with everyday objects by leveraging interactive machine teaching*, 2023. DOI: <https://doi.org/10.1145/3544548.3581449> [Online]. Available: <https://doi.org/10.1145/3544548.3581449>
- [34] A. Sayara, E. Chen, C. Nguyen, R. Xiao, and D. Yoon, *Gesturecanvas: A programming by demonstration system for prototyping compound freehand interaction in vr*, 2023. DOI: <https://doi.org/10.1145/3586183.3606736> [Online]. Available: <https://doi.org/10.1145/3586183.3606736>
- [35] A. Cai, C. Ardayfio, A. Nguyen, T. Lin, and E. L. Glassman, “Atomxr: Streamlined xr prototyping with natural language and immersive physical interaction,” *arXiv (Cornell University)*, 2023. DOI: <https://doi.org/10.48550/arxiv.2311.11238> [Online]. Available: <http://arxiv.org/abs/2311.11238>
- [36] R. J. Jacob, L. Deligiannidis, and S. Morrison, “A software model and specification language for non-wimp user interfaces,” *ACM Transactions on Computer-Human Interaction*, 1999. DOI: <https://doi.org/10.1145/310641.310642> [Online]. Available: <https://doi.org/10.1145/310641.310642>
- [37] P. Figueroa, M. Green, and B. Watson, “A framework for 3d interaction techniques,” *Proceedings of SPIE*, 2001. [Online]. Available: <https://arxiv.org/pdf/2507.02254>
- [38] J. M. E. Belo et al., “AUIT – the adaptive user interfaces toolkit for designing XR applications,” in *Proceedings of the 35th Annual ACM Symposium on User Interface Software and Technology (UIST ’22)*, ACM, 2022, 48:1–48:16. DOI: [10.1145/3526113.3545651](https://doi.org/10.1145/3526113.3545651)
- [39] K. John, Y. Li, and H. Seifi, *Adaptics: A toolkit for creative design and integration of real-time adaptive mid-air ultrasound tactons*, 2024. DOI: <https://doi.org/10.1145/3613904.3642090> [Online]. Available: <https://doi.org/10.1145/3613904.3642090>
- [40] R. Lunding, S. Hubenschmid, T. Feuchtner, and K. Grønbæk, “Arthur: Authoring human-robot collaboration processes with augmented reality using hybrid user interfaces,” *Virtual Reality*, 2025. DOI: <https://doi.org/10.1007/s10055-025-01149-6> [Online]. Available: <https://doi.org/10.1007/s10055-025-01149-6>
- [41] A. Plopski, T. Hirzle, N. Norouzi, L. Qian, G. Bruder, and T. Langlotz, “The eye in extended reality: A survey on gaze interaction and eye tracking in head-worn extended reality,” *ACM Comput. Surv.*, vol. 55, no. 3, Mar. 2022, ISSN: 0360-0300. DOI: [10.1145/3491207](https://doi.org/10.1145/3491207) [Online]. Available: <https://doi.org/10.1145/3491207>
- [42] J. S. Pierce, A. S. Forsberg, M. J. Conway, S. Hong, R. C. Zeleznik, and M. R. Mine, “Image plane interaction techniques in 3D immersive environments,” in *Proceedings of the 1997 Symposium on Interactive 3D Graphics (I3D ’97)*, Introduced the original HeadCrusher image-plane framing technique, 1997, pp. 39–43. DOI: [10.1145/253284.253303](https://doi.org/10.1145/253284.253303)

-
- [43] M. N. Lystbæk, P. Rosenberg, K. Pfeuffer, J. E. Grønbæk, and H. Gellersen, “Gaze-hand alignment: Combining eye gaze and mid-air pointing for interacting with menus in augmented reality,” *Proc. ACM Hum.-Comput. Interact.*, vol. 6, no. ETRA, May 2022. DOI: 10.1145/3530886 [Online]. Available: <https://doi.org/10.1145/3530886>
- [44] D. Bowman, C. Wingrave, J. Campbell, and V. Ly, “Using pinch gloves for both natural and abstract interaction techniques in virtual environments,” in *Proceedings of HCI International*, 2001, pp. 629–633.
- [45] D. Wolf, J. Gugenheimer, M. Combosch, and E. Rukzio, “Understanding the heisenberg effect of spatial interaction: A selection induced error for spatially tracked input devices,” in *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems*, ser. CHI ’20, Honolulu, HI, USA: Association for Computing Machinery, 2020, pp. 1–10, ISBN: 9781450367080. DOI: 10.1145/3313831.3376876 [Online]. Available: <https://doi.org/10.1145/3313831.3376876>
- [46] R. J. K. Jacob, “What you look at is what you get: Eye movement-based interaction techniques,” in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, ser. CHI ’90, Seattle, Washington, USA: Association for Computing Machinery, 1990, pp. 11–18, ISBN: 0201509326. DOI: 10.1145/97243.97246 [Online]. Available: <https://doi.org/10.1145/97243.97246>
- [47] J. D. Hincapié-Ramos, X. Guo, P. Moghadasian, and P. Irani, “Consumed endurance: A metric to quantify arm fatigue of mid-air interactions,” in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, ser. CHI ’14, New York, NY, USA: ACM, 2014, pp. 1063–1072. DOI: 10.1145/2556288.2557130
- [48] R. I. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, Second. Cambridge University Press, ISBN: 0521540518, 2004.
- [49] I. P. Howard and B. J. Rogers, *Seeing in Depth: Volume 1: Basic Mechanics/Volume 2: Depth Perception 2-Volume Set*. Oxford University Press, Feb. 2008, ISBN: 9780195367607. DOI: 10.1093/acprof:oso/9780195367607.001.0001 [Online]. Available: <https://doi.org/10.1093/acprof:oso/9780195367607.001.0001>
- [50] P. M. Fitts, “The information capacity of the human motor system in controlling the amplitude of movement,” *Journal of Experimental Psychology*, vol. 47, no. 6, pp. 381–391, 1954. DOI: 10.1037/h0055392
- [51] C. E. Shannon, “A mathematical theory of communication,” *Bell System Technical Journal*, vol. 27, no. 3, pp. 379–423, 1948. DOI: <https://doi.org/10.1002/j.1538-7305.1948.tb01338.x> [Online]. Available: <https://onlinelibrary.wiley.com/doi/abs/10.1002/j.1538-7305.1948.tb01338.x>
- [52] I. S. MacKenzie, “Fitts’ law,” in *The Wiley Handbook of Human Computer Interaction*, K. L. Norman and J. Kirakowski, Eds., vol. 1, John Wiley & Sons, 2018, pp. 349–370. DOI: 10.1002/9781118976005.ch17

- [53] I. S. MacKenzie, “Fitts’ law as a research and design tool in human-computer interaction,” *Human-Computer Interaction*, vol. 7, no. 1, pp. 91–139, 1992. DOI: 10.1207/s15327051hci0701_3
- [54] A. T. Welford, *Fundamentals of Skill*. Methuen, 1968.
- [55] “Ergonomic requirements for office work with visual display terminals (vdt) – part 9: Requirements for non-keyboard input devices,” International Organization for Standardization, Standard, 2000.
- [56] “Ergonomics of human-system interaction – part 411: Evaluation methods for the design of physical input devices,” International Organization for Standardization, Standard, 2012.
- [57] S. K. Card, W. K. English, and B. J. Burr, “Evaluation of mouse, rate-controlled isometric joystick, step keys, and text keys for text selection on a CRT,” *Ergonomics*, vol. 21, no. 8, pp. 601–613, 1978. DOI: 10.1080/00140137808931762
- [58] R. W. Soukoreff and I. S. MacKenzie, “Towards a standard for pointing device evaluation, perspectives on 27 years of Fitts’ Law research in HCI,” *International Journal of Human-Computer Studies*, vol. 61, no. 6, pp. 751–789, 2004. DOI: 10.1016/j.ijhcs.2004.09.001
- [59] R. Kopper, D. A. Bowman, M. G. Silva, and R. P. McMahan, “A human motor behavior model for distal pointing tasks,” *International Journal of Human-Computer Studies*, vol. 68, no. 10, pp. 603–615, 2010. DOI: 10.1016/j.ijhcs.2010.05.001
- [60] L. Lane, F. Lu, S. Davari, R. J. Teather, and D. A. Bowman, “Revisiting performance models of distal pointing tasks in virtual reality,” in *Proceedings of the 2023 IEEE Conference on Virtual Reality and 3D User Interfaces (VR)*, 2023, pp. 231–241. DOI: 10.1109/VR55154.2023.00035
- [61] A. U. Batmaz, M. Amini, W. Stuerzlinger, and R. J. Teather, “An effective angular formulation of Fitts’ Law for 3D pointing in virtual reality,” in *Unpublished*, Angular effective Fitts’ Law — direct angular effective width computation. Marked "To appear" in original thesis; still no published venue found as of June 2026. Verify before submission., 2025.
- [62] J. Bergström, T.-S. Dalsgaard, J. Alexander, and K. Hornbæk, “How to evaluate object selection and manipulation in VR? guidelines from 20 years of studies,” in *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems (CHI ’21)*, ACM, 2021. DOI: 10.1145/3411764.3445193
- [63] J. O. Wobbrock, L. Findlater, D. Gergle, and J. J. Higgins, “The aligned rank transform for nonparametric factorial analyses using only anova procedures,” in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, ser. CHI ’11, New York, NY, USA: ACM, 2011, pp. 143–146. DOI: 10.1145/1978942.1978963
- [64] R. Bakeman, “Recommended effect size statistics for repeated measures designs,” *Behavior Research Methods*, vol. 37, no. 3, pp. 379–384, 2005. DOI: 10.3758/BF03192707

- [65] M. Amini, W. Stuerzlinger, R. J. Teather, and A. U. Batmaz, “A systematic review of fitts’ law in 3d extended reality,” in *Conference on Human Factors in Computing Systems*, ser. CHI ’25, Apr. 2025. DOI: 10.1145/3706598.3713623
- [66] B. A. Myers, S. E. Hudson, and R. Pausch, “Past, present, and future of user interface software tools,” *ACM Transactions on Computer-Human Interaction*, vol. 7, no. 1, pp. 3–28, 2000. DOI: 10.1145/344949.344959
- [67] T. Munzner, *Visualization Analysis and Design* (AK Peters Visualization Series). AK Peters/CRC Press, 2014.
- [68] B. Shneiderman, “Direct manipulation: A step beyond programming languages,” *IEEE Computer*, vol. 16, no. 8, pp. 57–69, 1983. DOI: 10.1109/MC.1983.1654471
- [69] D. A. Bowman and L. F. Hodges, “Formalizing the design, evaluation, and application of interaction techniques for immersive virtual environments,” *Journal of Visual Languages and Computing*, vol. 10, no. 1, pp. 37–53, 1999. DOI: 10.1006/jvlc.1998.0111
- [70] Meta Platforms, Inc., *Design principles for Meta Quest*, <https://developers.meta.com/horizon/resources/design-principles/>, Retrieved 2025–2026, 2025.
- [71] S. K. Card, T. P. Moran, and A. Newell, *The Psychology of Human-Computer Interaction*. Hillsdale, NJ: Lawrence Erlbaum Associates, 1983.
- [72] B. Shneiderman, C. Plaisant, M. Cohen, S. Jacobs, N. Elmqvist, and N. Diakopoulos, *Designing the User Interface: Strategies for Effective Human-Computer Interaction*, 6th. Pearson, 2016.
- [73] J. Collins, P. Gurumurthy, E. J. Gonzalez, and M. Gonzalez-Franco, “Sticky and magnetic: Evaluating error correction and user adaptation in gaze and pinch interaction,” in *Proceedings of the Extended Abstracts of the 2026 CHI Conference on Human Factors in Computing Systems*, ser. CHI EA ’26, New York, NY, USA: Association for Computing Machinery, 2026, ISBN: 9798400722813. DOI: 10.1145/3772363.3798839 [Online]. Available: <https://doi.org/10.1145/3772363.3798839>
- [74] E. J. Williams, “Experimental Designs Balanced for the Estimation of Residual Effects of Treatments,” *Australian Journal of Scientific Research A Physical Sciences*, vol. 2, pp. 149–168, Jun. 1949. DOI: 10.1071/CH9490149
- [75] Meta Platforms, Inc., *Meta XR all-in-one SDK*, <https://assetstore.unity.com/packages/tools/integration/meta-xr-all-in-one-sdk-269657>, Version 83.0.4. Retrieved 2025–2026, 2025.
- [76] F. J. Roethlisberger and W. J. Dickson, *Management and the Worker: An Account of a Research Program Conducted by the Western Electric Company, Hawthorne Works, Chicago* (The early sociology of management and organizations), Reprinted. London, UK: Routledge, 2003, ISBN: 9780415279871.
- [77] S. G. Hart, “NASA-Task Load Index (NASA-TLX); 20 Years Later,” in *Proceedings of the Human Factors and Ergonomics Society Annual Meeting*, NASA-TLX reference; establishes ordinal nature of subjective workload ratings, vol. 50, 2006, pp. 904–908. DOI: 10.1177/154193120605000909